

Computer Organization and Architecture

Module 4

Design of Control Unit

Department of Computer Science and Engineering

IIT Kharagpur

Design of Control Unit

Instructions

- Instructions are stored in main memory.
- Program Counter (*PC*) points to the next instruction.
 - MIPS32 instructions are 4 bytes (32 bits) long.
 - All instructions starts from an address that is multiple of 4 (last 2 bits 00).
 - Normally, *PC* is incremented by 4 to point to the next instruction.
 - For branch, *PC* is loaded with the address of the target instruction.

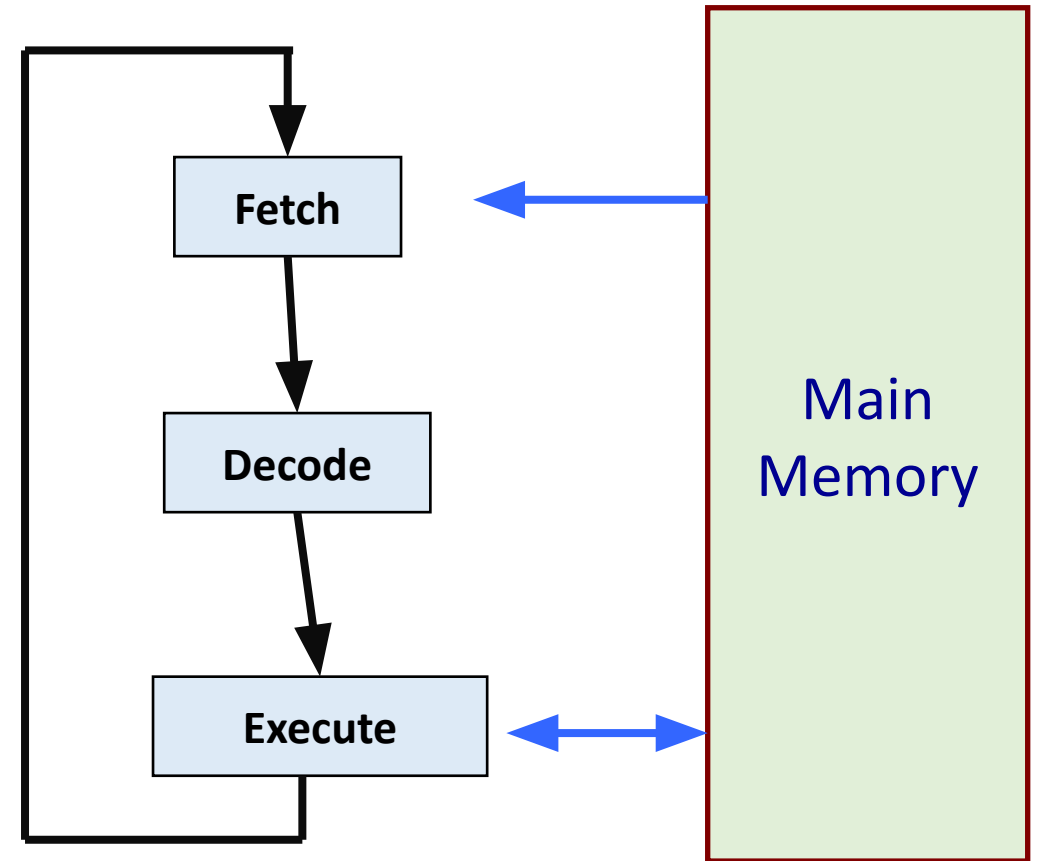
12					
8					
4	instruction word				
0	instruction word				

Addressing a Byte in Memory

- Each byte in memory has a unique address.
 - Memory is said to be *byte addressable*.
- Typically the instructions are of 4 bytes, hence the instruction memory is addressed in terms of 4 bytes (word length = 32 bits).
- When an instruction is executed, PC is *incremented by 4* to point to the next instruction.
 - In MIPS32, words are byte aligned.
 - Every word (including instruction) starts from a memory address that is some multiple of 4 (i.e., last two bits are `00').

How an instruction Gets Executed?

```
repeat forever
    // till power off or
    // system failure
{
    Fetch instruction
    Decode instruction
    Execute instruction
}
```



The Fetch-Execute Cycle

- 1) Fetch the next instruction from memory.
- 2) Decode the instruction.
- 3) Execute the operation.
 - Get data from memory if needed (data not available in the processor).
 - Perform the required operation on the data.
 - May also store the result back in memory or register.

Example: Add R1, R2

Address	Instruction
1000	ADD R1, R2
1004	MUL R3, R4

- a) PC = 1000
- b) MAR = 1000
- c) PC = PC + 4 = 1004
- d) MDR = "ADD R1, R2"
- e) IR = "ADD R1, R2"

(Decode and finally execute)

- f) R1 = R1 + R2

May require one or more steps depending on the target architecture.

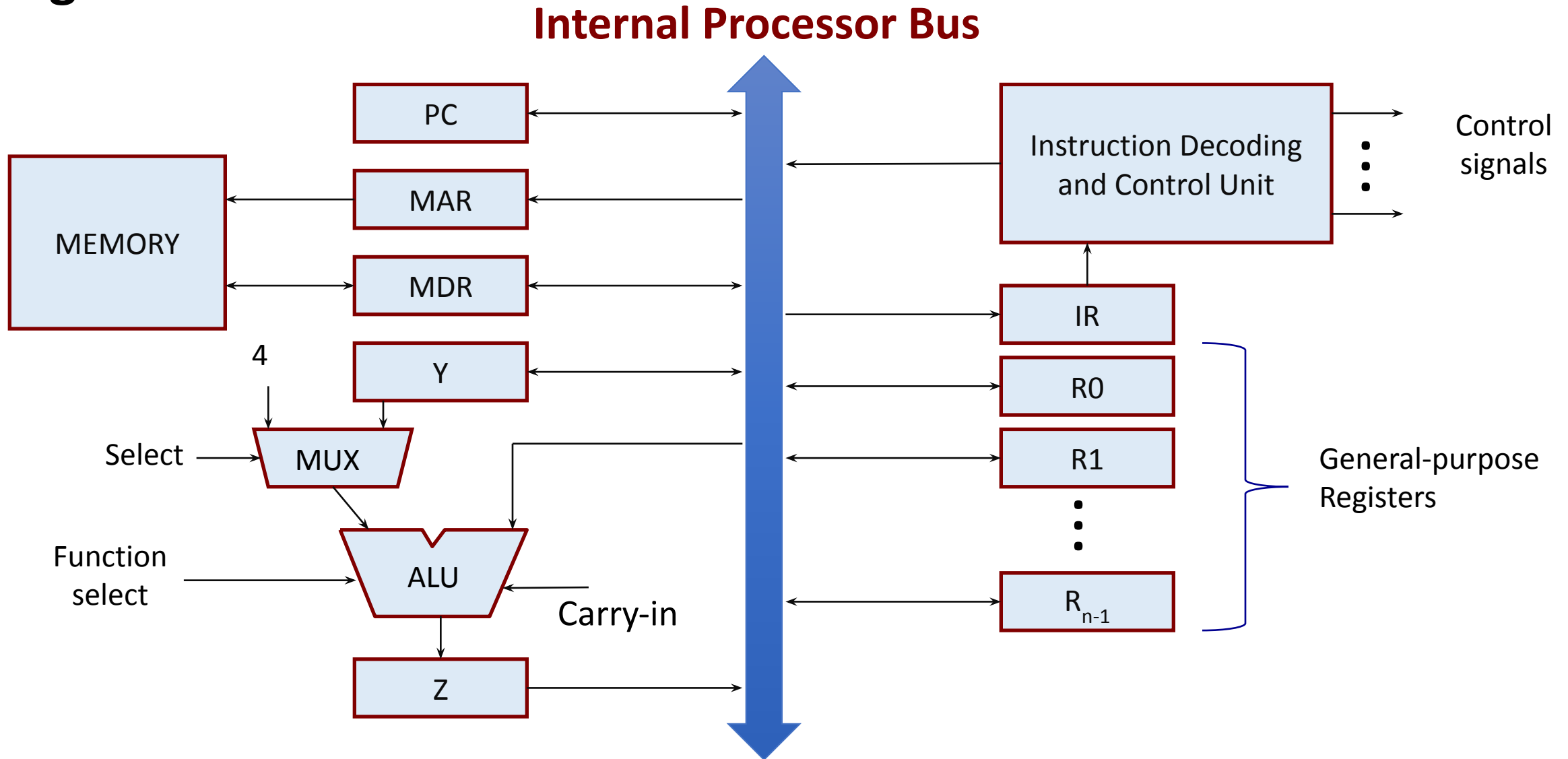
Requirement for Instruction Execution

- The necessary registers must be present.
- The internal organization of the registers must be known.
- The data path must be known.
- For instruction execution, a number of *micro-operations* are carried out on the data path.
 - An instruction consists of several micro-operations or micro-instructions.
 - Typically involves movement of data.

Kinds of Data Movement

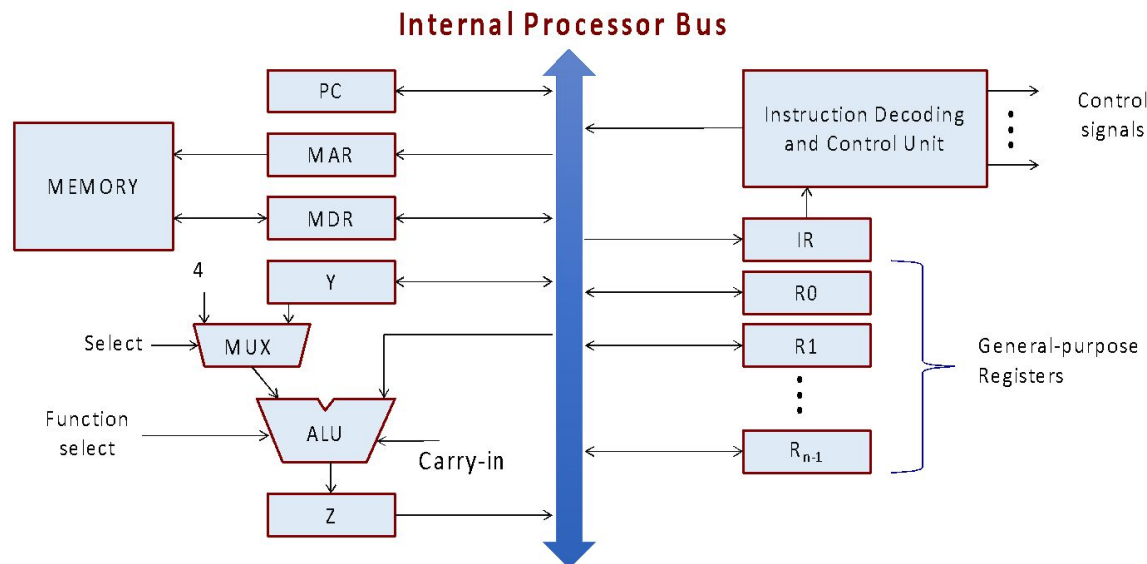
- Broadly three types:
 - a) Register to Register
 - b) Register to ALU
 - c) ALU to Register
- Data movement is supported in the data path by:
 - The Registers
 - The Bus (single or multiple)
 - The ALU temporary Register (*Y* and *Z*)

Single Bus Organization

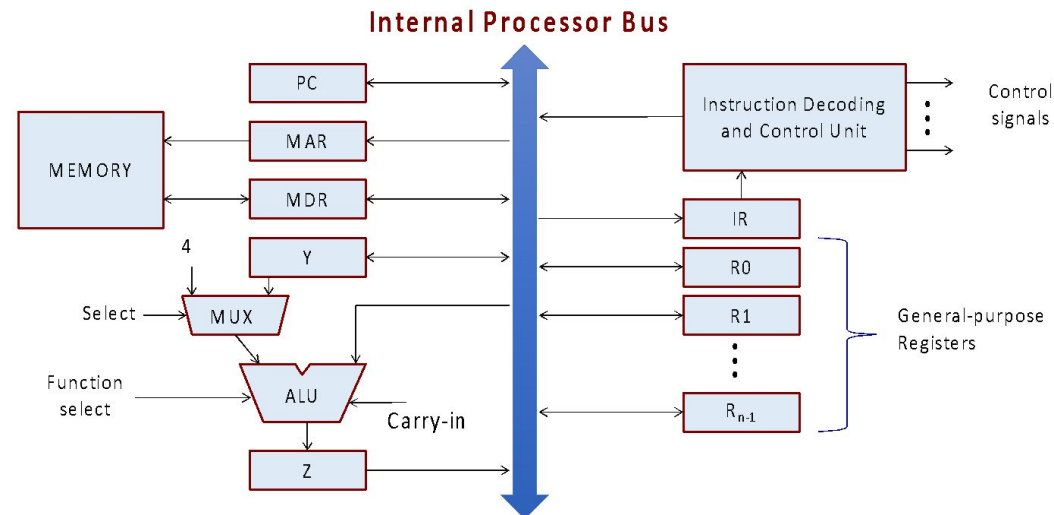


Single Internal Bus Organization

- All the registers and various units are connected using a single internal bus.
- Registers R_0-R_{n-1} are general-purpose registers used for various purposes.
- Registers Y and Z are used for storing intermediate results and never used by instructions explicitly.
- The multiplexer selects either a *constant 4* or output of register Y .
 - When PC is incremented, a constant 4 has to be added.



- The instruction decoder and control unit is responsible for performing the actions specified by the instruction loaded into *IR*.
- The decoder generates all the control signals in the proper sequence required to execute the instruction specified by the *IR*.
- The registers, the ALU and the interconnecting bus are collectively referred to as the *data path*.
- The control unit that generates the control signals in proper sequence is referred to as the *control path*.



Kinds of Operations

- Transfer of data from one register to another.

MOVE R1, R2 // R1 = R2

- Perform arithmetic or logic operation on data loaded into registers.

ADD R1, R2 // R1 = R1 + R2

- Fetch the content of a memory location and load it into a register.

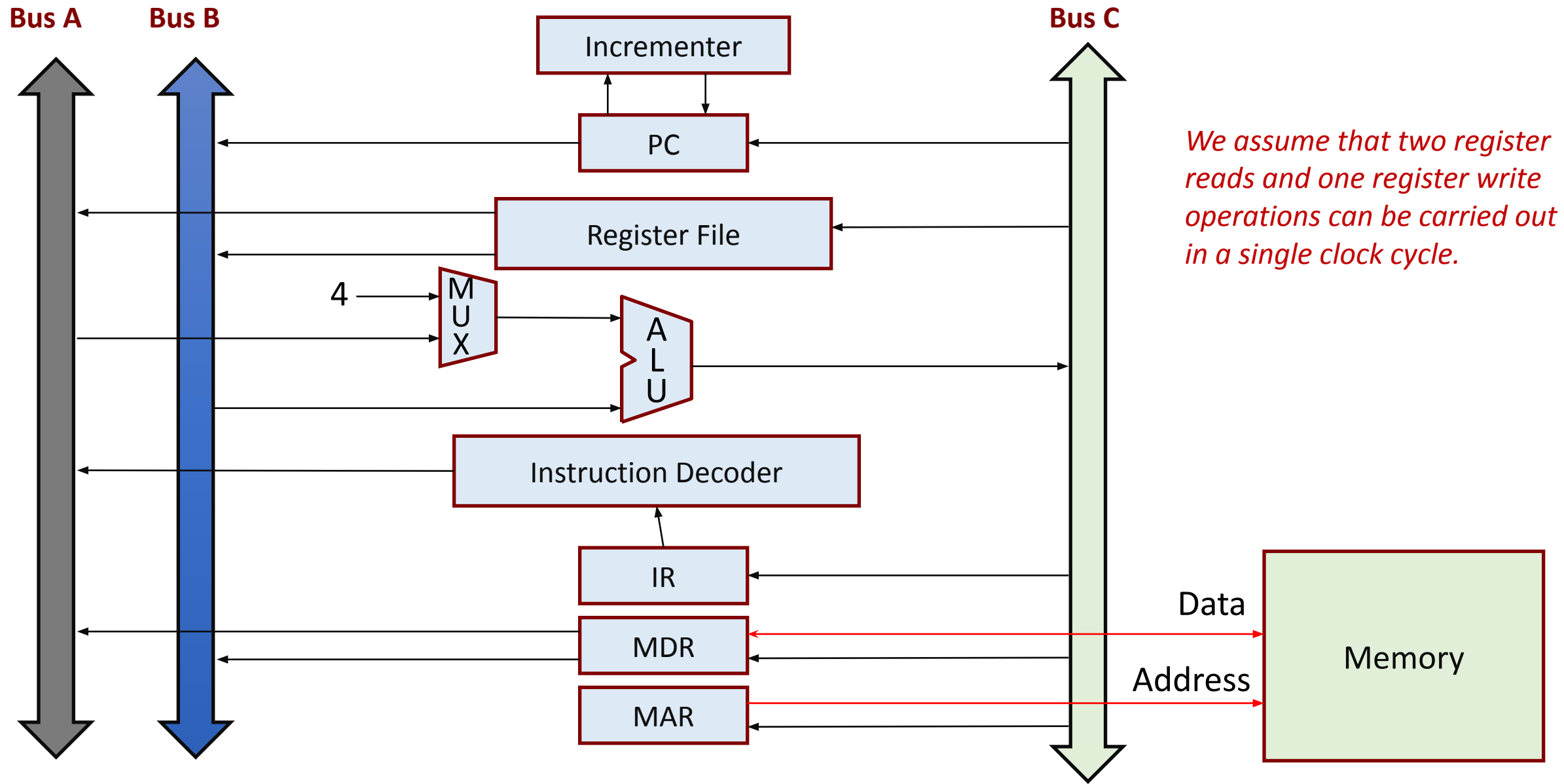
LOAD R1, LOCA // R1 = Mem[LOCA]

- Store a word of data from a register into a given memory location.

STOR LOCA, R1 // Mem[LOCA] = R1

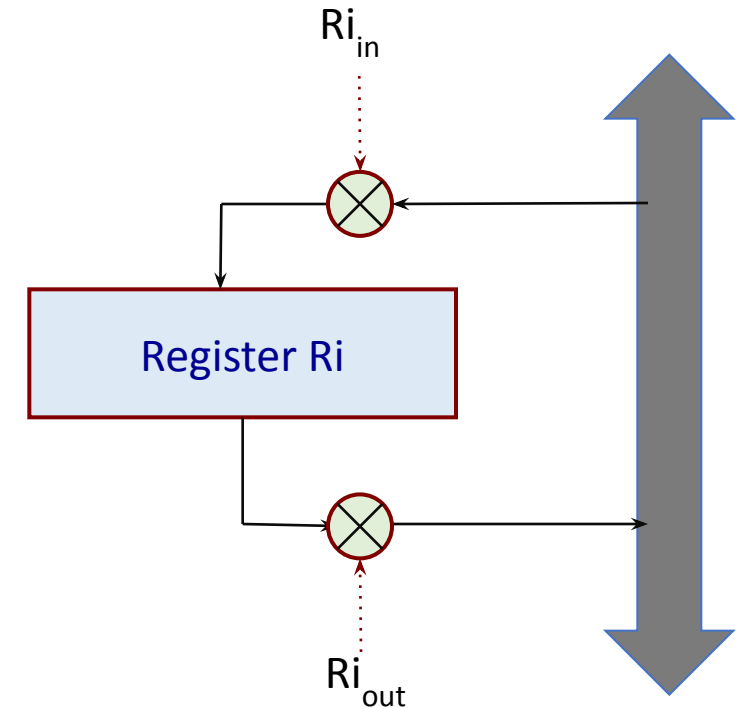
Three Bus Organization

- A typical 3-bus architecture for the processor datapath is shown in the next slide.
 - The 3-bus organization is internal to the CPU.
 - Three buses allow three parallel data transfer operations to be carried out.
- Less number of cycles required to execute an instruction compared to single bus organization.

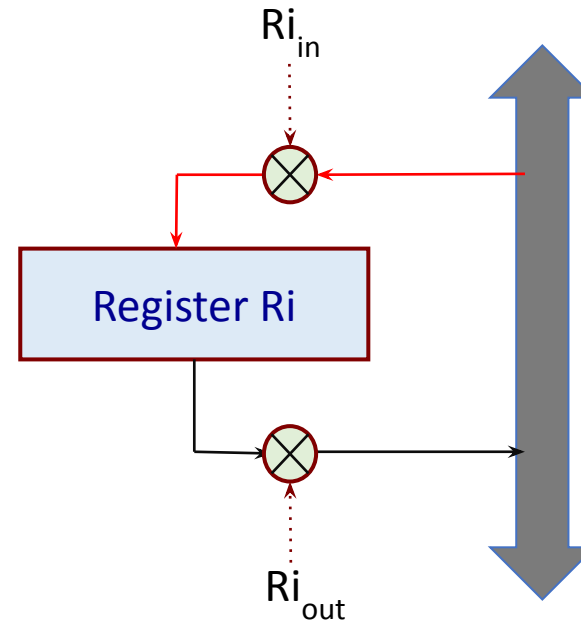


Organization of a Register

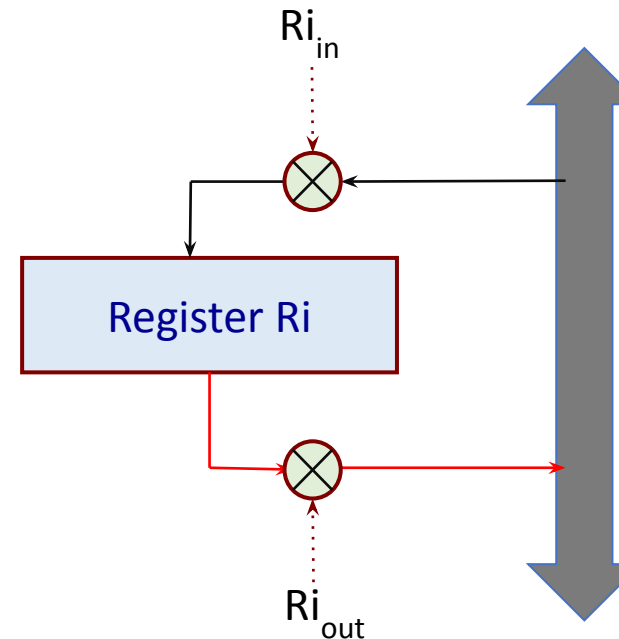
- A register is used for temporary storage of data (parallel-in, parallel-out, etc.).
- A register R_i typically has two control signals.
 - $R_{i_{in}}$: used to load the register with data from the bus.
 - $R_{i_{out}}$: used to place the data stored in the register on the bus.
- Input and output lines of the register R_i are connected to the bus via controlled switches.
 - If $R_{i_{out}}$ is not selected, the register outputs are set in the high impedance state.



- When ($Ri_{in} = 1$), the data available on bus is loaded into Ri .



- When ($Ri_{out} = 1$), the data from register Ri are placed on the bus.



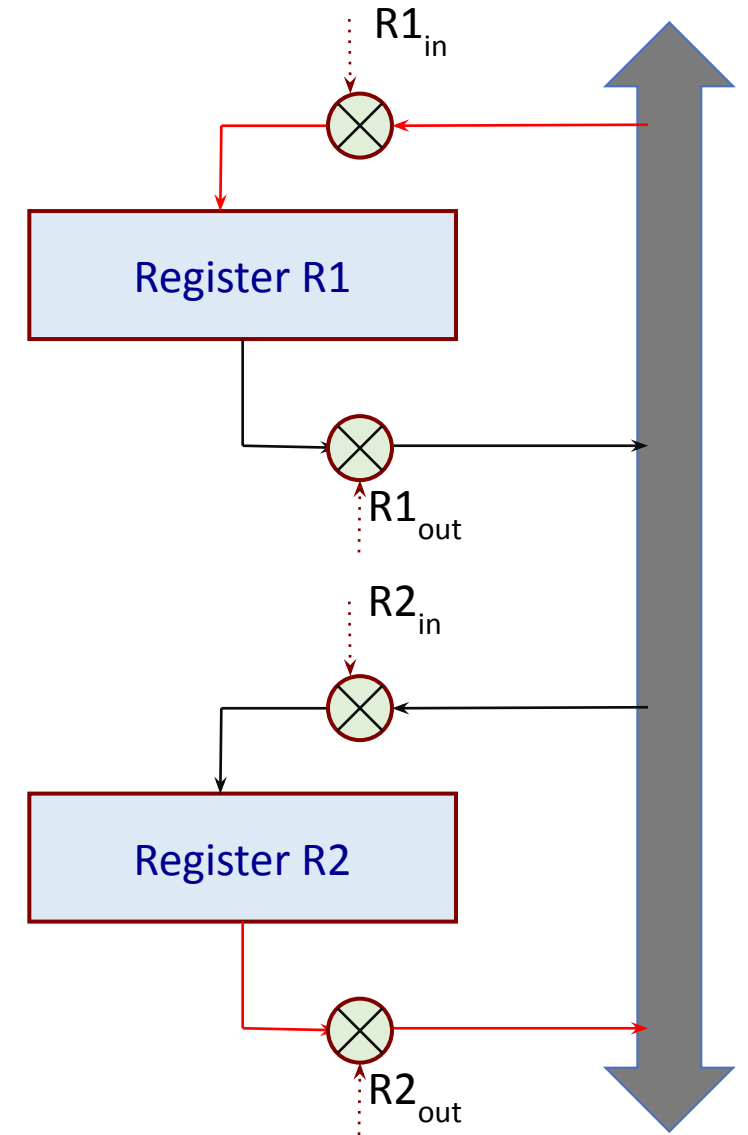
Register Transfer

MOVE R1, R2 // $R1 \leftarrow R2$

- Enable the output of **R2** by setting $R2_{out} = 1$.
- Enable the input of register **R1** by setting $R1_{in} = 1$.
- All operations are performed in synchronism with the processor clock.
 - The control signals are asserted at the start of the clock cycle.
 - After data transfer the control signals will return to 0.
- We write as $T1: R2_{out}, R1_{in}$

Time Step

Control Signals



ALU Operation

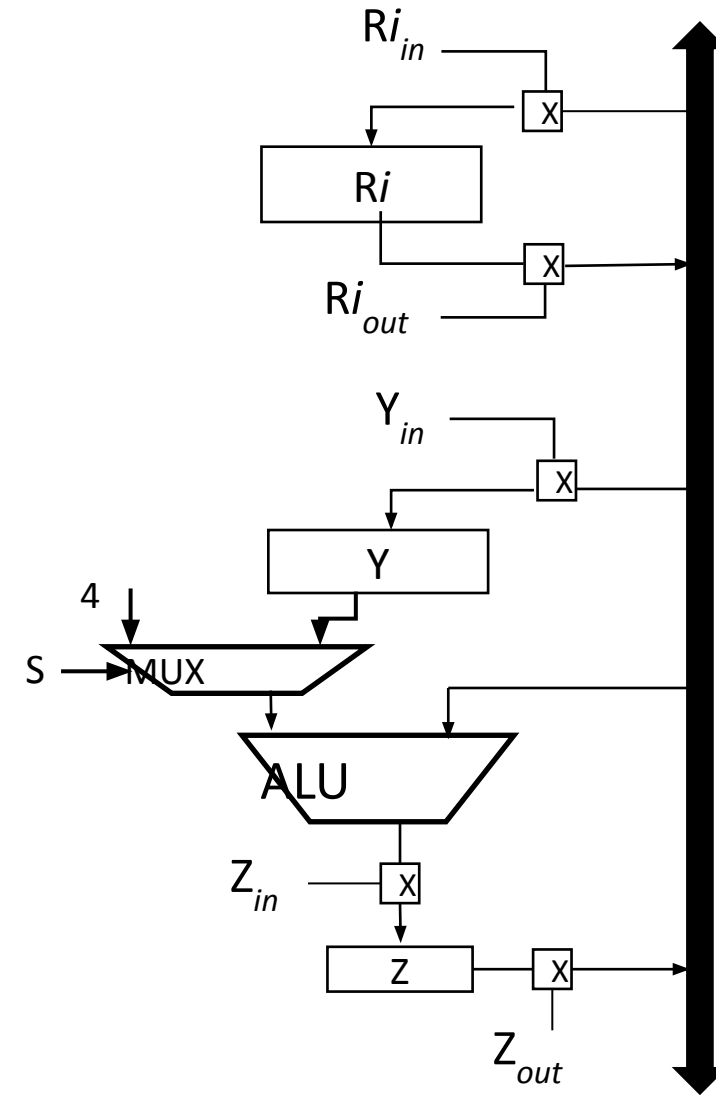
ADD R1, R2 // $R1 = R1 + R2$

- Bring the two operands (**R1** and **R2**) to the two inputs of the ALU.
One through **Y** (**R1**) and another (**R2**) directly from internal bus.
- Result is stored in **Z** and finally transferred to **R1**.

T1: $R1_{out}, Y_{in}$

T2: $R2_{out}, SelectY, ADD, Z_{in}$

T3: $Z_{out}, R1_{in}$



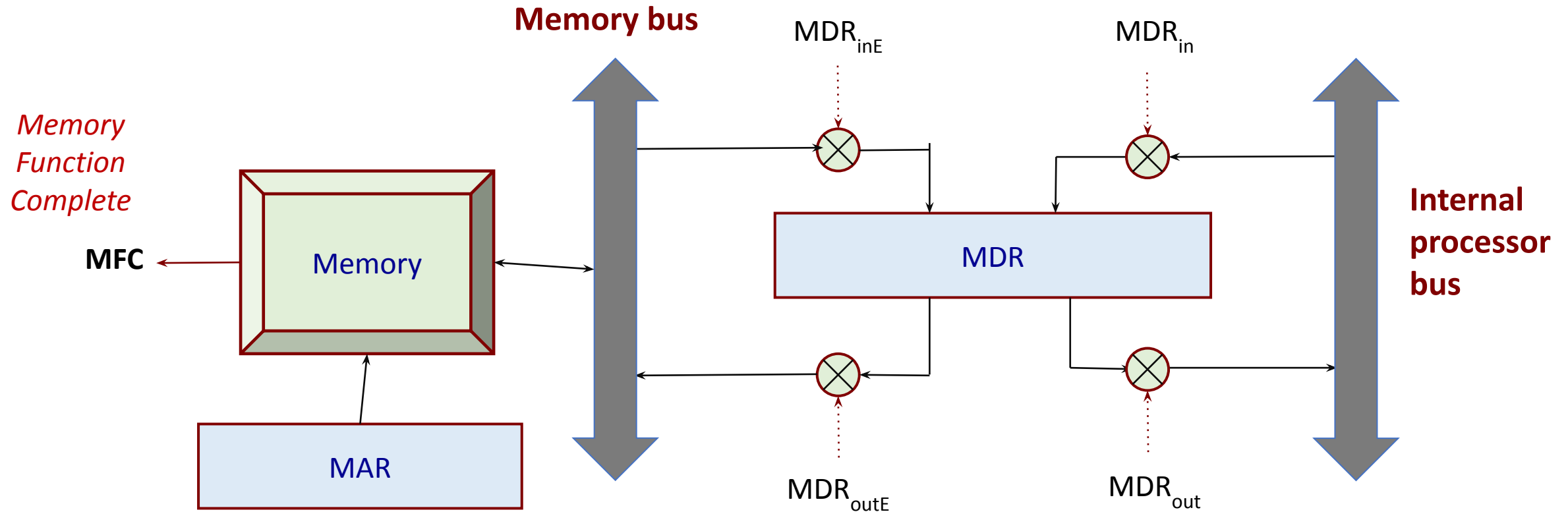
Fetching a Word from Memory

- The steps involved to fetch a word from memory:
 - The processor specifies the address of the memory location where the data or instruction is stored (move to *MAR*).
 - The processor requests a *read* operation.
 - The information to be fetched can either be an instruction or an operand of the instruction.
 - The data read is brought from the memory to *MDR*.
 - Then it is transferred to the required register or ALU for further operation.

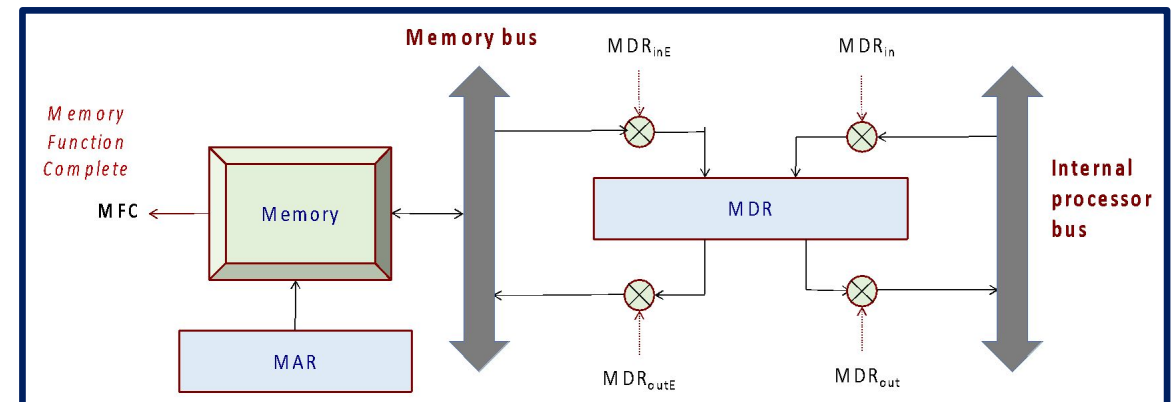
Storing a Word into Memory

- The steps involved to store a word into the memory:
 - The processor specifies the address of the memory location where the data is to be written (move to *MAR*).
 - The data to be written is loaded into *MDR*.
 - The processor requests a *write* operation.
 - The content of *MDR* will be written to the specified memory location.

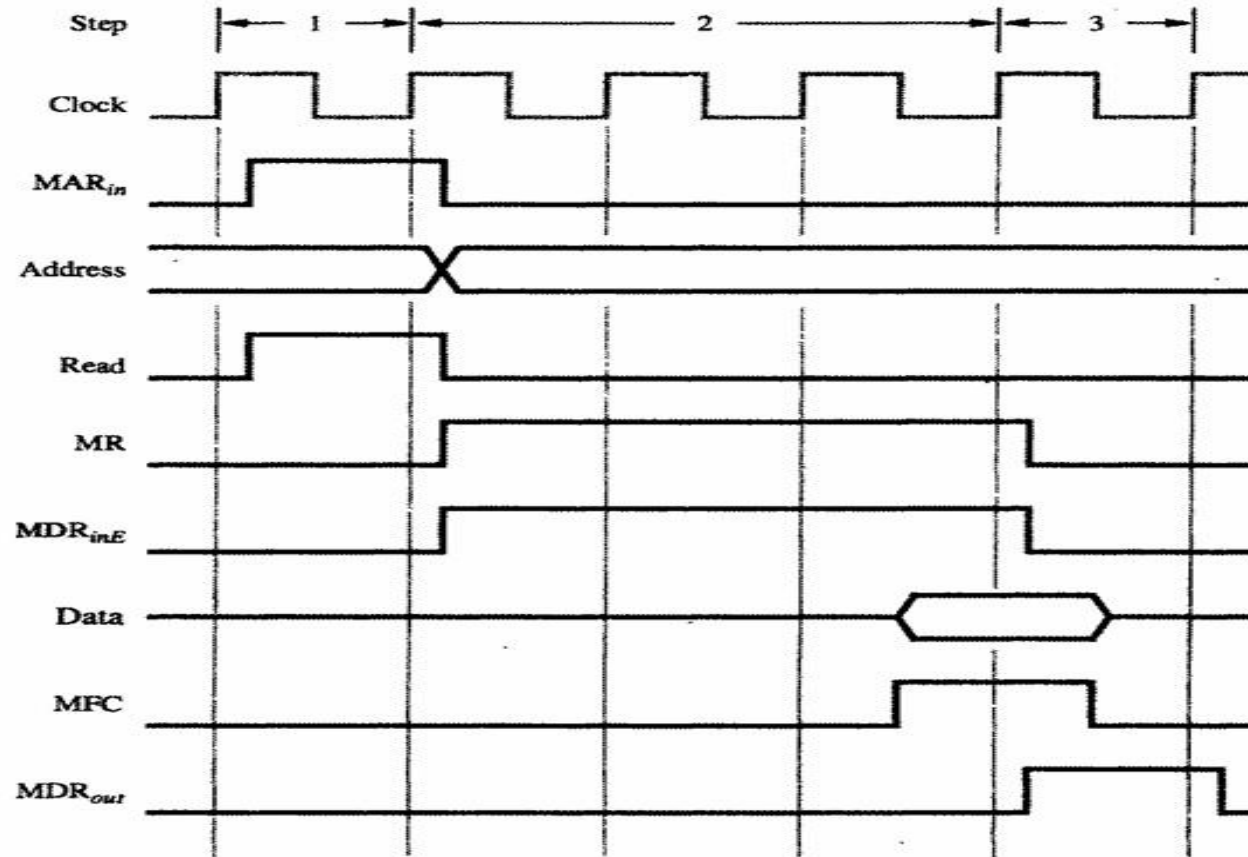
Connecting MDR to Memory Bus and Internal Bus



- Memory read/write operation:
 - The address of memory location is transferred to *MAR*.
 - At the same time a *read/write* control signal is provided to indicate the operation.
 - For read, the data from memory data bus comes to *MDR* by activating *MDR_{inE}*.
 - For write, the data from *MDR* goes to memory data bus by activating the signal *MDR_{outE}*.
 - When the processor sends a read request, it has to wait until the data is read from the memory and written into *MDR*.
 - To accommodate the variability in response time, the process has to wait until it receives an indication from the memory that the read operation has been completed.
 - A control signal called *Memory Function Complete (MFC)* is used for this purpose.
 - When this signal is 1, indicates that the contents of the specified location is read and are available on the data line of the memory bus.
 - Then the data can be made available to *MDR*.



Timing Diagram for Read Operation



1. $MAR \leftarrow R2$
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory
5. $R1 \leftarrow MDR$

Control steps:

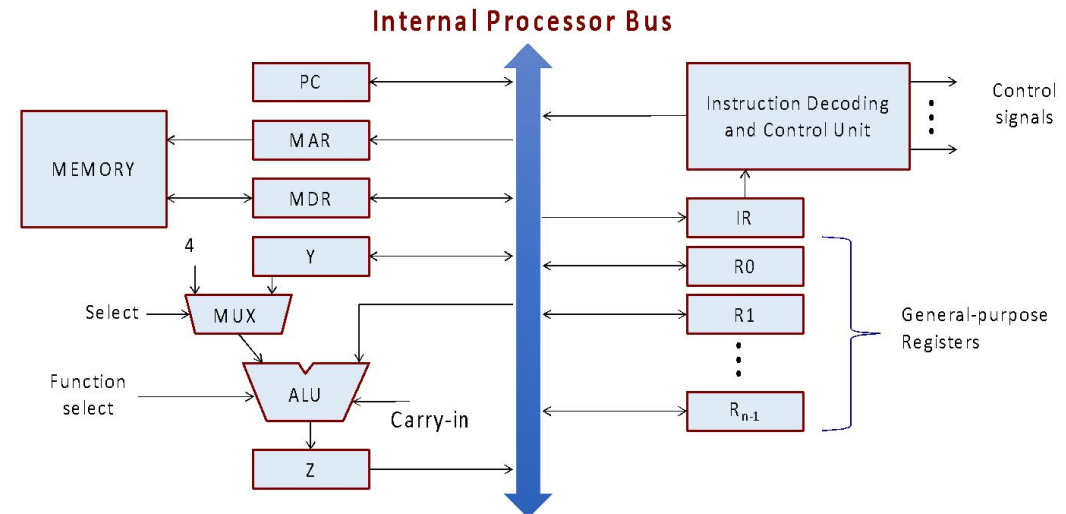
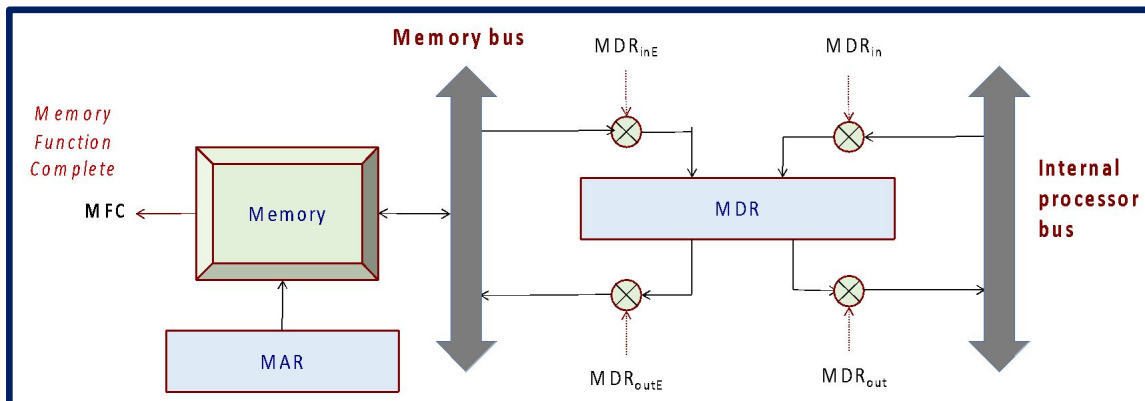
1. $R2_{out}$, MAR_{in} , Read
2. MDR_{inE} , WMFC
3. MDR_{out} , $R1_{in}$

Fetch a word: **MOVE R1, (R2)**

1. $MAR \leftarrow R2$
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory
5. $R1 \leftarrow MDR$

Control steps:

- a) $R2_{out}, MAR_{in}, \text{Read}$
- b) MDR_{inE}, WMFC
- c) $MDR_{out}, R1_{in}$

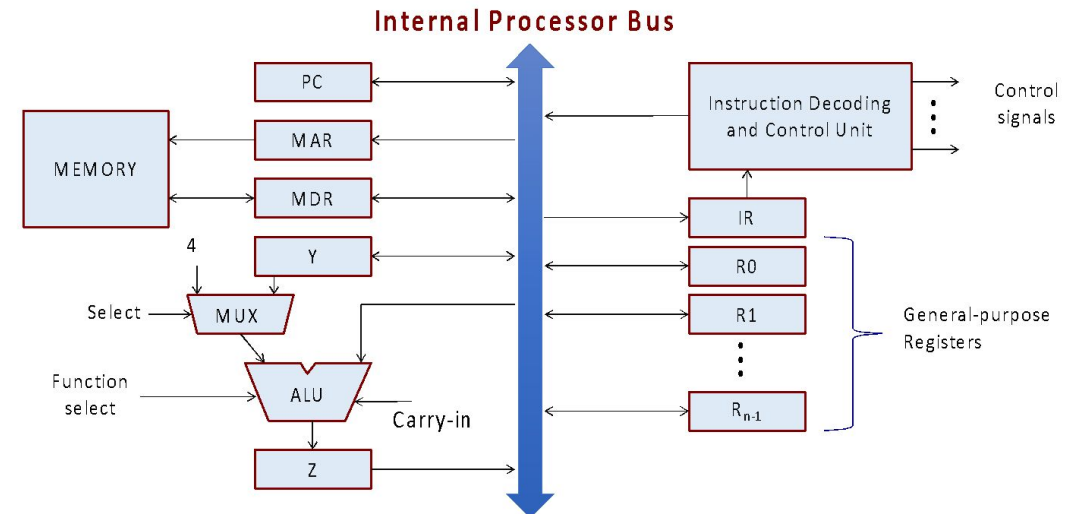
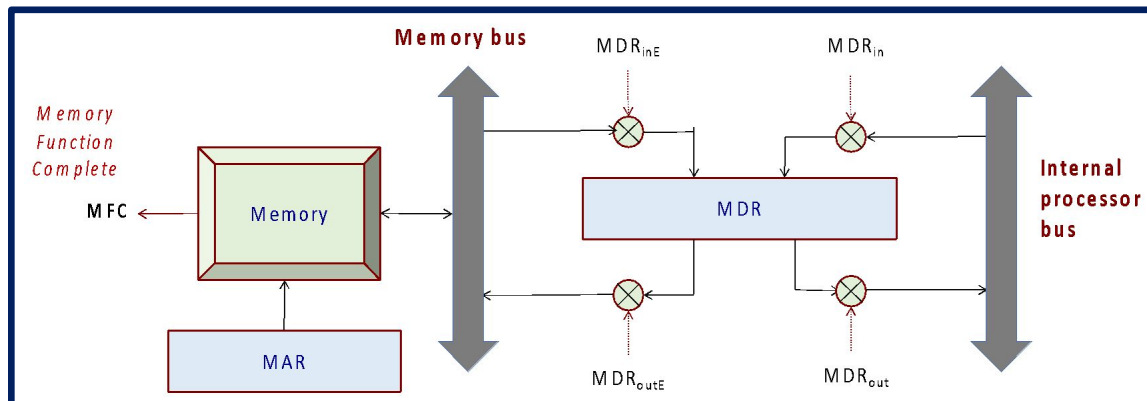


Fetch a word: **MOVE R1, (R2)**

1. $MAR \leftarrow R2$
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory
5. $R1 \leftarrow MDR$

Control steps:

- a) $R2_{out}, MAR_{in}, \text{Read}$
- b) MDR_{inE}, WMFC
- c) $MDR_{out}, R1_{in}$

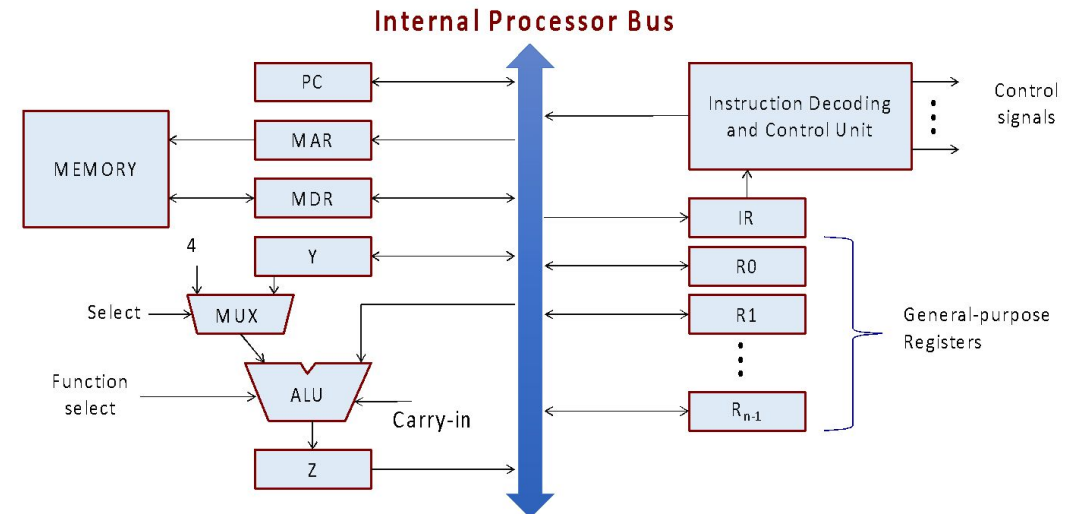
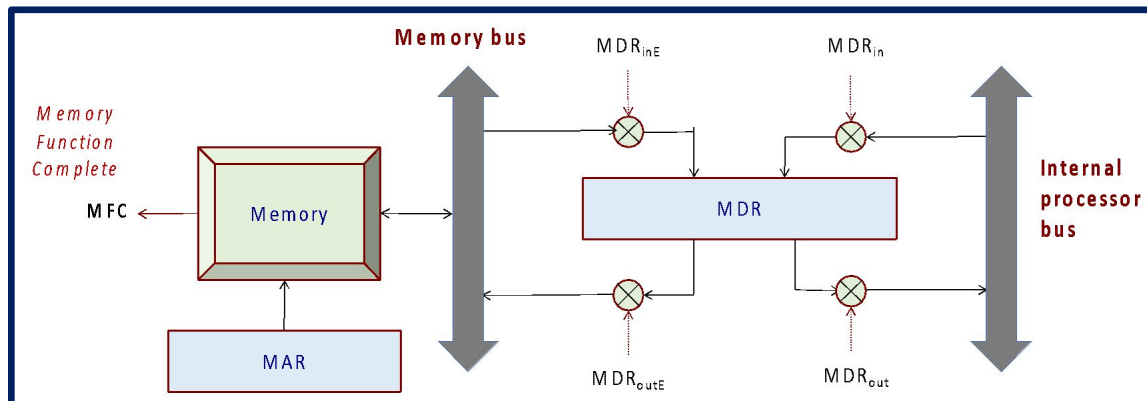


Fetch a word: **MOVE R1, (R2)**

1. $MAR \leftarrow R2$
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory
5. $R1 \leftarrow MDR$

Control steps:

- a) $R2_{out}, MAR_{in}, \text{Read}$
- b) MDR_{inE}, WMFC
- c) $MDR_{out}, R1_{in}$

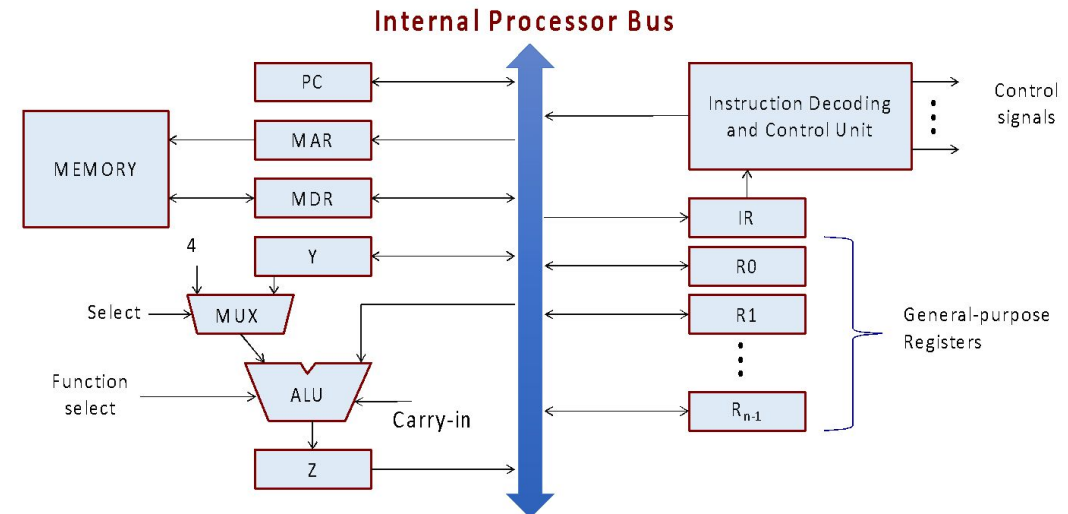
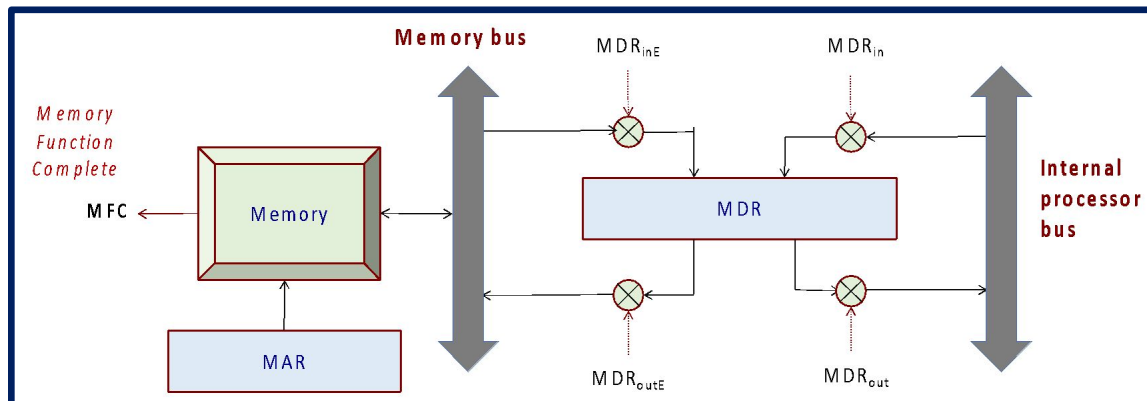


Fetch a word: **MOVE R1, (R2)**

1. $MAR \leftarrow R2$
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory
5. $R1 \leftarrow MDR$

Control steps:

- a) $R2_{out}, MAR_{in}, \text{Read}$
- b) MDR_{inE}, WMFC
- c) $MDR_{out}, R1_{in}$

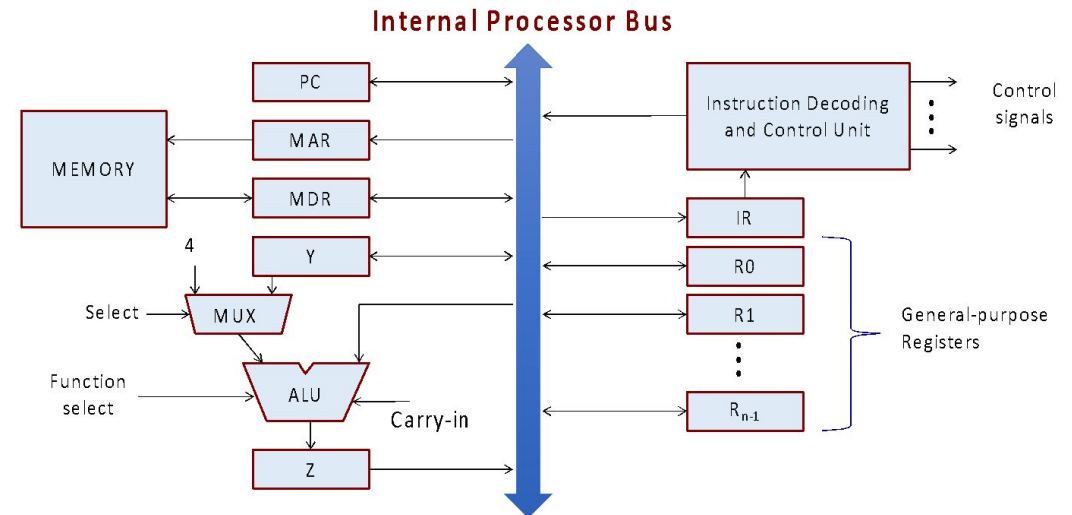
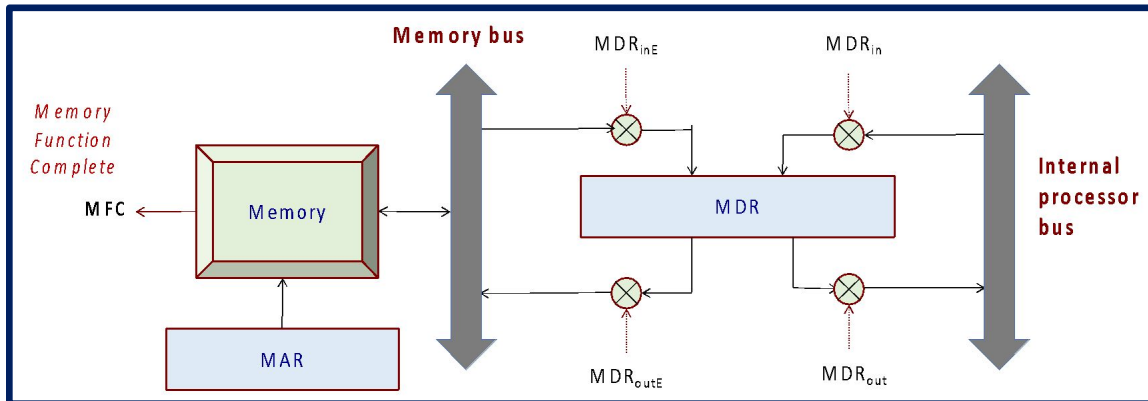


Store a word: **MOVE (R1), R2**

1. $MAR \leftarrow R1$
2. $MDR \leftarrow R2$
3. Start a Write operation on the memory bus
4. Wait for the MFC response from the memory

Control steps:

- a) $R1_{out}, MAR_{in}$
- b) $R2_{out}, MDR_{in}, Write$
- c) $MDR_{outE}, WMFC$

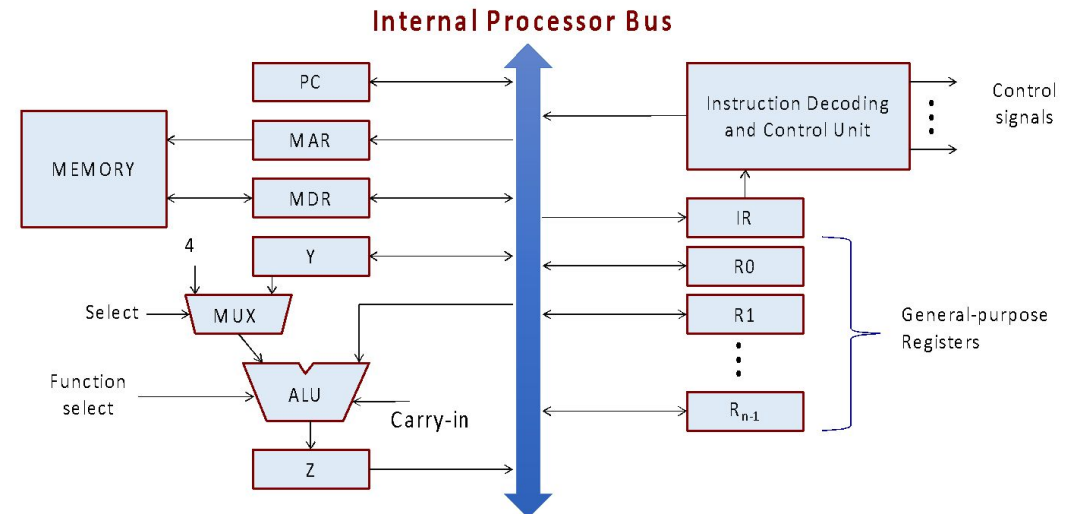
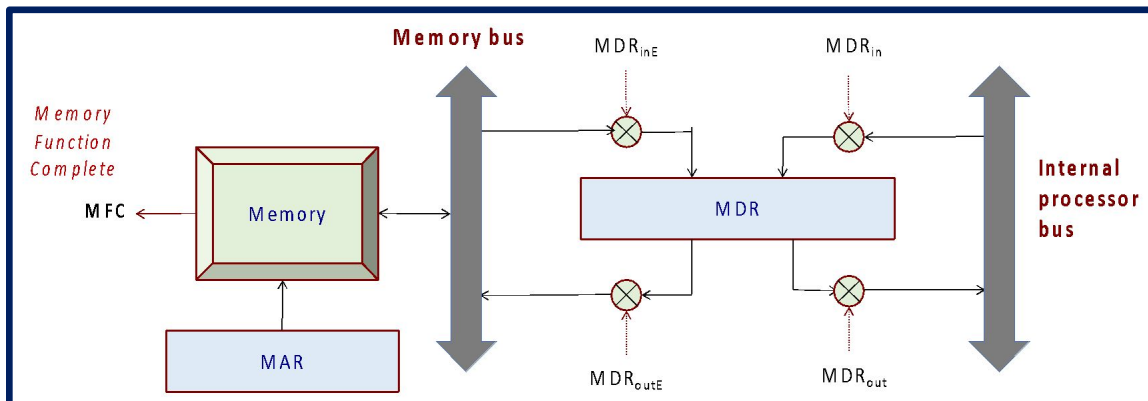


Store a word: **MOVE (R1), R2**

1. $MAR \leftarrow R1$
2. $MDR \leftarrow R2$
3. Start a Write operation on the memory bus
4. Wait for the MFC response from the memory

Control steps:

- a) $R1_{out}, MAR_{in}$
- b) $R2_{out}, MDR_{in}, \text{Write}$
- c) $MDR_{outE}, WMFC$

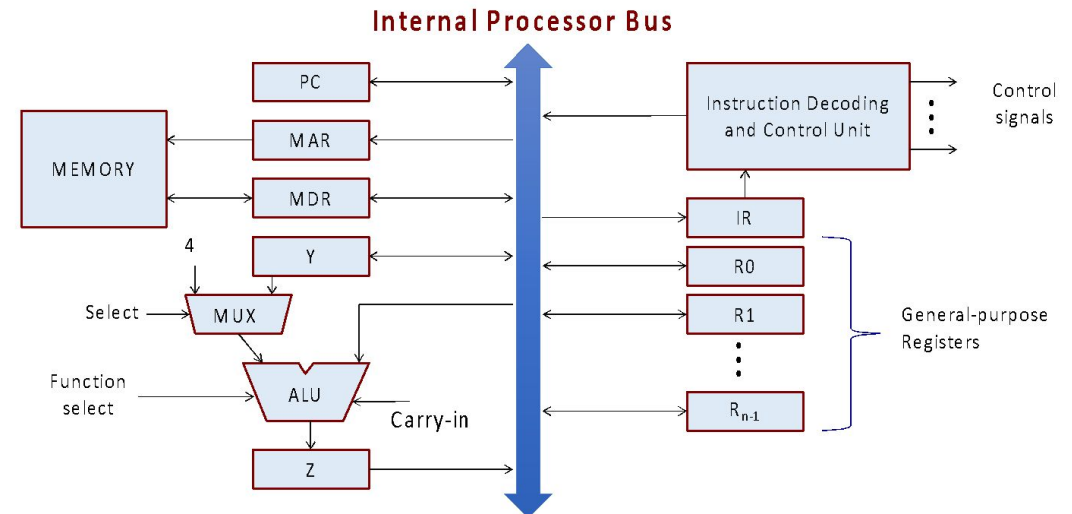
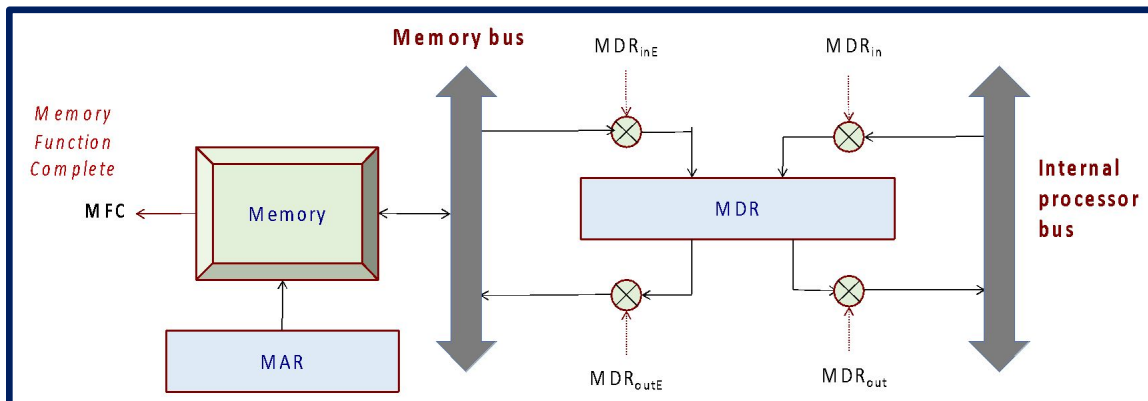


Store a word: **MOVE (R1), R2**

1. $MAR \leftarrow R1$
2. $MDR \leftarrow R2$
3. Start a Write operation on the memory bus
4. Wait for the MFC response from the memory

Control steps:

- a) $R1_{out}, MAR_{in}$
- b) $R2_{out}, MDR_{in}, \text{Write}$
- c) $MDR_{outE}, WMFC$

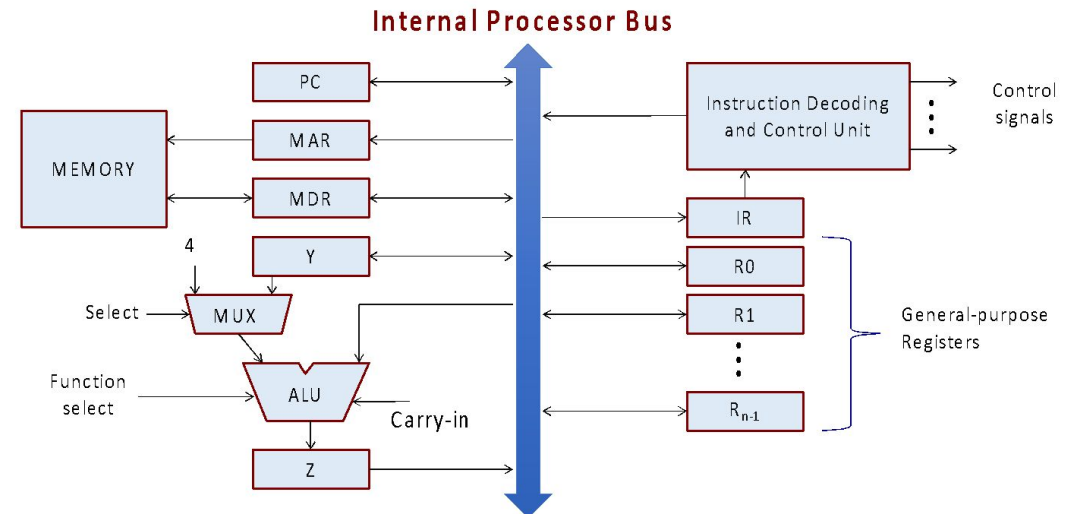
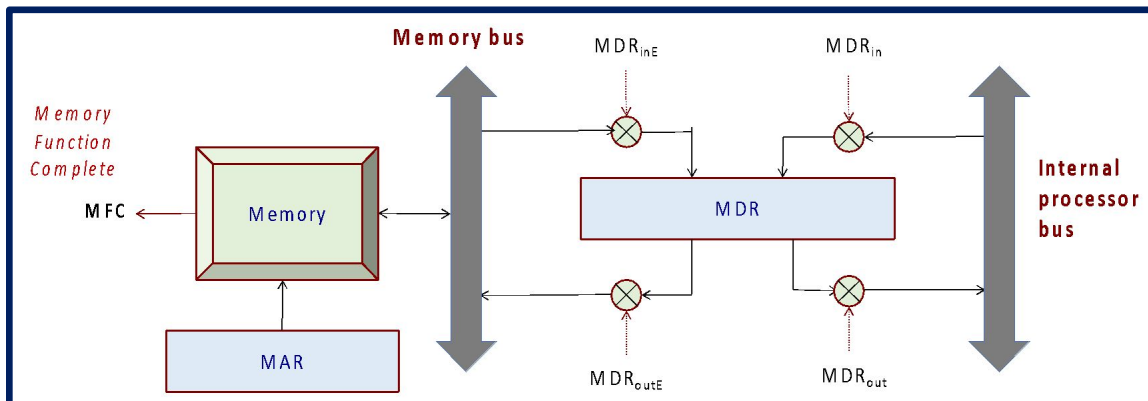


Store a word: **MOVE (R1), R2**

1. $MAR \leftarrow R1$
2. $MDR \leftarrow R2$
3. Start a Write operation on the memory bus
4. Wait for the MFC response from the memory

Control steps:

- a) $R1_{out}, MAR_{in}$
- b) $R2_{out}, MDR_{in}, \text{Write}$
- c) $MDR_{outE}, WMFC$



Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

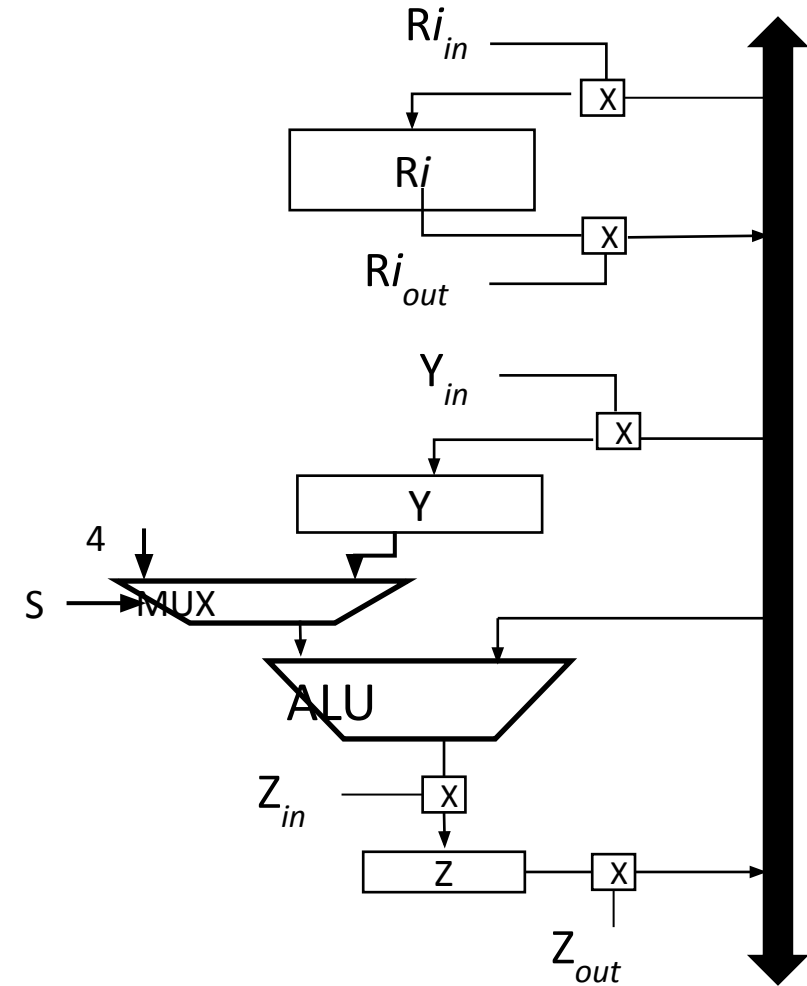
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

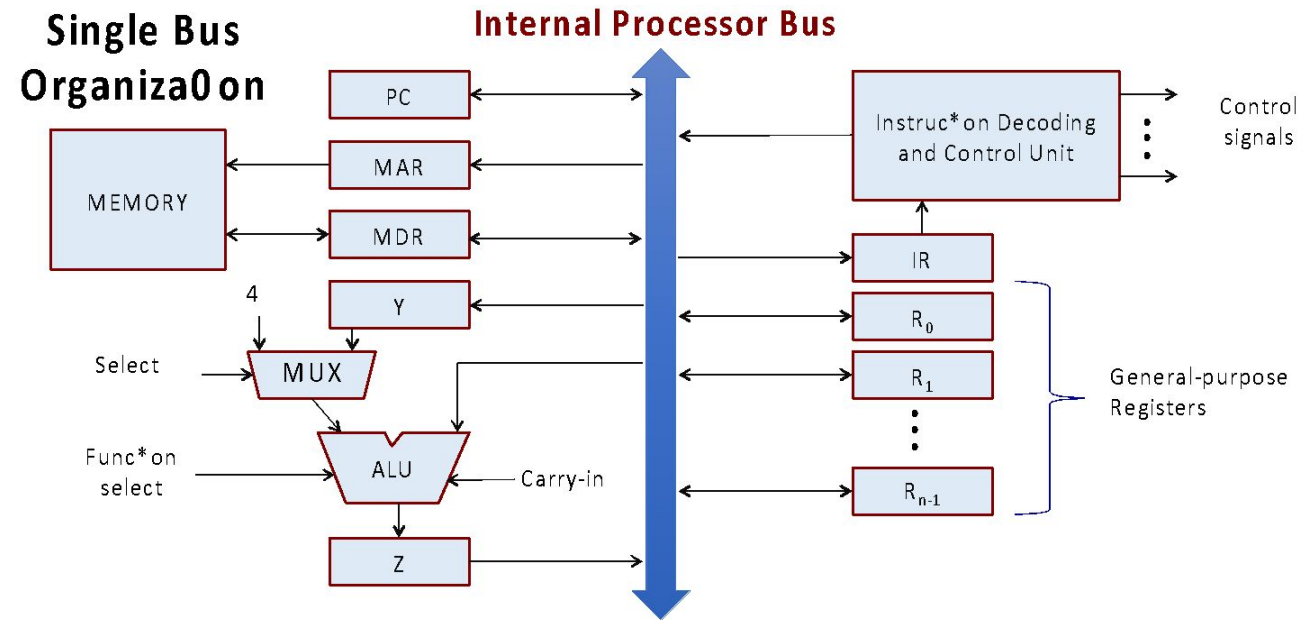
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

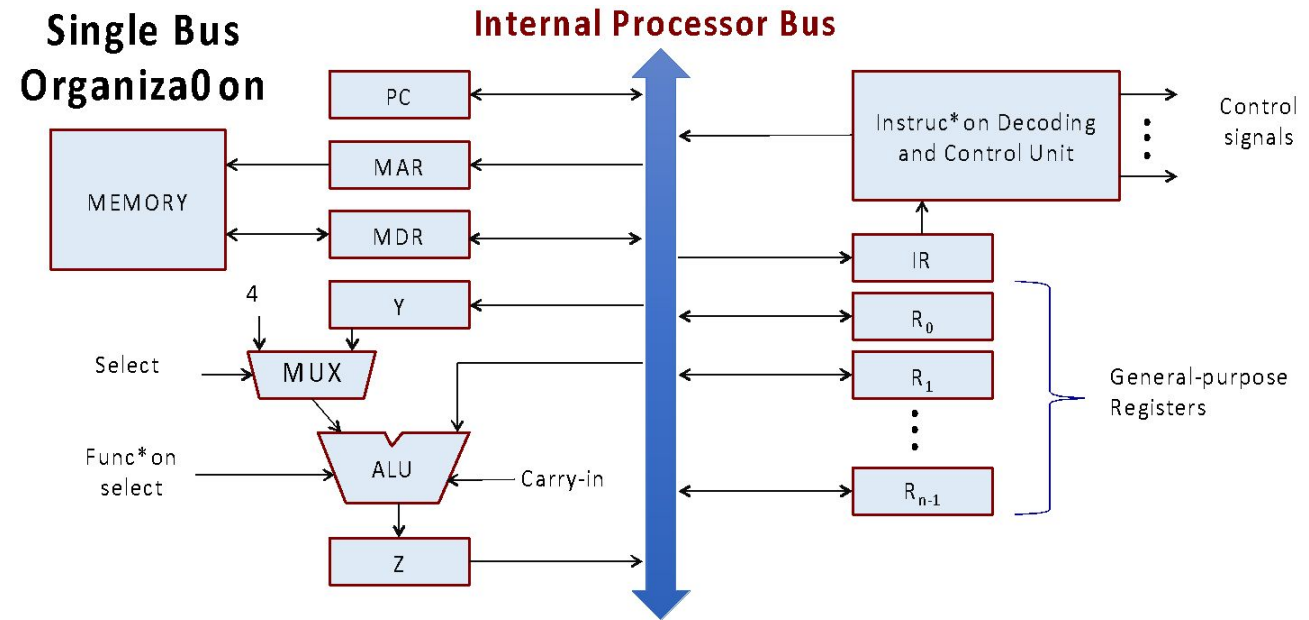
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

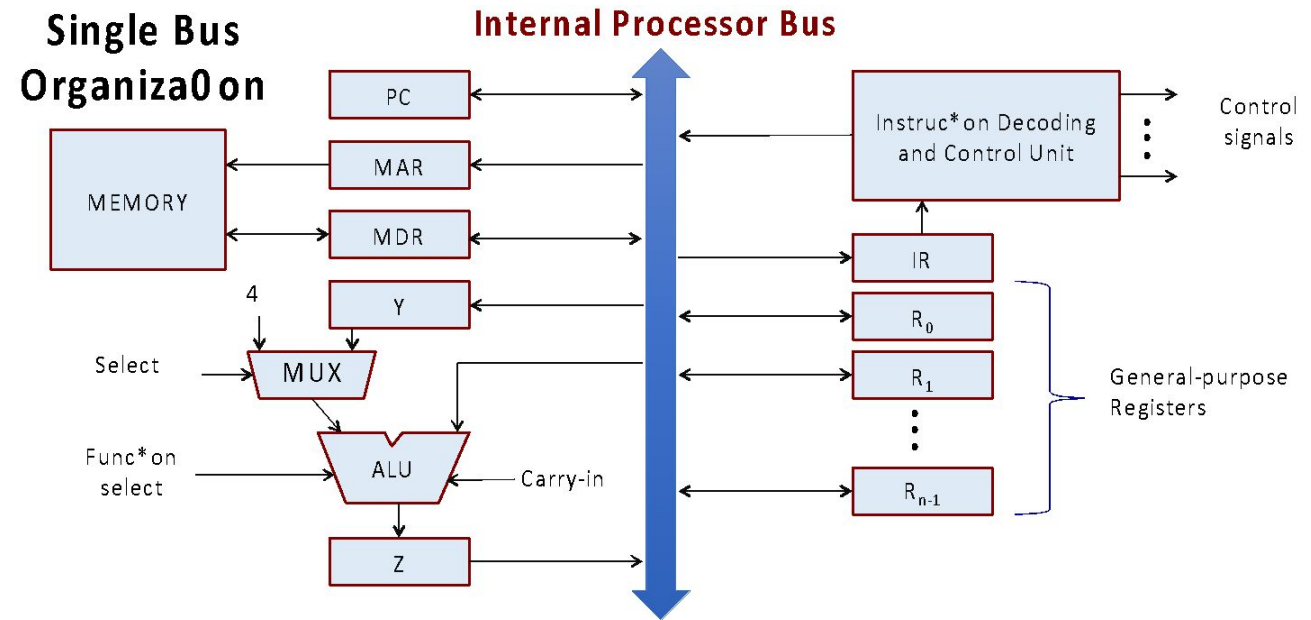
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

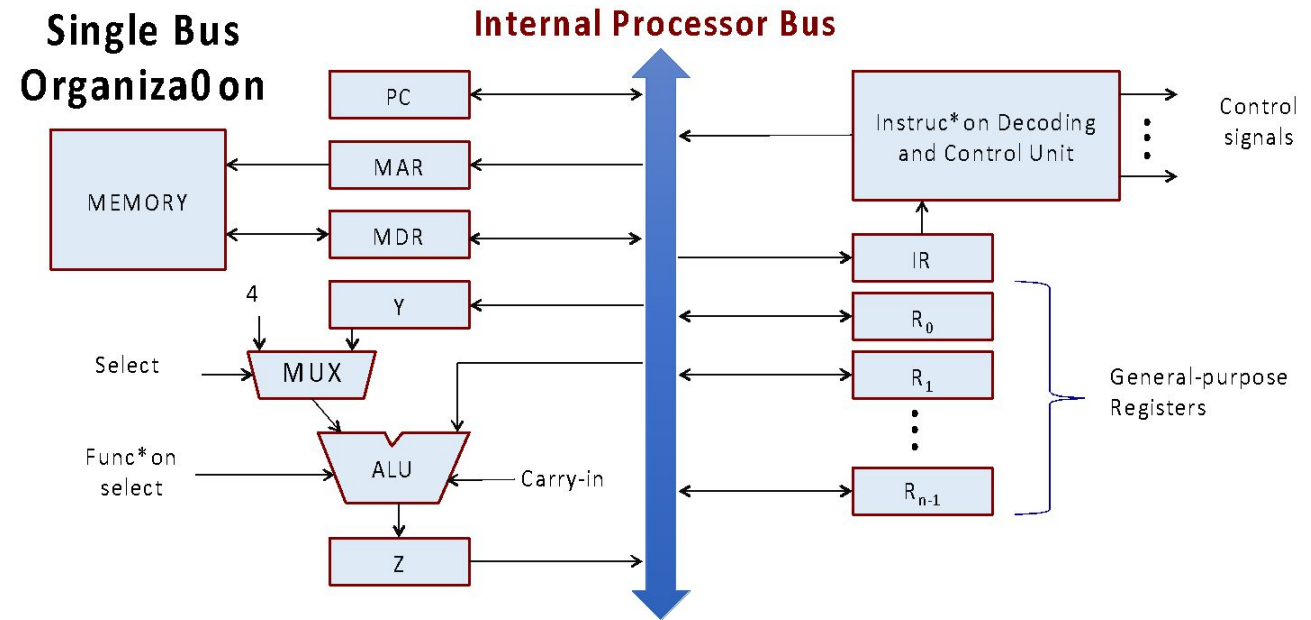
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

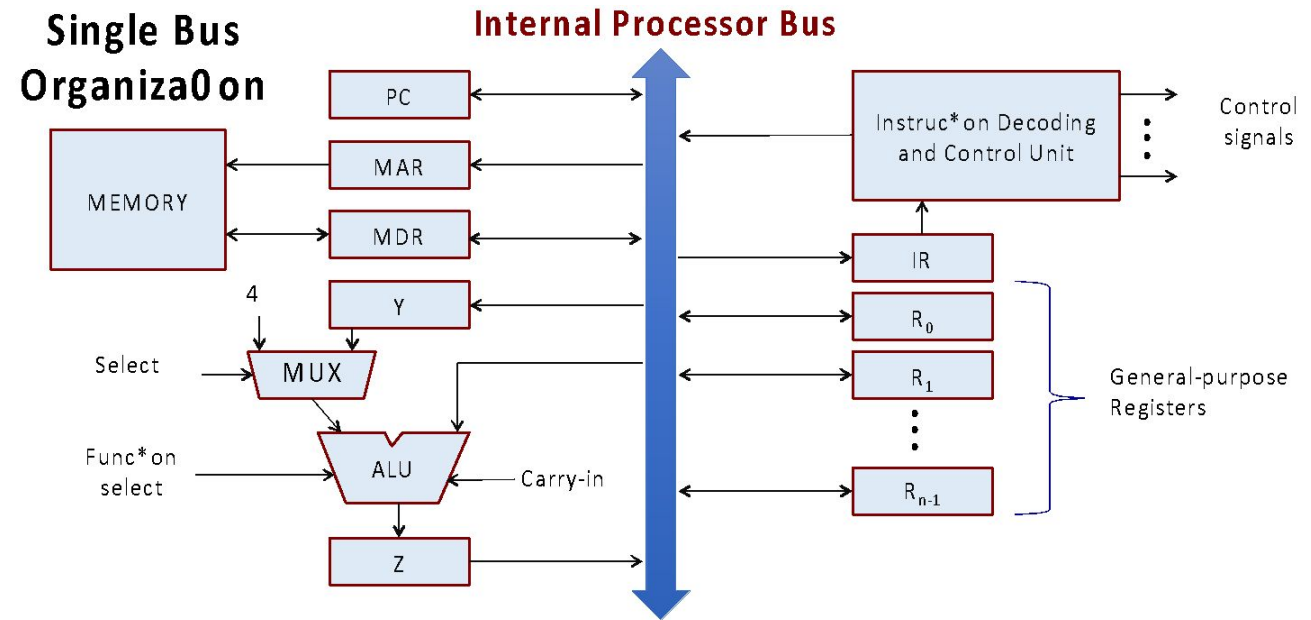
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



Execution of a Complete Instruction

ADD R1, R2 // $R1 = R1 + R2$

T1: PC_{out} , MAR_{in} , Read, Select4, ADD, Z_{in}

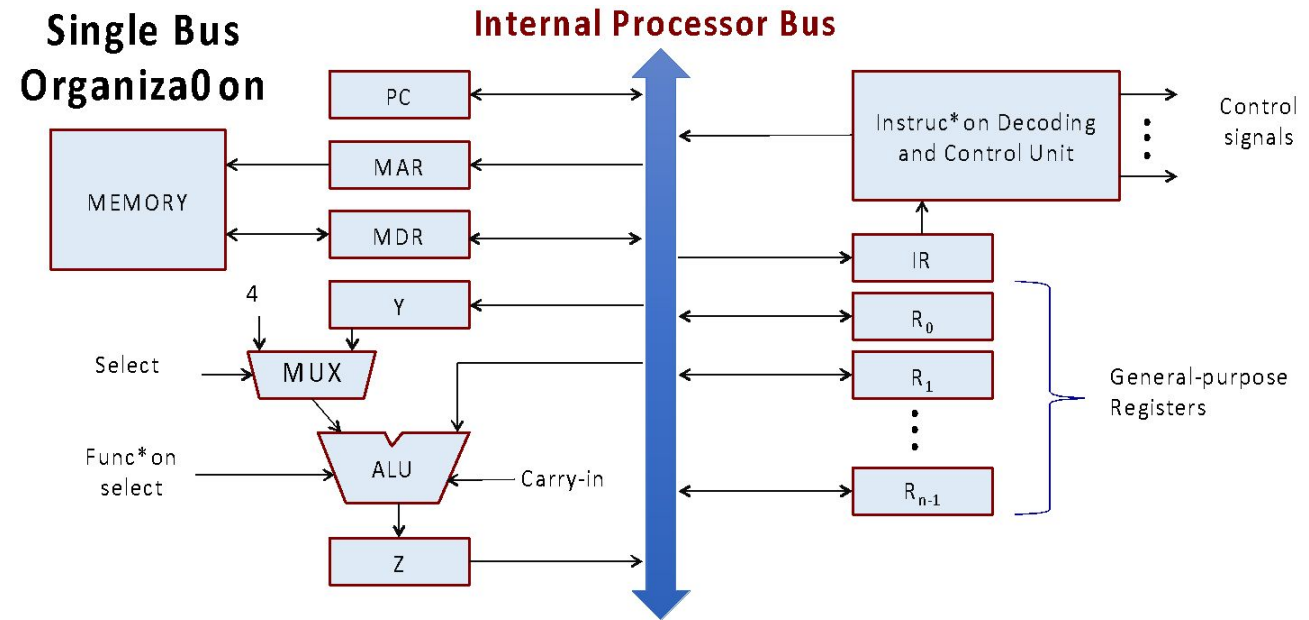
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{in} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in} , SelectY

T5: $R2_{out}$, ADD, Z_{in}

T6: Z_{out} , $R1_{in}$



SelectY can be placed in T4 or T5 since mux is a combinatorial circuit and its output will be generated almost instantly.

Example for a Three Bus Organization

SUB R1, R2, R3 // $R1 = R2 - R3$

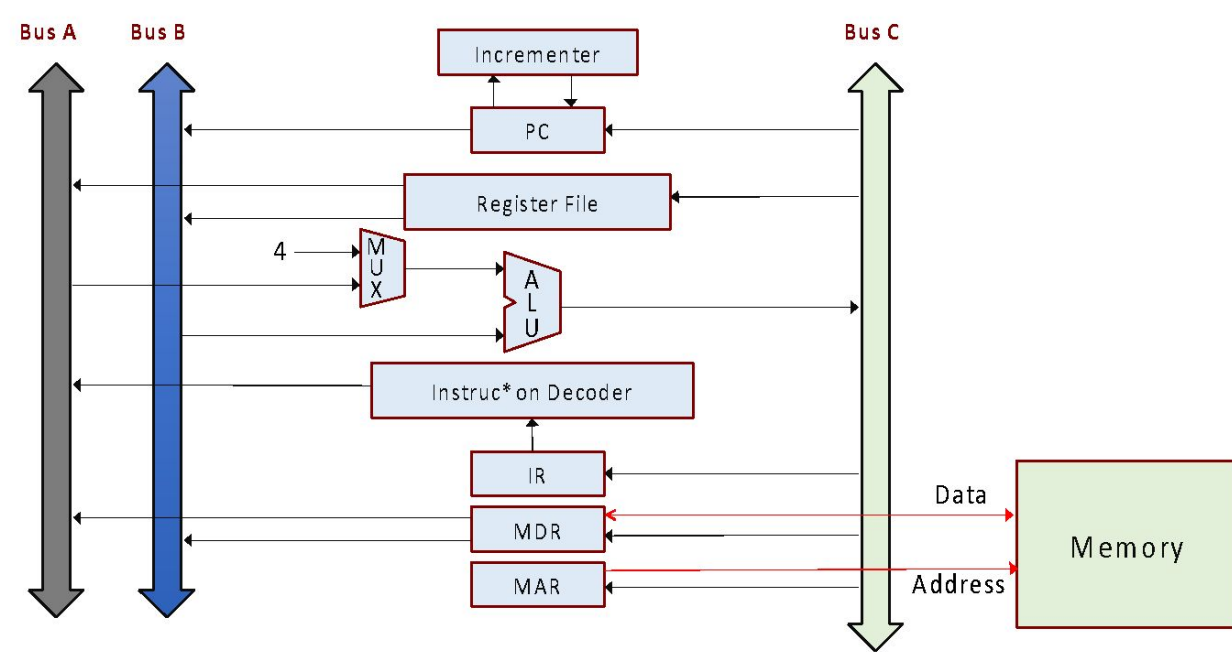
T1: PC_{out} , $R = B$, MAR_{in} , READ, IncPC

T2: WMFC

T3: MDR_{outB} , $R = B$, IR_{in}

T4: $R2_{outA}$, $R3_{outB}$, SelectA, SUB, $R1_{in}$, End

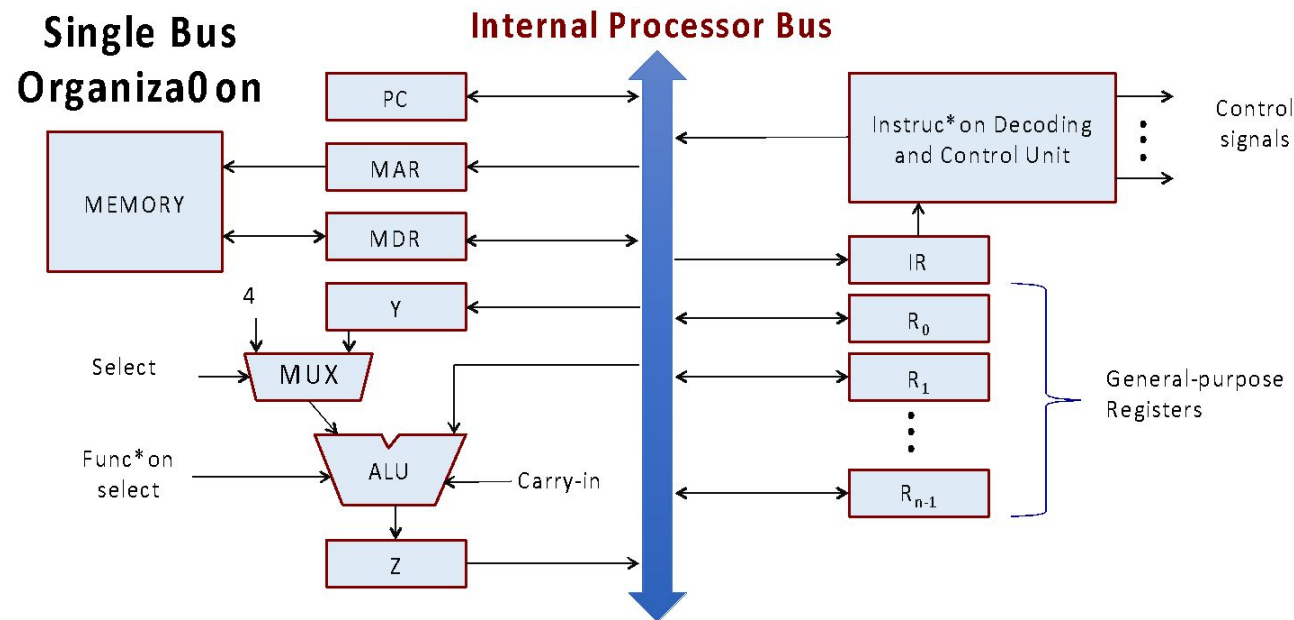
$R = B$ means that the ALU function is selected such that data on Bus-B is transferred to the ALU output (i.e., Bus-C).



Micro-operations Examples

Introduction

- We select a set of 12 instructions.
- Discuss the control signals required to execute these instructions on the single-bus processor architecture.



The set of 12 instructions chosen

1. ADD R1, R2 // $R1 = R1 + R2$
2. ADD R1, LOCA // $R1 = R1 + \text{Mem}[\text{LOCA}]$
3. LOAD R1, LOCA // $R1 = \text{Mem}[\text{LOCA}]$
4. STORE LOCA, R1 // $\text{Mem}[\text{LOCA}] = R1$
5. MOVE R1, R2 // $R1 = R2$
6. MOVE R1, #10 // $R1 = 10$
7. BR LOCA // $\text{PC} = \text{LOCA}$
8. BZ LOCA // $\text{PC} = \text{LOCA}$ if Zero flag is set
9. INC R1 // $R1 = R1 + 4$
10. DEC R1 // $R1 = R1 - 4$
11. CMP R1, R2 // $R1 - R2$
12. HALT // Machine Halt

1. ADD R1, R2 ($R1 = R1 + R2$)

T1: PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}

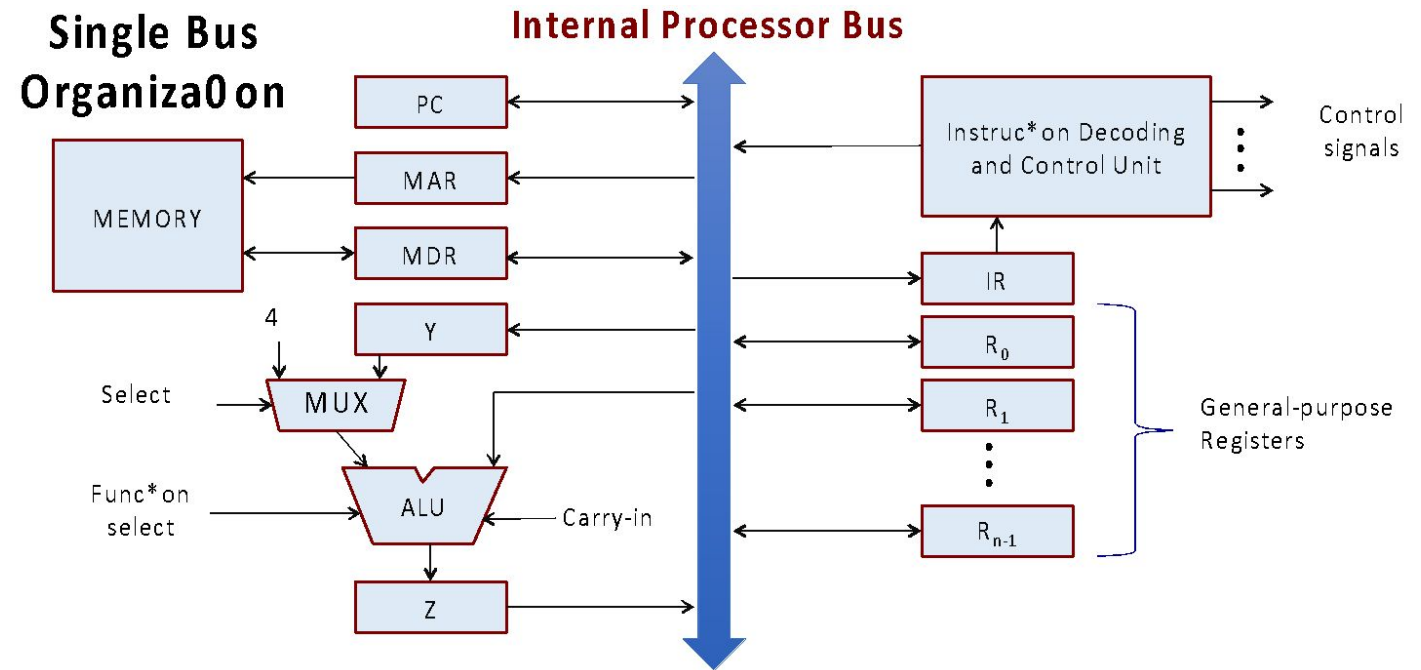
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in}

T5: $R2_{out}$, SelectY, Add, Z_{in}

T6: Z_{out} , $R1_{in}$, End



2. ADD R1, LOCA (R1 = R1 + Mem[LOCA])

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}

T2: Z_{out}, PC_{in}, Y_{in}, MDR_{inE}, WMFC

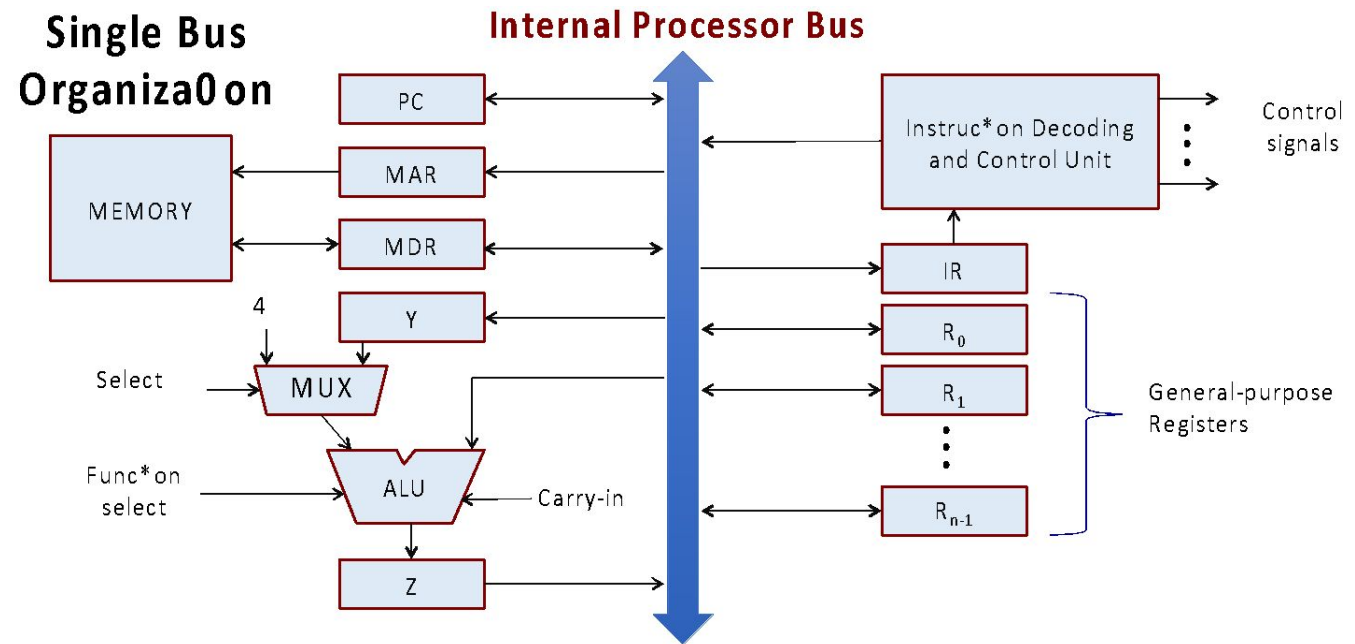
T3: MDR_{out}, IR_{in}

T4: Address field of IR_{out}, MAR_{in}, Read

T5: R1_{out}, Y_{in}, MDR_{inE}, WMFC

T6: MDR_{out}, SelectY, Add, Z_{in}

T7: Z_{out}, R1_{in}, End



3. LOAD R1, LOCA (R1 = Mem[LOCA])

T1: PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}

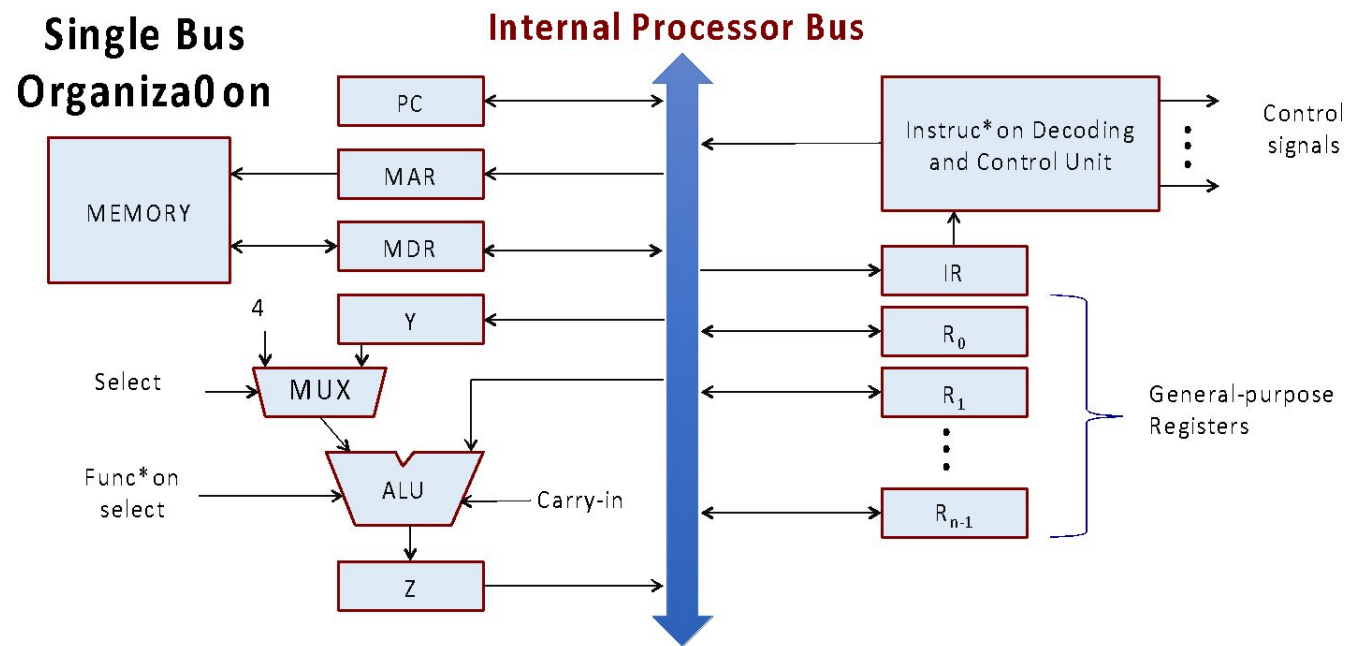
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: Address field of IR_{out} , MAR_{in} , Read

T5: MDR_{inE} , WMFC

T6: MDR_{out} , $R1_{in}$, End



4. STORE LOCA, R1

(Mem[LOCA] = R1)

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}

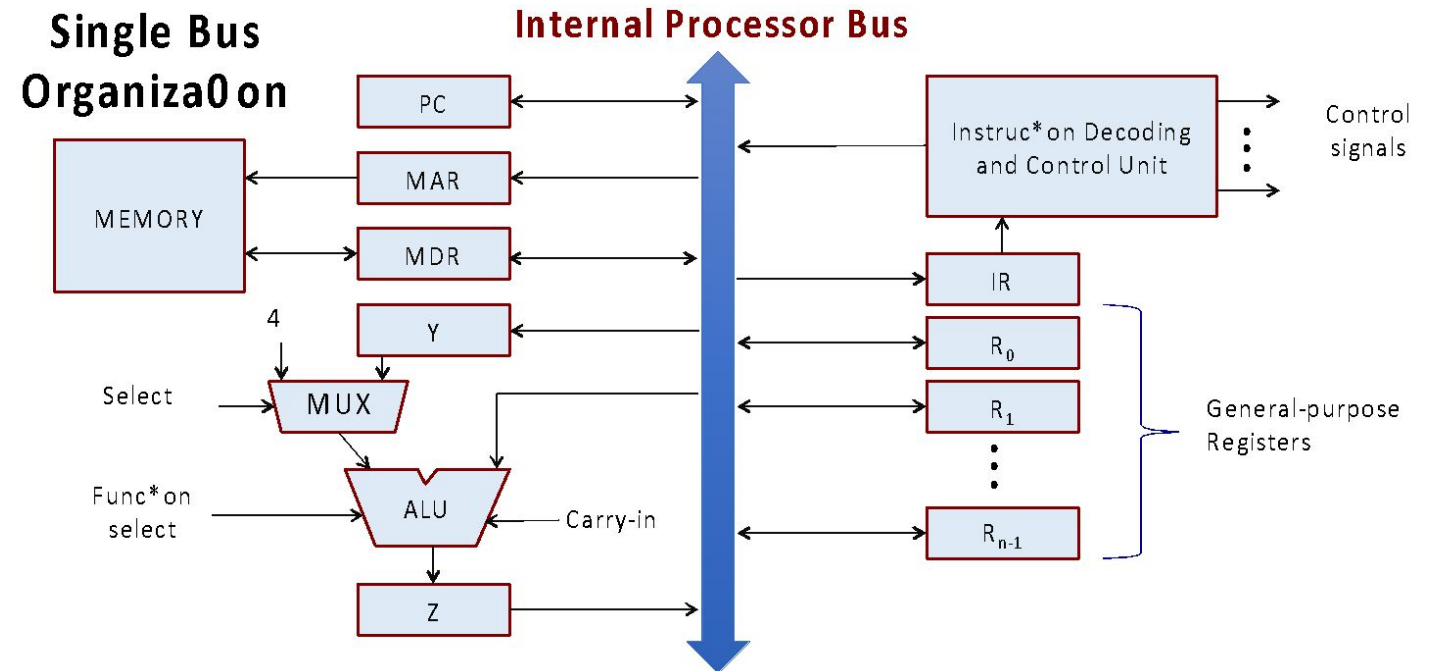
T2: Z_{out}, PC_{in}, Y_{in}, MDR_{inE}, WMFC

T3: MDR_{out}, IR_{in}

T4: Address field of IR_{out}, MAR_{in}

T5: R1_{out}, MDR_{in}, Write

T6: MDR_{outE}, WMFC, End



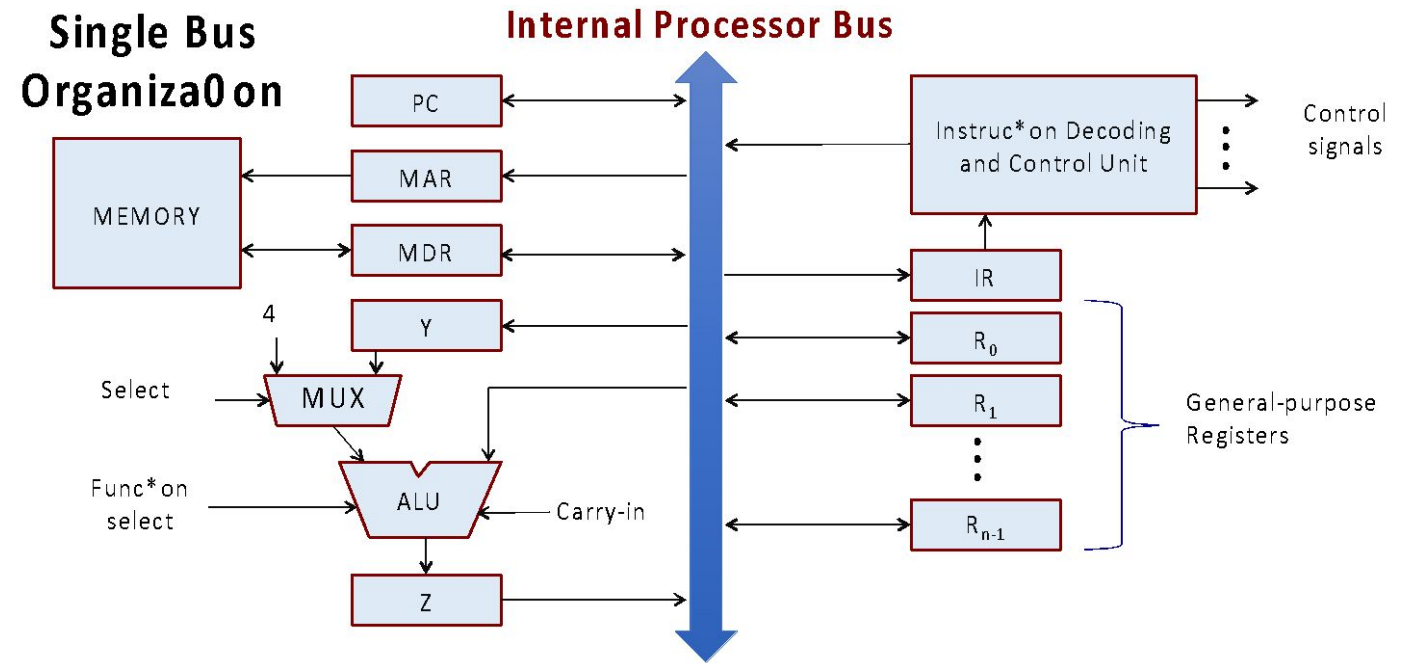
5. MOVE R1, R2 (R1 = R2)

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}

T2: Z_{out}, PC_{in}, Y_{in}, MDR_{inE}, WMFC

T3: MDR_{out}, IR_{in}

T4: R2_{out}, R1_{in}, END



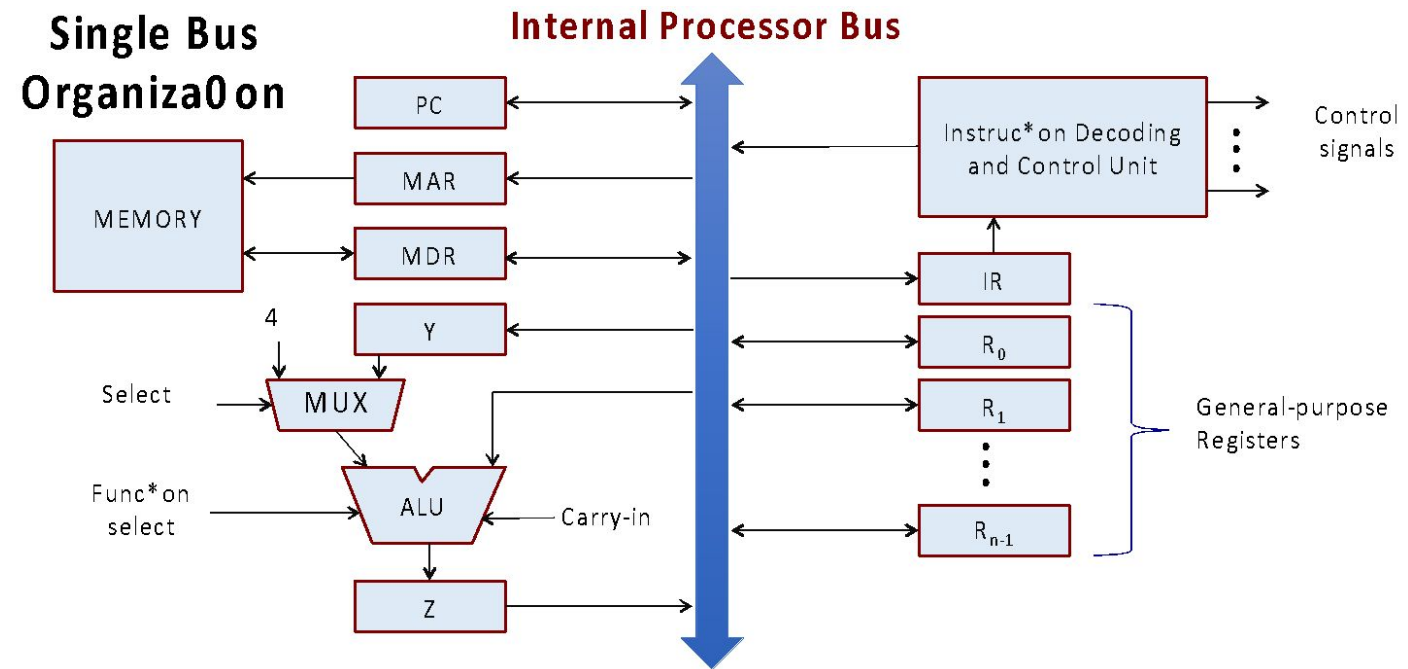
6. MOVE R1, #10 (R1 = 10)

T1: PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}

T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: Immediate field of IR_{out} , $R1_{in}$, END



7. BRANCH Label (PC = PC + offset)

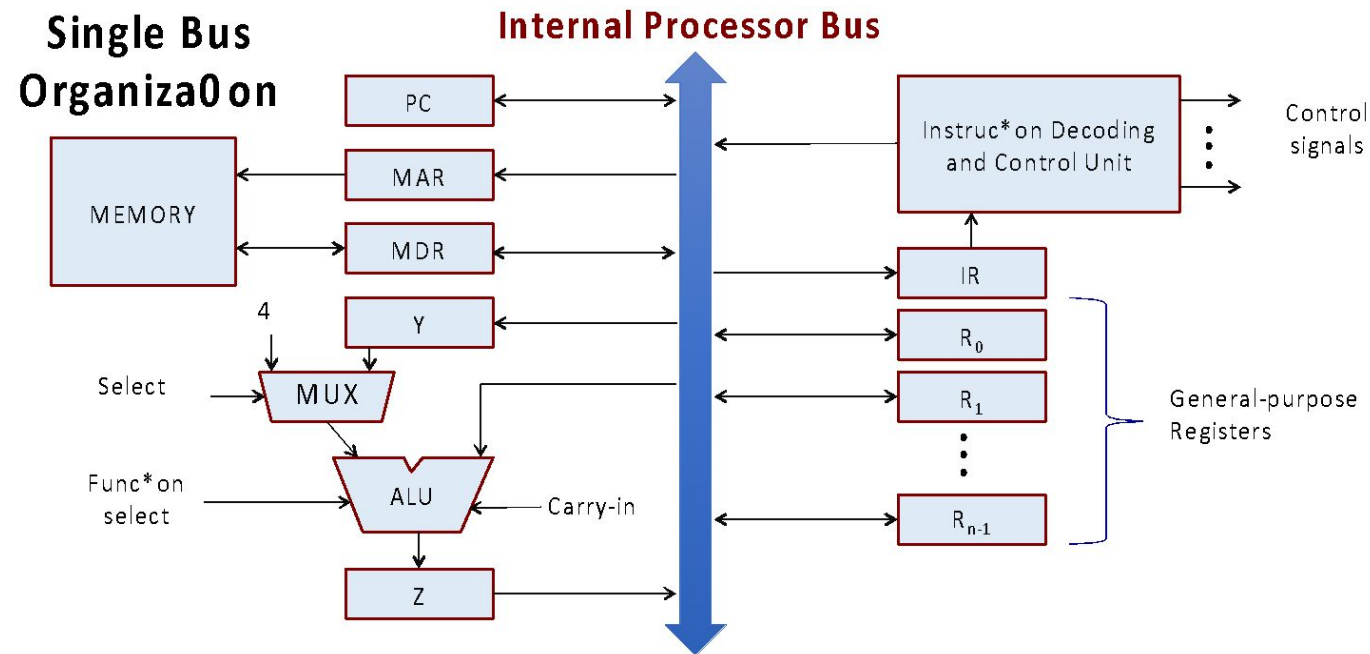
T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}

T2: Z_{out}, PC_{in}, Y_{in}, MDR_{inE}, WMFC

T3: MDR_{out}, IR_{in}

T4: Offset-field-of-IR_{out}, SelectY, Add, Z_{in}

T5: Z_{out}, PC_{in}, End



8. BZ Label (if $Z=1$, $PC = PC + \text{offset}$)

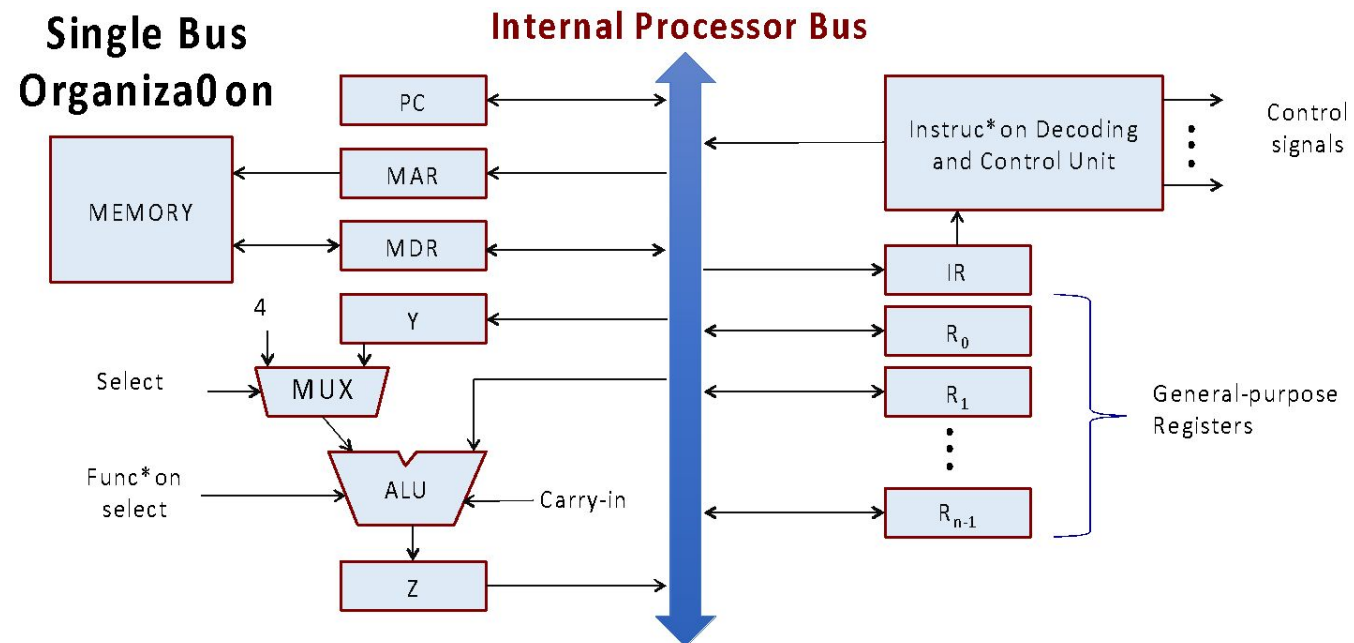
T1: PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}

T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: Offset-field-of- IR_{out} , SelectY, Add, Z_{in} , If $Z=0$ then End

T5: Z_{out} , PC_{in} , End



9. INC R1 (R1 = R1 + 4)

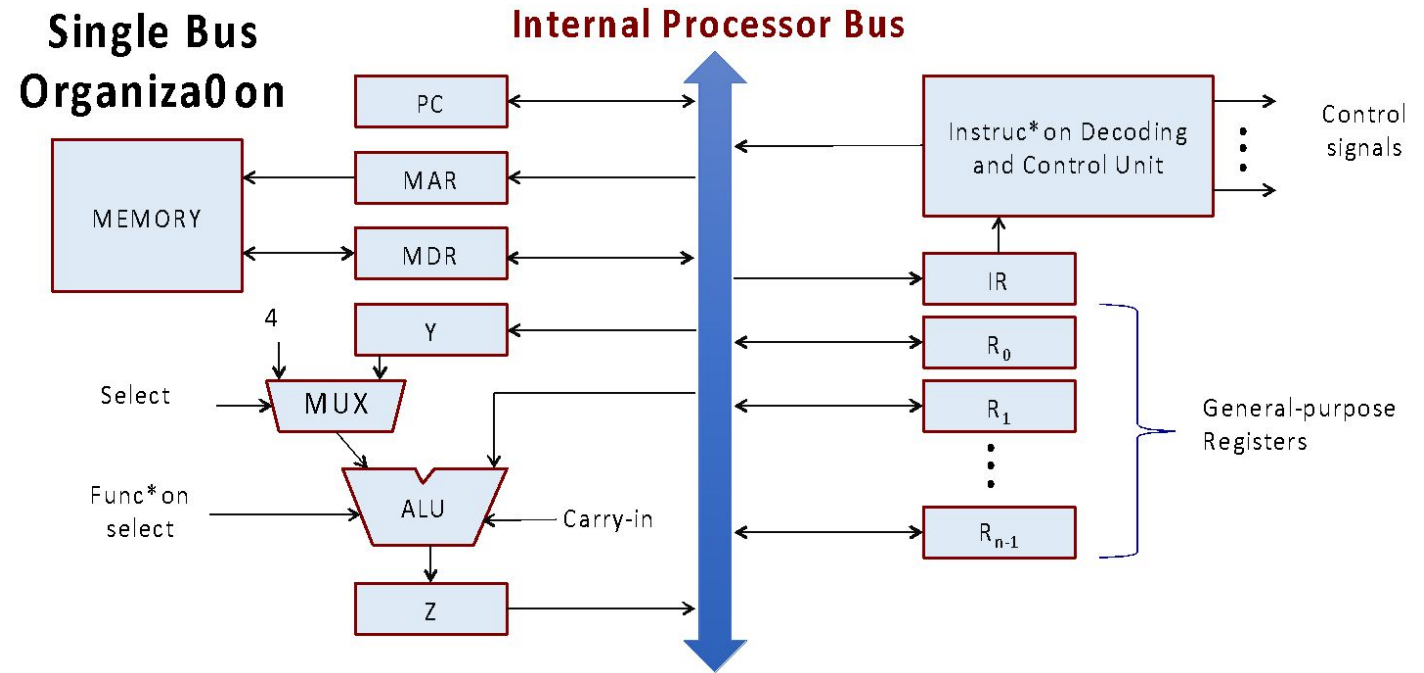
T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}

T2: Z_{out}, PC_{in}, Y_{in}, MDR_{inE}, WMFC

T3: MDR_{out}, IR_{in}

T4: R1_{out}, Select4, Add, Z_{in}

T5: Z_{out}, R1_{in}, End



10. DEC R1 (R1 = R1 - 4)

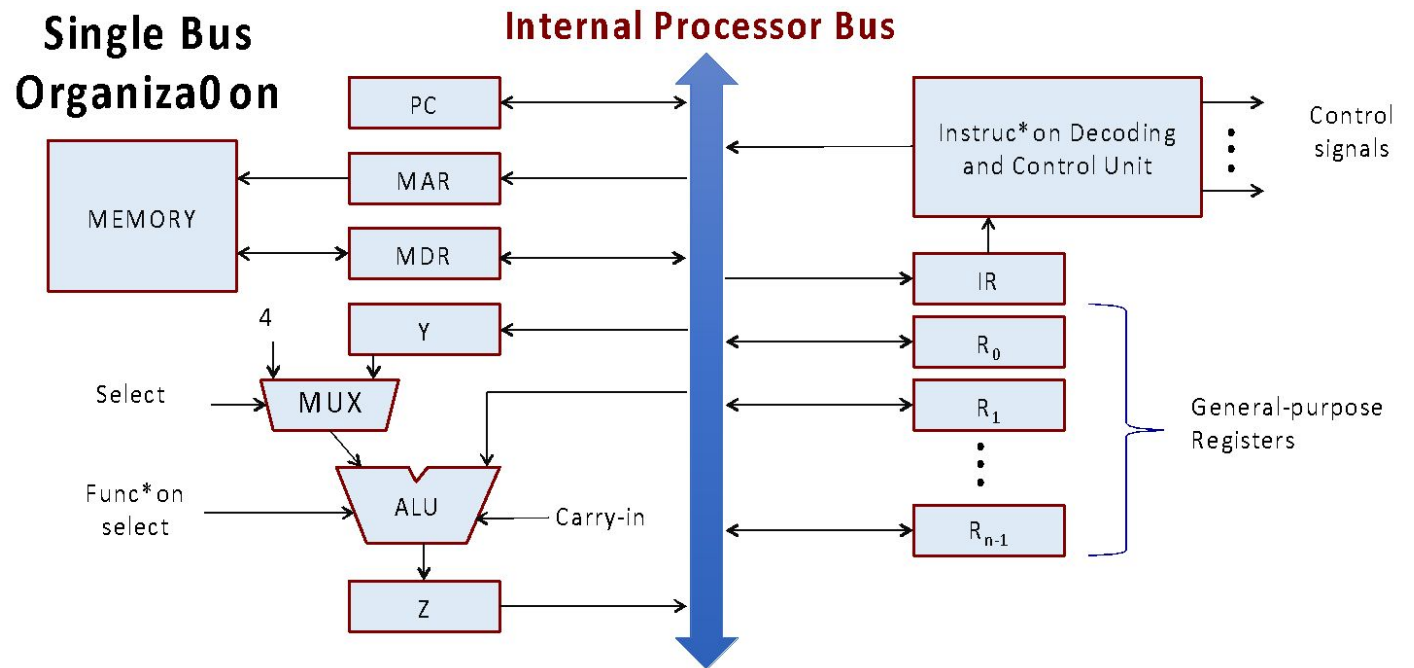
T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}

T2: Z_{out}, PC_{in}, Y_{in}, MDR_{inE}, WMFC

T3: MDR_{out}, IR_{in}

T4: R1_{out}, Select4, Sub, Z_{in}

T5: Z_{out}, R1_{in}, End



11. CMP R1, R2

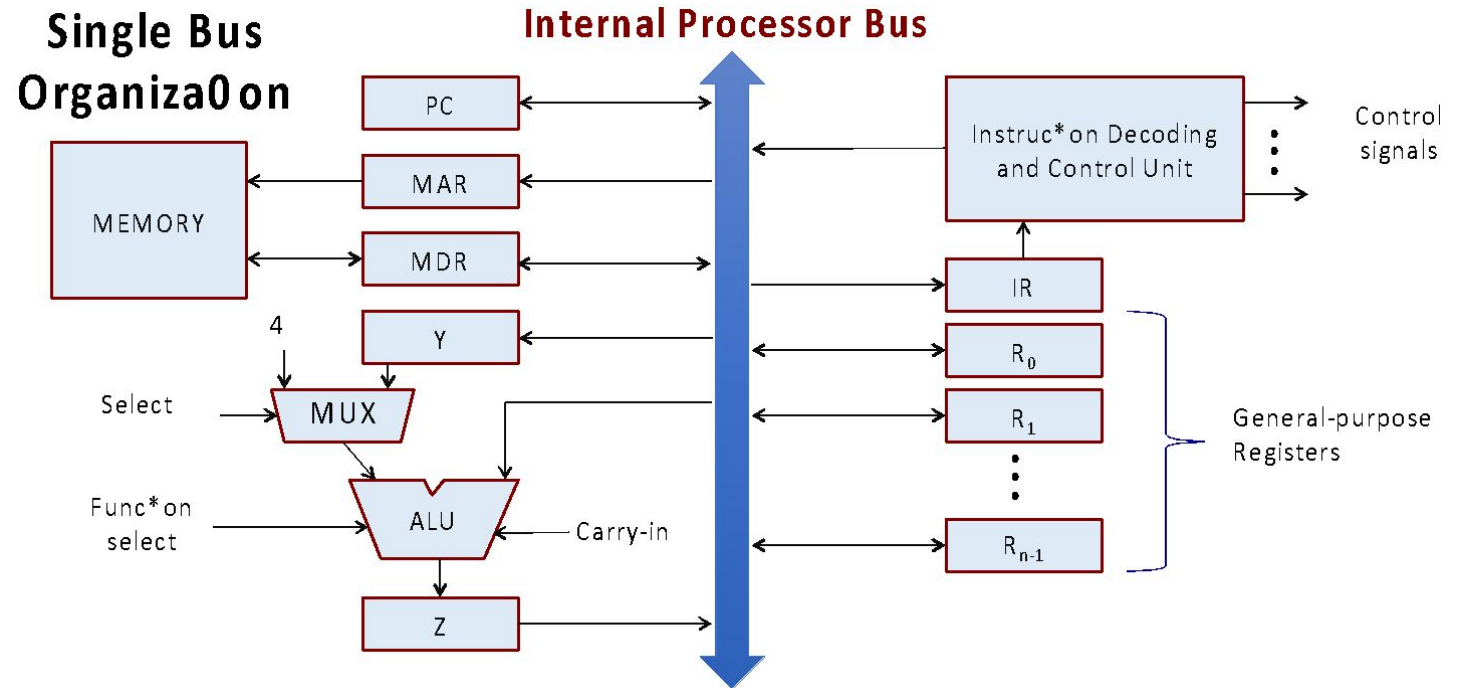
T1: PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}

T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: $R1_{out}$, Y_{in}

T5: $R2_{out}$, SelectY, Sub, Z_{in} , End



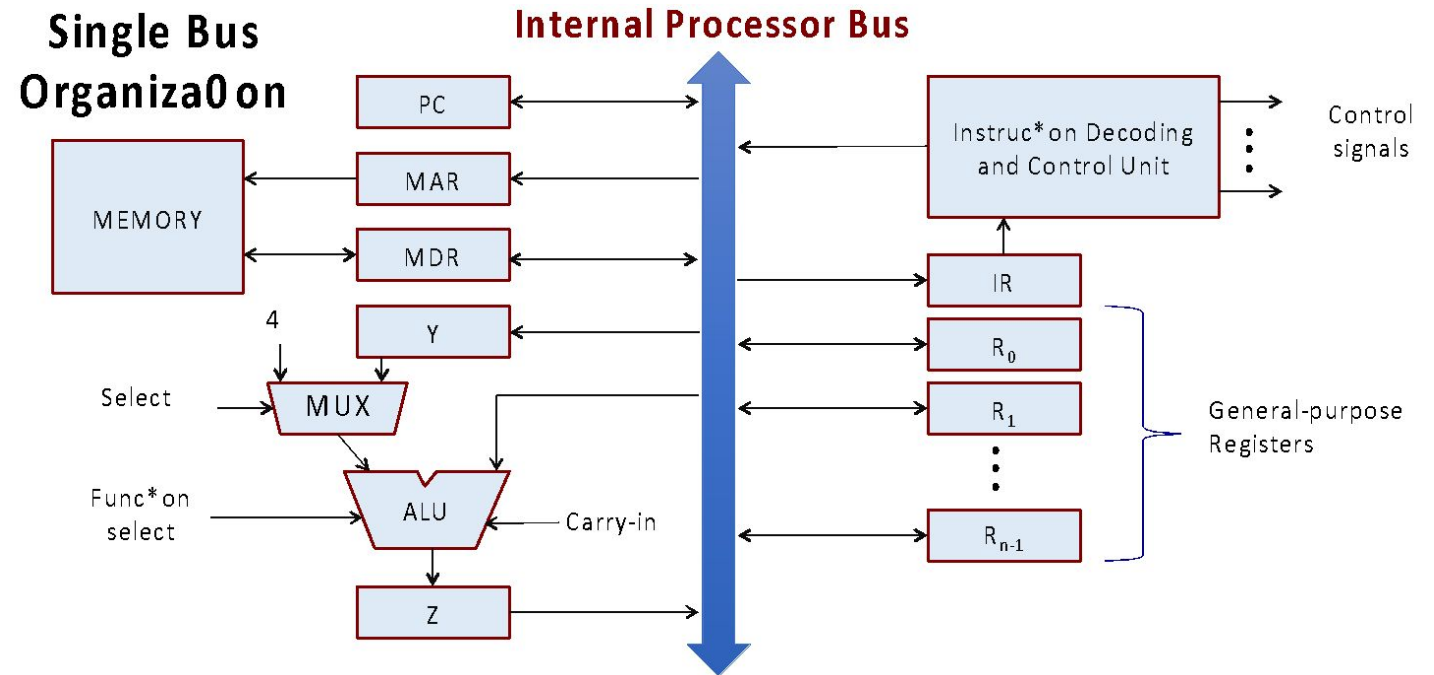
12. HALT

T1: PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}

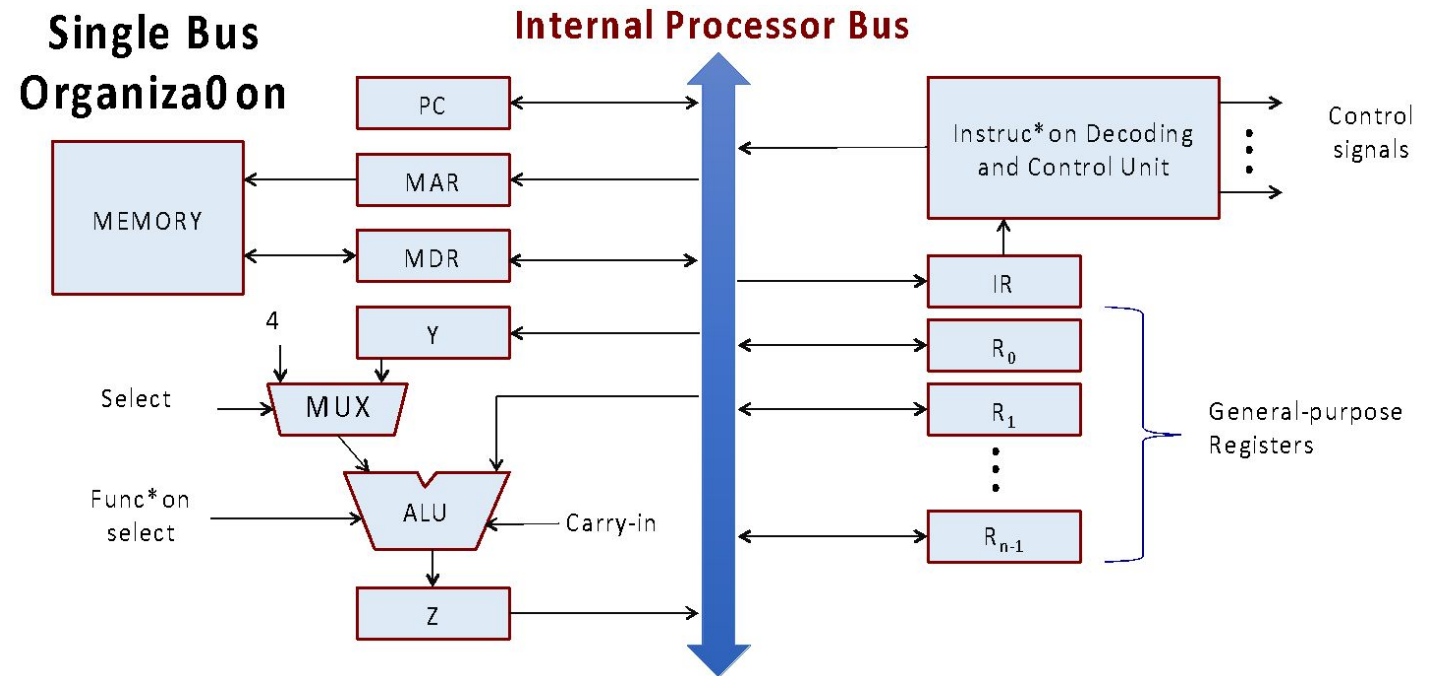
T2: Z_{out} , PC_{in} , Y_{in} , MDR_{inE} , WMFC

T3: MDR_{out} , IR_{in}

T4: END

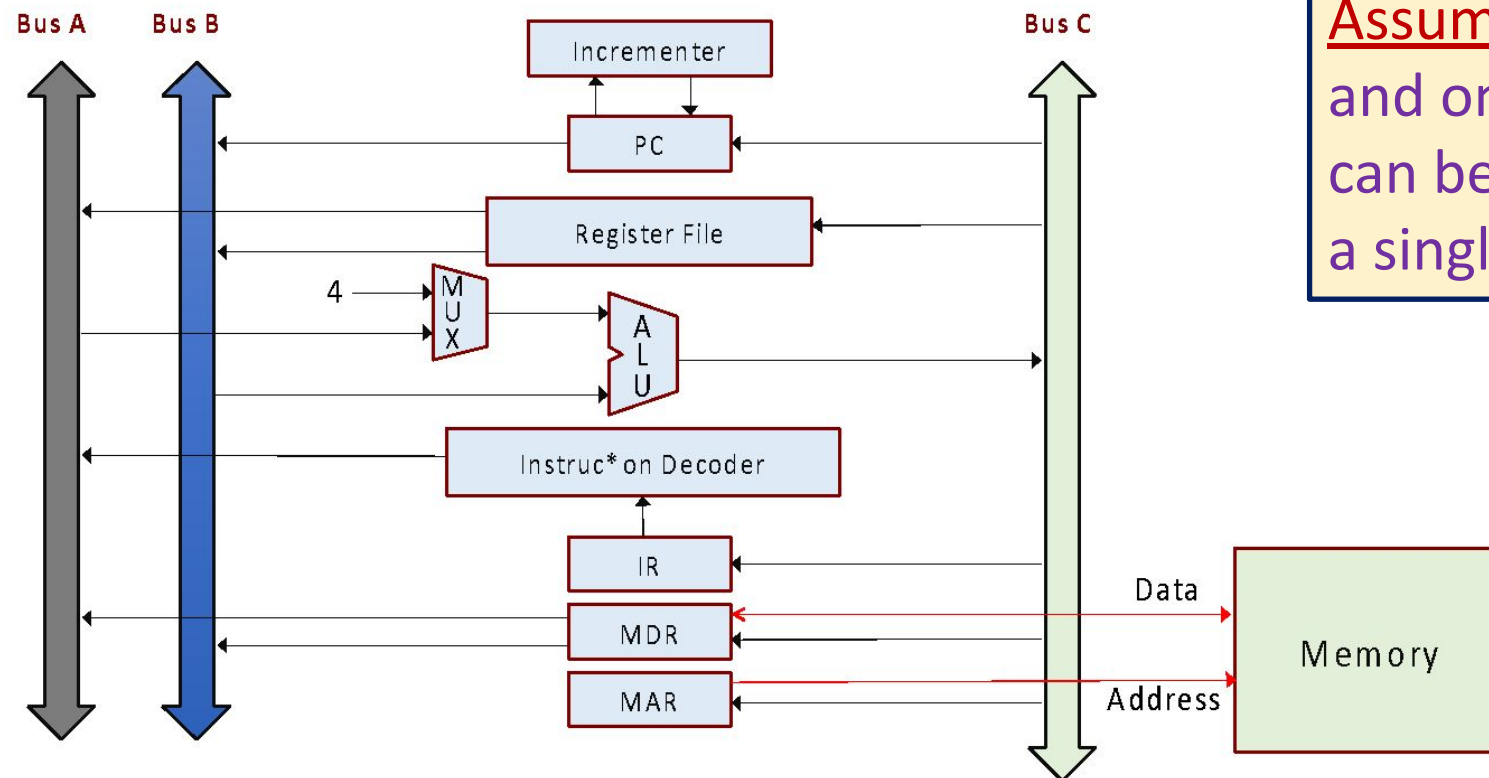


ADD (R3), R1



Practice Assignment 1

- Write down the micro-operations for the 12 instructions with respect to the 3-bus architecture.



Assumption: Two register read and one register write operations can be carried out concurrently in a single time step.

Practice Assignment 2

- Complete the following table for both the 1-bus and 3-bus architectures.
 - Ins-1 to Ins-12 indicates the instructions.
 - The entries in the table will contain the corresponding micro-operations.

	Ins-1	Ins-2	Ins-3	Ins-4	Ins-5	Ins-6	Ins-7	Ins-8	Ins-9	Ins-10	Ins-11	Ins-12
T1												
T2												
T3												
T4												
T5												
T6												
T7												

Hardwired and Microprogrammed Control Unit Design

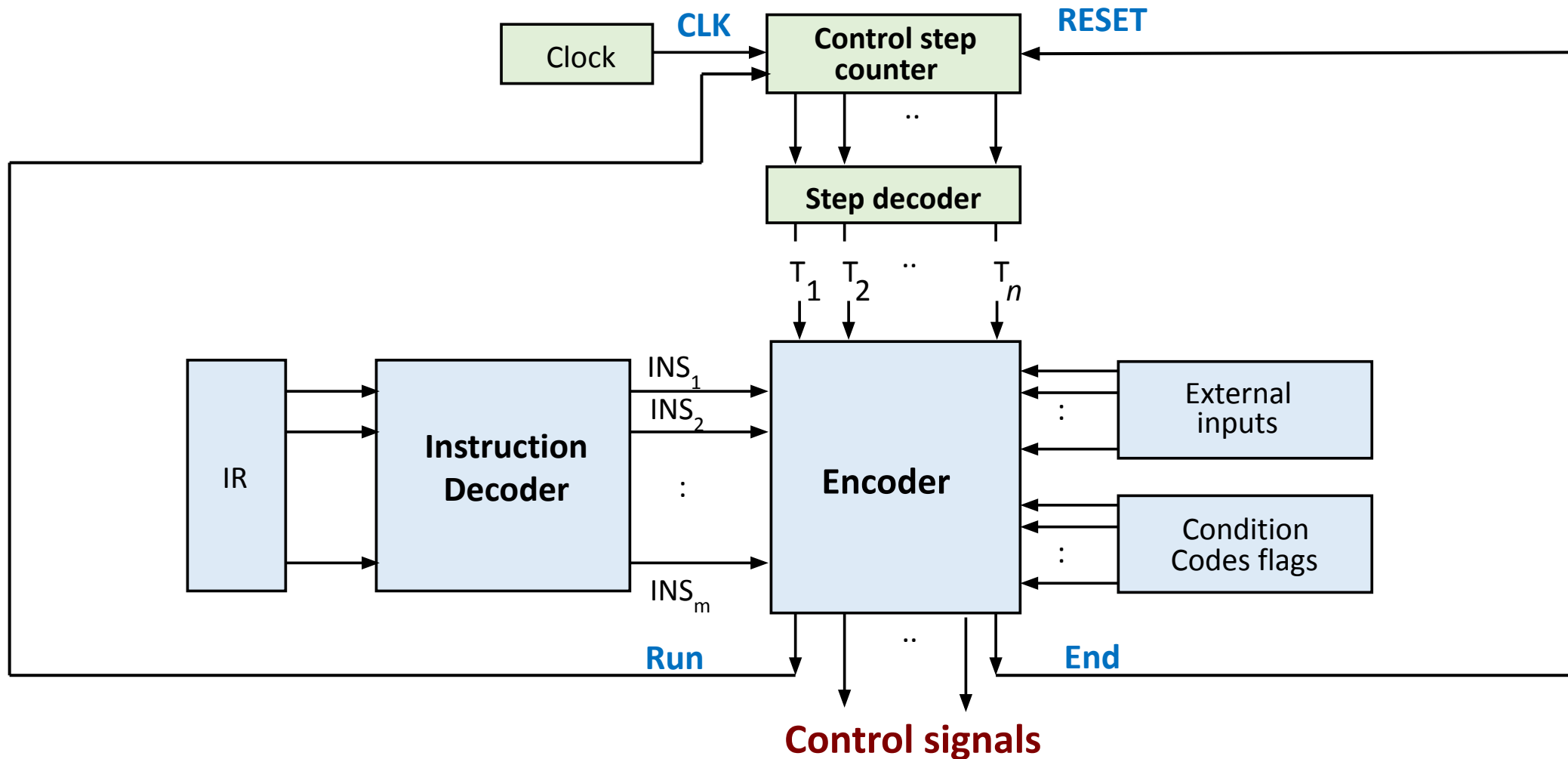
Introduction

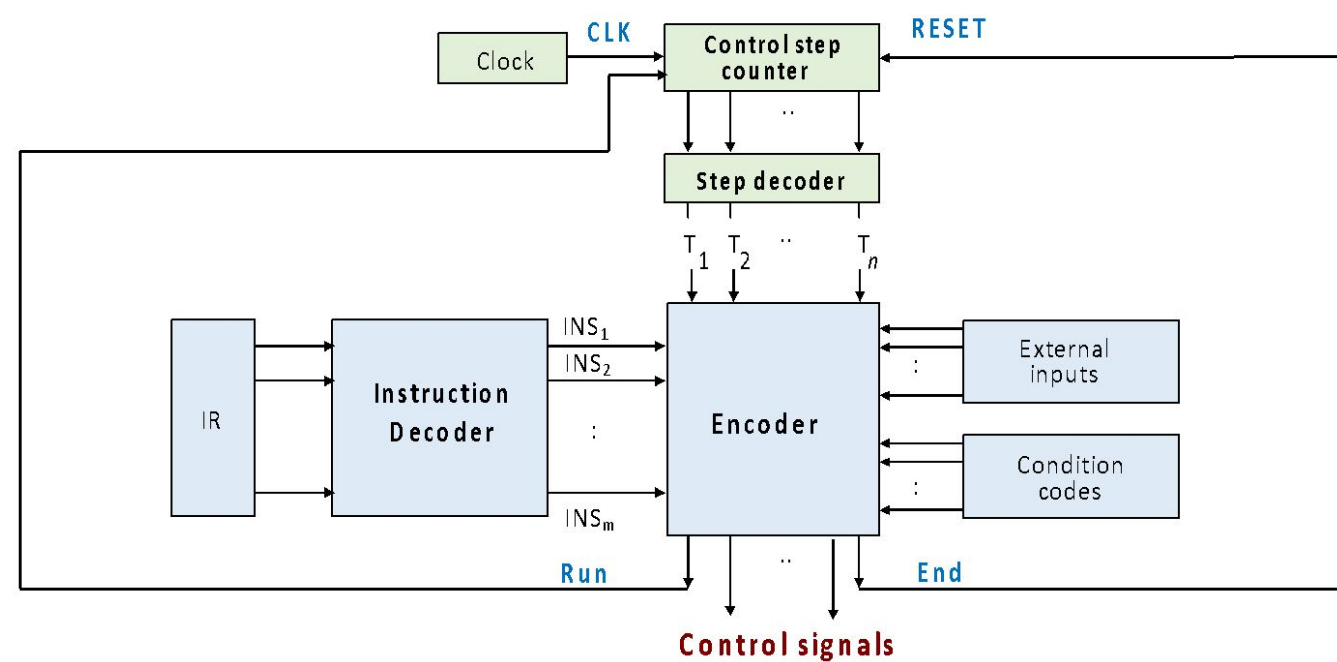
- To execute an instruction, the processor must generate control signals for the data path in proper sequence.
 - Example: **ADD R1, R2**
 - a) $R1_{out}, Y_{in}, SelectY$
 - b) $R2_{out}, ADD, Z_{in}$
 - c) $Z_{out}, R1_{in}$
- Two alternate approaches:
 1. Hardwired control unit design
 2. Microprogrammed control unit design

Hardwired Control Unit Design

- What does the control unit do?
 - It generates the control signals in proper sequence so as to execute any given instruction.
 - We have already seen some example control signals for some instructions.
- Basic question?
 - For an instruction I_k and time step T_m , what are the control signals that need to be generated?
- What is hardwired control unit?
 - The control signals are generated by specific hardware circuits that cannot be easily modified.
 - It is fixed or hardwired for a given <architecture, ISA> pair.

Overall Schematic Diagram



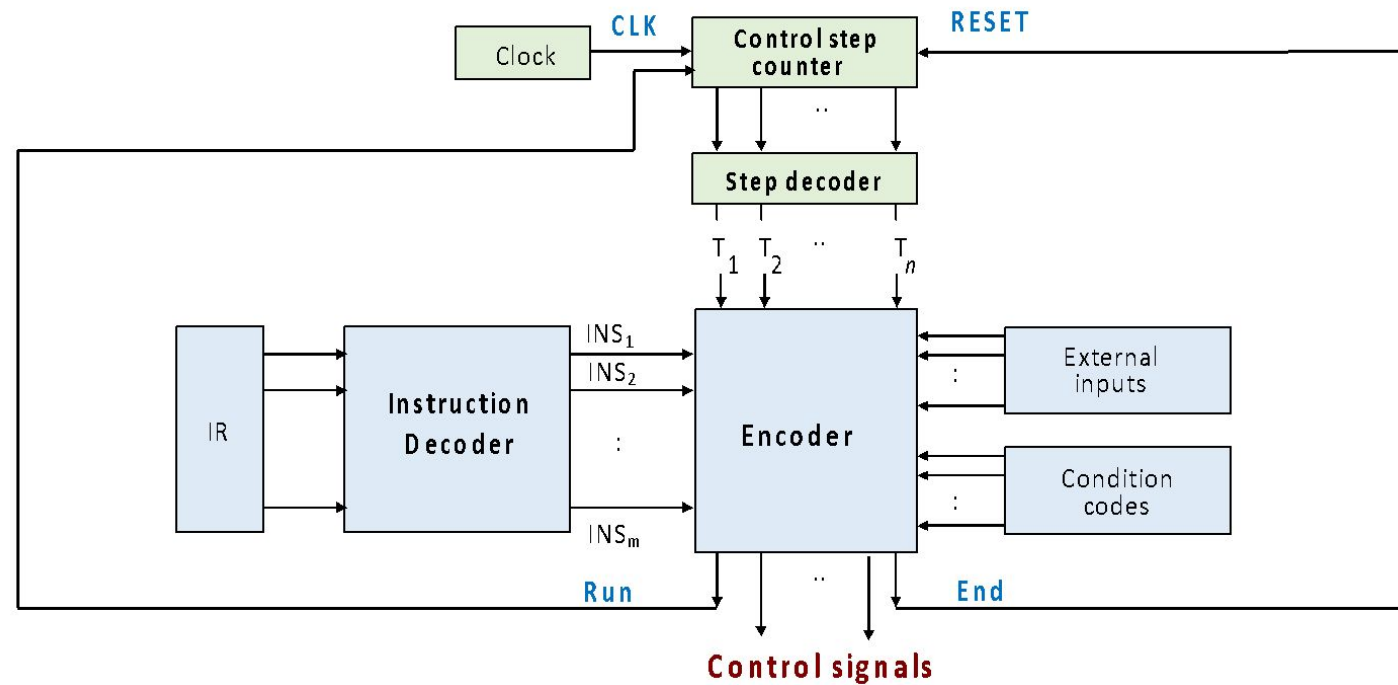


- Basic Principle of Operation:

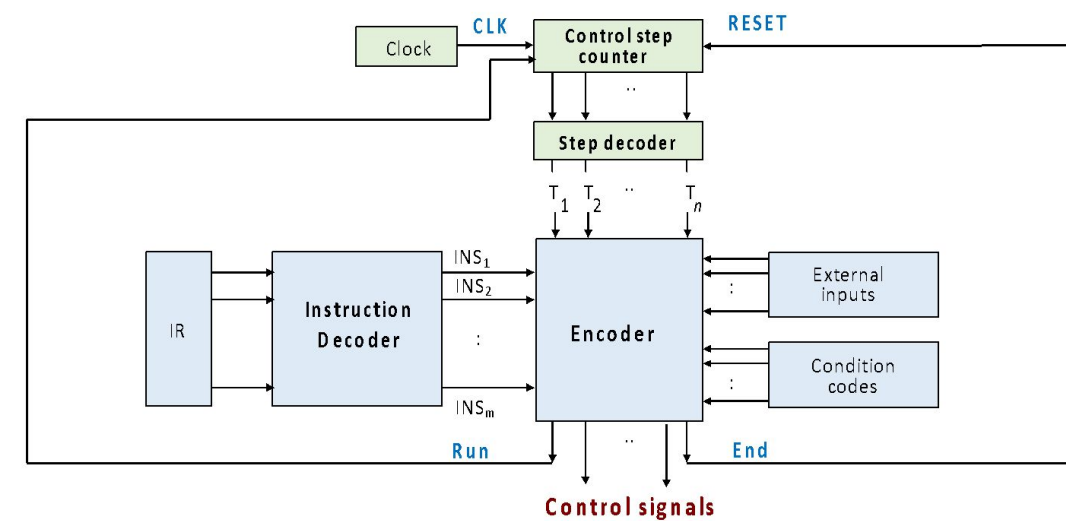
- The *step decoder* provides separate signal for each step (T_1, T_2, T_3 , etc.)..
- The *instruction decoder* has separate output lines for each machine instruction.
- For any instruction loaded in IR, one of the output line $INS_1, \dots, INS_m$ is set to 1 (others are 0).
- The input signals to the *encoder* block are combined to generate individual control signals.
- The signal **RUN** causes the counter to be incremented by 1, stops counting when **RUN = 0**. This is needed when the WMFC signal is issued, causing the processor wait for a memory reply.

- Assumption:

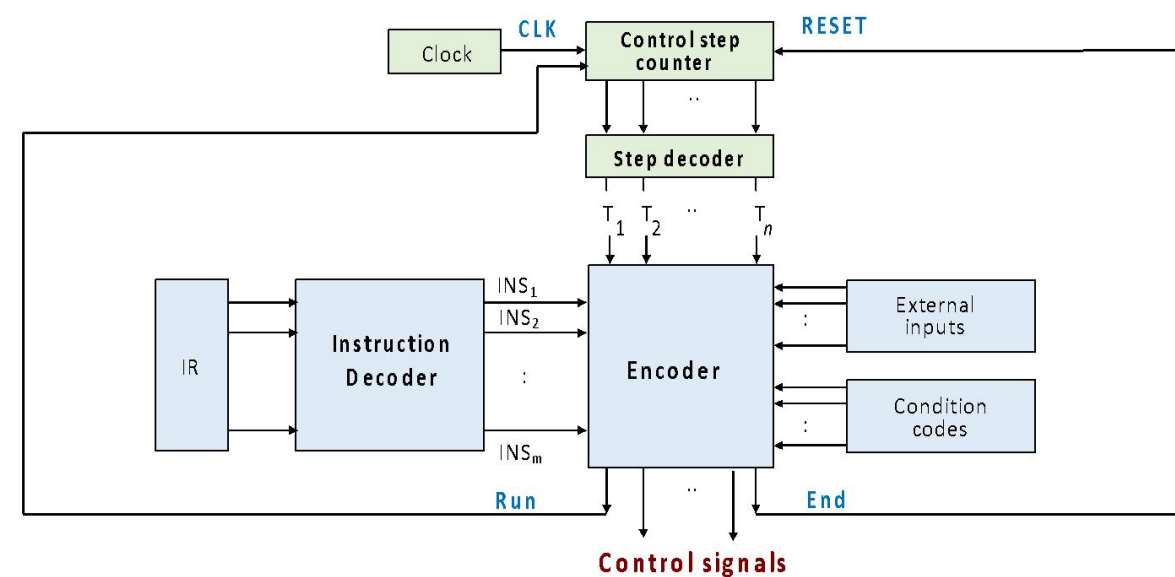
- Each time step is completed in one clock cycle.
- The control step counter keeps track of the time step.



- The control signals are determined by the following information:
 - Content of control step counter
 - Content of instruction register
 - Content of conditional code flags
 - External input signals such as WMFC



- The *Encoder* circuit is a combinational circuit which generates control signals depending on the inputs provided.
- The *Step Decoder* generates separate signal lines for each step in the control sequence (T₁, T₂, T₃, etc.).
 - Depending on maximum steps required for an instruction, the step decoder is designed.
 - If a maximum of 10 steps are required, then a 4 x 16 step decoder is used.
- Among the total set of instructions, the instruction decoder is used to select one of them. (That particular line will be 1 and rest will be 0).
 - If a maximum of 100 instructions are present in the ISA, then a 7 x 128 instruction decoder is used.



- At every clock cycle the **RUN** signal is used to increment the counter by one.
 - When RUN is 0 the counter stops counting.
 - This signal is needed when **WMFC** is issued.
- The **END** signal starts a new instruction.
 - It resets the control step counter to its starting value.
- The sequence of operations carried out by the control unit is determined by the wiring of the logic elements and hence it is named **hardwired**.
 - This approach of control unit design is fast but limited to the complexity of instruction set that is implemented.

Generation of Control Signals (Hardwired Control)

ADD R1,R2

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
T2: Z_{out}, PC_{in}, Y_{in}, WMFC
T3: MDR_{out}, IR_{in}
T4: R1_{out}, Y_{in}
T5: R2_{out}, SelectY, Add, Z_{in}
T6: Z_{out}, R1_{in}, End

BR Label

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
T2: Z_{out}, PC_{in}, Y_{in}, WMFC
T3: MDR_{out}, IR_{in}
T4: Offset-field-of-IR_{out}, SelectY, Add, Z_{in}
T5: Z_{out}, PC_{in}, End

ADD R1,LOCA

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
T2: Z_{out}, PC_{in}, Y_{in}, WMFC
T3: MDR_{out}, IR_{in}
T4: Address field of IR_{out}, MAR_{in}, Read
T5: R1_{out}, Y_{in}, WMFC
T6: MDR_{out}, SelectY, Add, Z_{in}
T7: Z_{out}, R1_{in}, End

LOAD R1,LOCA

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
T2: Z_{out}, PC_{in}, Y_{in}, WMFC
T3: MDR_{out}, IR_{in}
T4: Address field of IR_{out}, MAR_{in}, Read
T5: WMFC
T6: MDR_{out}, R1_{in}, End

INC R1

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
T2: Z_{out}, PC_{in}, Y_{in}, WMFC
T3: MDR_{out}, IR_{in}
T4: R1_{out}, Select4, Add, Z_{in}
T5: Z_{out}, R1_{in}, End

	ADD R1,R2	ADD R1,LOCA	LOAD R1,LOCA	BR Label	INC R1
	I1	I2	I3	I4	I5
T1	PC _{out} , MAR _{in} , Read, Select4, ADD, Z _{in}	PC _{out} , MAR _{in} , Read, Select4, ADD, Z _{in}	PC _{out} , MAR _{in} , Read, Select4, ADD, Z _{in}	PC _{out} , MAR _{in} , Read, Select4, ADD, Z _{in}	PC _{out} , MAR _{in} , Read, Select4, ADD, Z _{in}
T2	Z _{out} , PC _{in} , Y _{in} , WMFC	Z _{out} , PC _{in} , Y _{in} , WMFC	Z _{out} , PC _{in} , Y _{in} , WMFC	Z _{out} , PC _{in} , Y _{in} , WMFC	Z _{out} , PC _{in} , Y _{in} , WMFC
T3	MDR _{out} , IR _{in}	MDR _{out} , IR _{in}	MDR _{out} , IR _{in}	MDR _{out} , IR _{in}	MDR _{out} , IR _{in}
T4	R1 _{out} , Y _{in}	Address field of IR _{out} , MAR _{in} , Read	Address field of IR _{out} , MAR _{in} , Read	Offset field of IR _{out} , SelectY, ADD, Z _{in}	R1 _{out} , Select4, ADD, Z _{in}
T5	R2 _{out} , SelectY, ADD, Z _{in}	R1 _{out} , Y _{in} , WMFC	WMFC	Z _{out} , PC _{in} , END	Z _{out} , R1 _{in} , END
T6	Z _{out} , R1 _{in} , END	MDR _{out} , SelectY, ADD, Z _{in}	MDR _{out} , R1 _{in} , END		
T7		Z _{out} , R1 _{in} , END			

Logic for Generation of Some of the Control Signals

$$PC_{in} = T2 + T5.I4$$

$$PC_{out} = T1$$

$$MAR_{in} = T1 + T4.(I2 + I3)$$

$$Z_{in} = T1 + T4.(I4 + I5) + T6.I2$$

$$Z_{out} = T2 + T5.(I4 + I5) + T6.I1 + T7.I2$$

$$MDR_{out} = T3 + T6.(I2 + I3)$$

$$IR_{in} = T3$$

$$Y_{in} = T2 + T4.I1 + T5.I2$$

$$Select4 = T1 + T4.I5$$

$$SelectY = T4.I4 + T5.I1 + T6.I2$$

$$END = T5.(I4 + I5) + T6.(I1 + I3) + T7.I2$$

$$Rx_{in} = T5.I5 + T6.(I1 + I3) + T7.I2$$

$$Rx_{out} = T4.(I1 + I5) + T5.(I1 + I2)$$

$$WMFC = T2 + T5.(I2 + I3)$$

Design of the Encoder Block

- Implement the logic functions to generate the control signals.
 - Using logic gates or any other hardware module.
- This method of generating control signals is very fast.

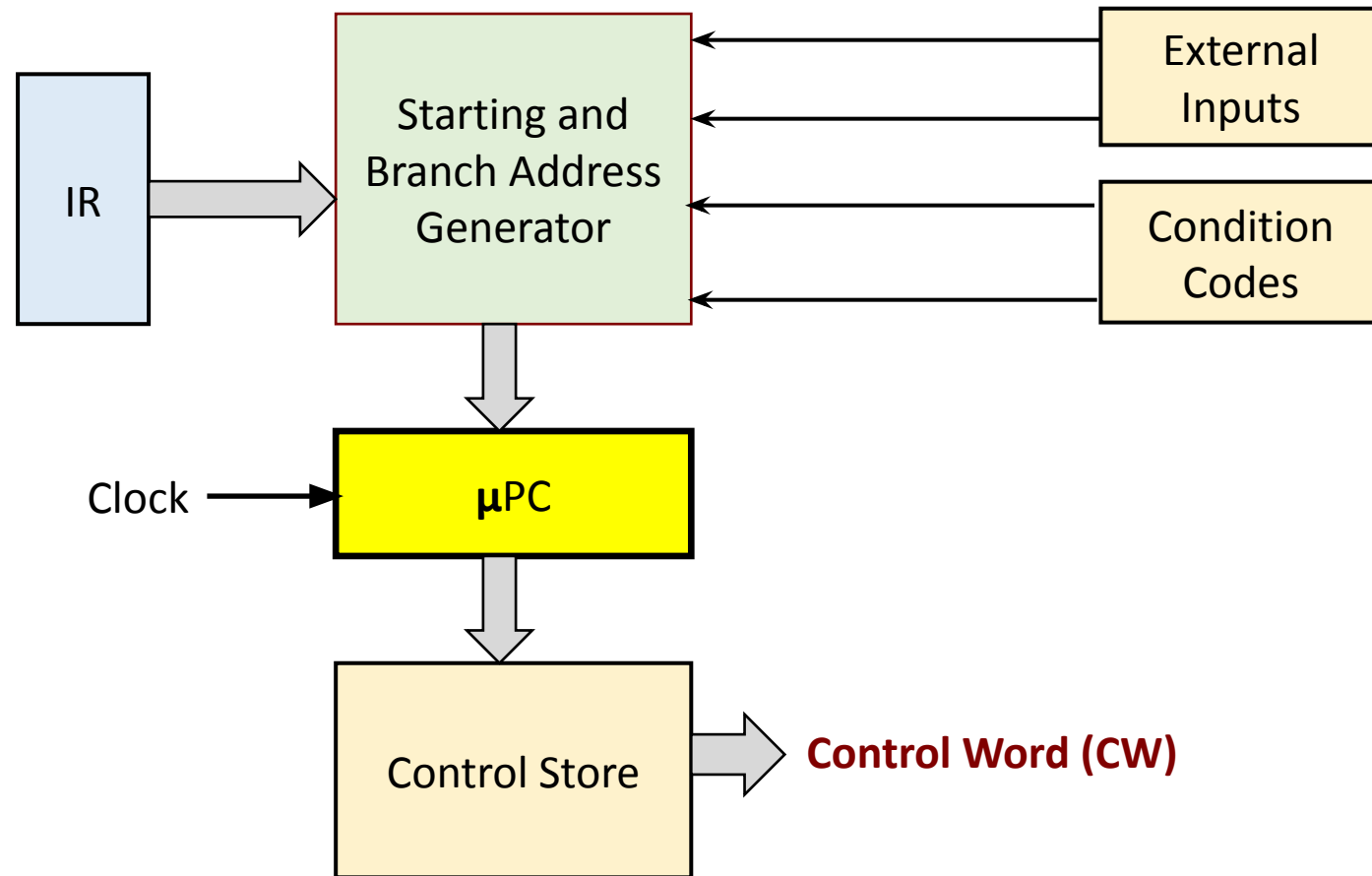
Practice Assignment 3

- Design the hardwired control unit for the 12 instructions as given for:
 - a) 1-bus architecture
 - b) 3-bus architecture

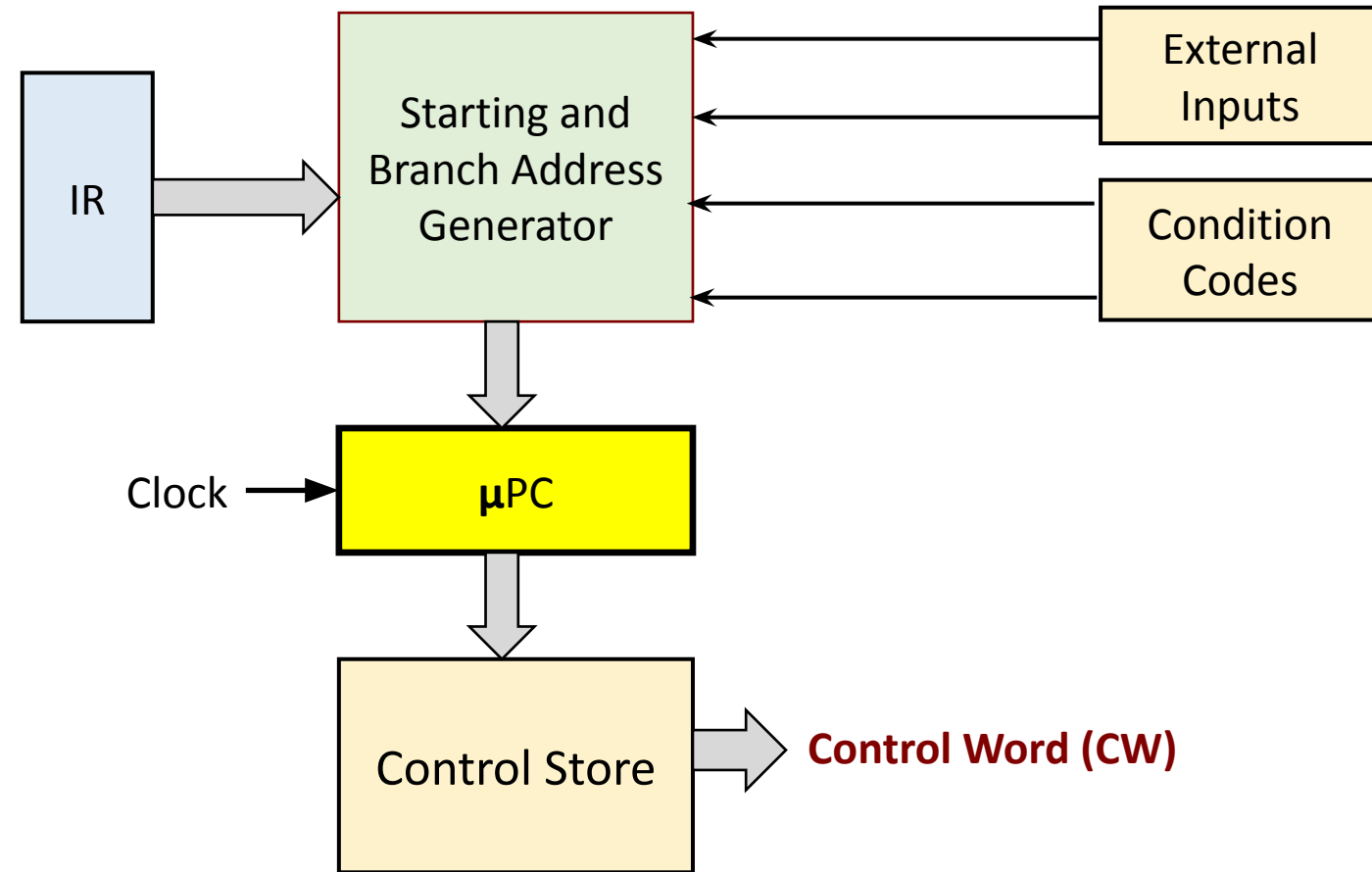
Generation of Control Signals (Microprogrammed Control)

Microprogrammed Control Unit Design

- Control signals are generated by a program similar to machine language program.
- A *Control Store* (CS) stores the microroutines for all instructions of an ISA.
- The sequence of steps corresponding to the control sequence of a machine instruction is the *microroutine*.
- Each sequence of steps is a *Control Word* (CW) whose individual bits represent the various control signals.
- Individual control words in a microroutine are called *microinstructions*.



- Control-unit generates the control signals for an instruction by sequentially reading CWs of corresponding microroutine from CS.
- The *micro-program counter* (μ PC) is used to read CWs sequentially from CS.
- Every time a new instruction is loaded into IR, output of *Starting Address Generator* is loaded into μ PC.
- Then, μ PC is automatically incremented by *clock* causing successive microinstructions to be read from CS.



Control Store Contents for “ADD R1, R2”

Micro- instr.	...	PC in	PC out	M A R _{in}	Re ad	M D R _o ut	IR _i n	Y _{in}	Se le ct	Ad d	Z _{in}	Z _o ut	R1 out	R1 in	R2 out	W M FC	En d	...
1	0	0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	0
2	0	1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	0
3	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	0	0	0
6	0	0	0	0	T1: PC _{out} , MAR _{in} , Read, Select4, Add, Z _{in}								0	1	0	0	1	0

T2: Z_{out}, PC_{in}, Y_{in}, WMFC

T3: MDR_{out}, IR_{in}

T4: R1_{out}, Y_{in}

T5: R2_{out}, SelectY, Add, Z_{in}

T6: Z_{out}, R1_{in}, End

Control Store Contents for “BR Label”

Micro-instr.	...	PC _{in}	PC _{out}	MAR _{in}	Read	MDR _{out}	IR _{in}	Y _{in}	Select	Add	Z _{in}	Z _{out}	IR _{out}	WMFC	End	...
1	0	0	1	1	1	0	0	0	1	1	1	0	0	0	0	0
2	0	1	0	0	0	0	0	1	0	0	0	1	0	1	0	0
3	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0
5	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0

T1: PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}

T2: Z_{out}, PC_{in}, Y_{in}, WMFC

T3: MDR_{out}, IR_{in}

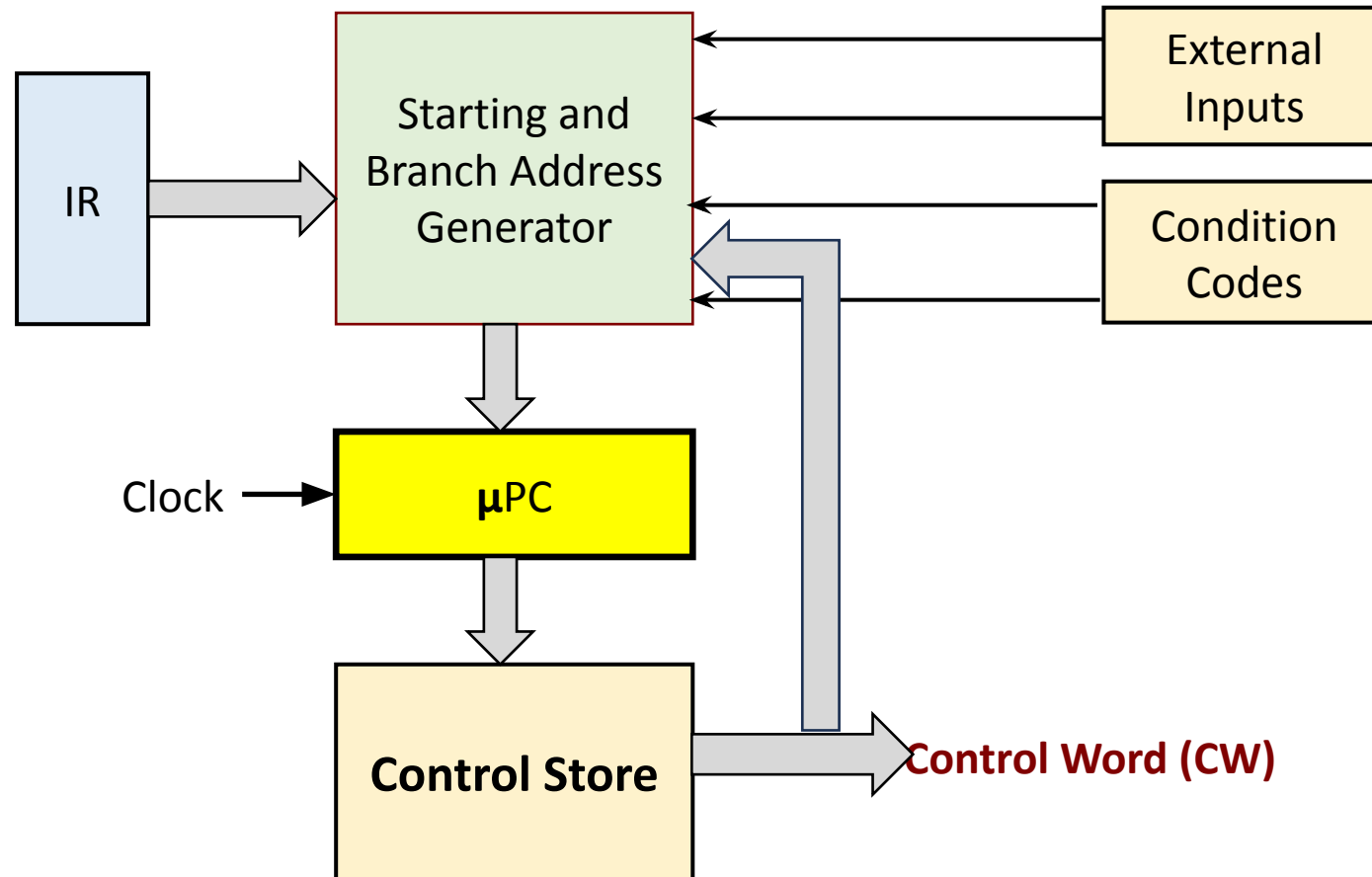
T4: Offset-field-of-IR_{out}, SelectY, Add, Z_{in}

T5: Z_{out}, PC_{in}, End

Using Conditional Branching

- The previous organization cannot handle the situation when the control unit is required to check the status of the condition codes or external inputs to choose between alternative courses of action.
- Possible solution:
 - Use conditional branch *microinstruction*.
 - An example is shown on the next slide.

Address	Microinstruction
0	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
1	Z_{out} , PC_{in} , Y_{in} , WMF
2	MDR_{out} , IR_{in} C
3	Branch to starting address of appropriate microroutine
.....	
25	If $Z=0$, then branch to microinstruction
26	Offset-field-of- IR_{out} , $SelectY$, Add, Z_{in}
27	Z_{out} , PC_{in} , End



Microprogram Sequencing

- If all microprograms require only straightforward sequential execution of microinstructions except for branches, letting a μ PC govern the sequencing would be efficient.
- However, two disadvantages:
 - Having a separate microroutine for each machine instruction results in a large total number of microinstructions and a large control store.
 - Longer execution time because it takes more time to carry out the required branches.

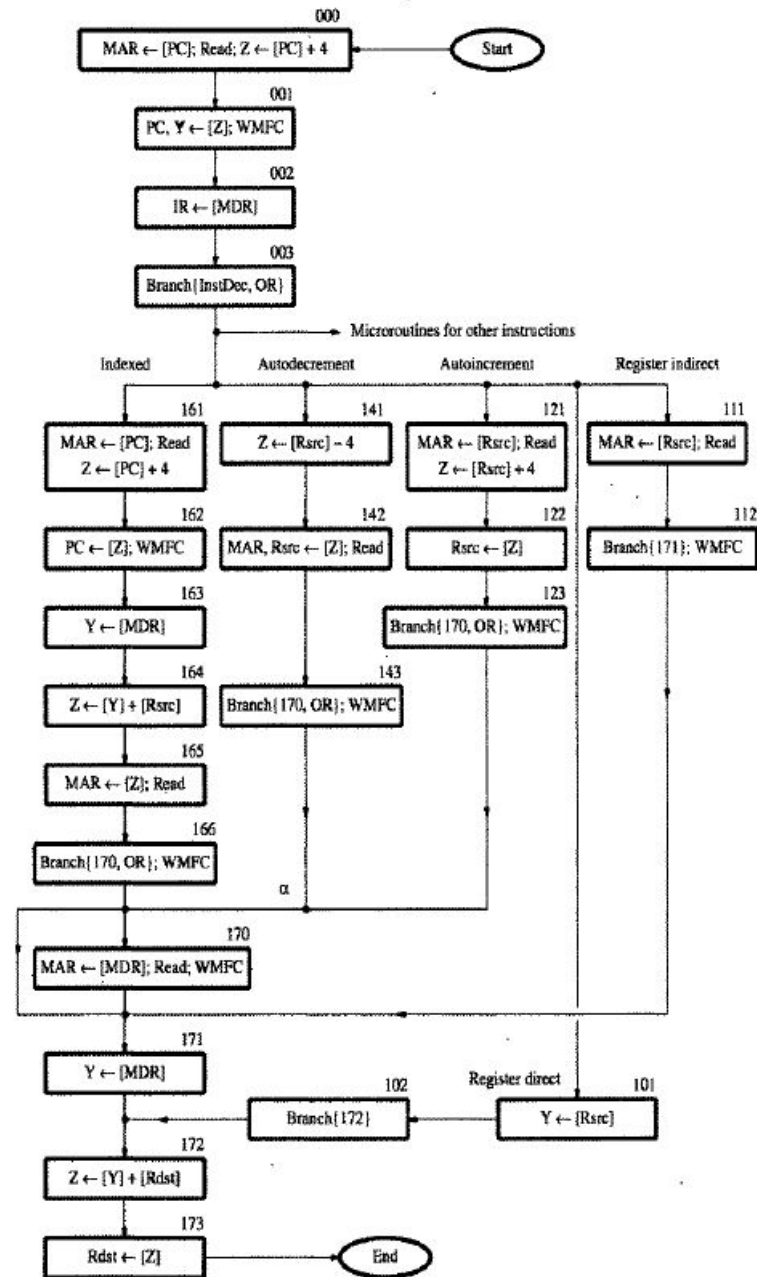
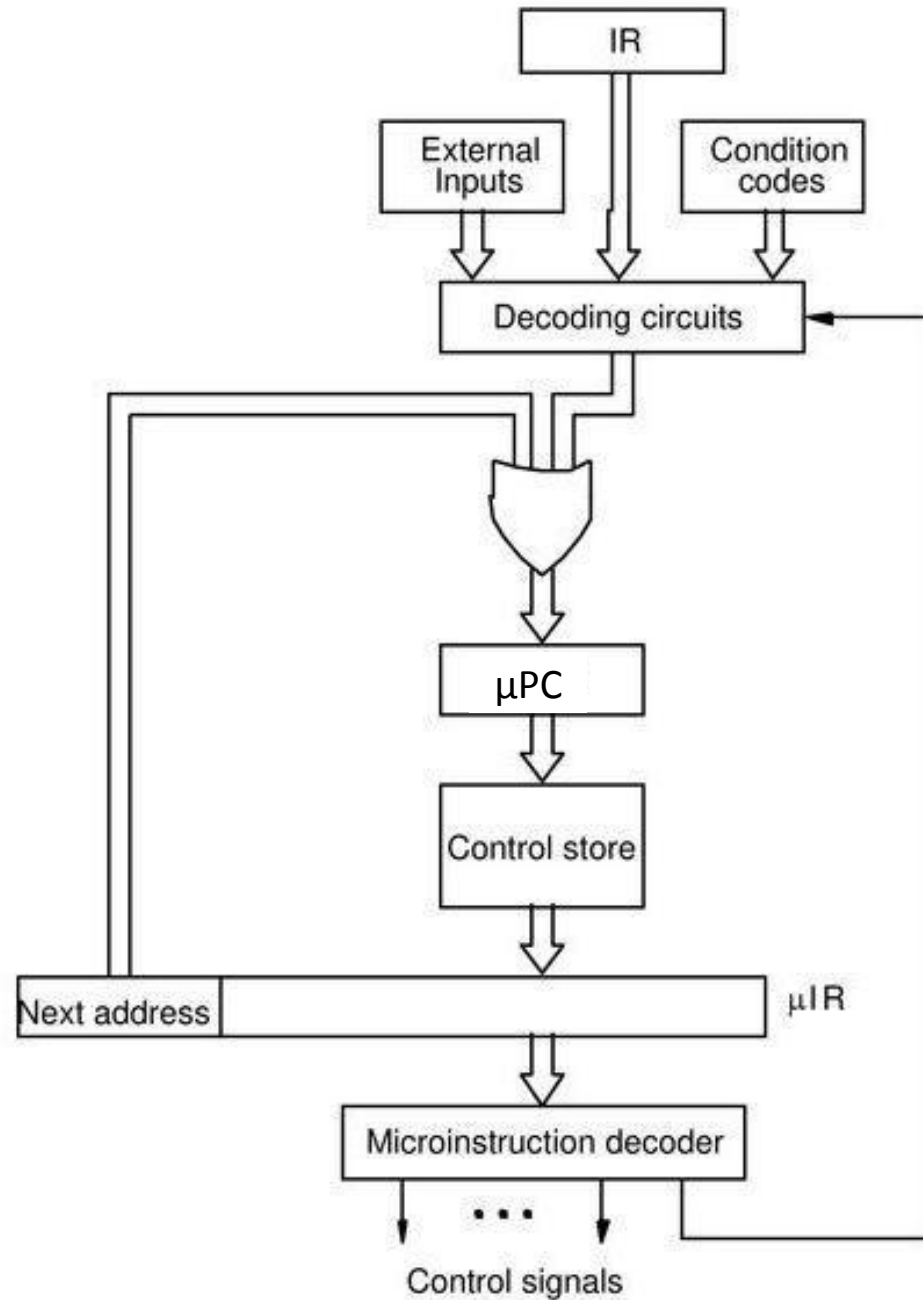


Figure 7.20 Flowchart of a microprogram for the Add src, Rdst instruction.

Microinstructions with Next-Address Field

- The microprogram requires several branch microinstructions, which perform no useful operation in the datapath.
- An alternative approach:
 - Include an address field as a part of every microinstruction to indicate the location of the next microinstruction to be fetched.
- **Pros:** separate branch microinstructions are virtually eliminated; few limitations in assigning addresses to microinstructions.
- **Cons:** additional bits required for the address field.



From the instruction opcode, some constant bits are appended (both at the left and right) to get the starting address of the corresponding micro-routine in control store.

Illustrative Example

- Some example instructions and their 4-bit opcodes:

	<i>Opcodes</i>	<i>Starting Address of Micro-routine</i>
ADD	0010	0 0010 000
SUB	0011	0 0011 000
LOAD	0100	0 0100 000
STORE	0101	0 0101 000
BR	1010	0 1010 000

Practice Assignment 4

Design the microprogrammed control unit for the 12 instructions as given for the 1-bus architecture.

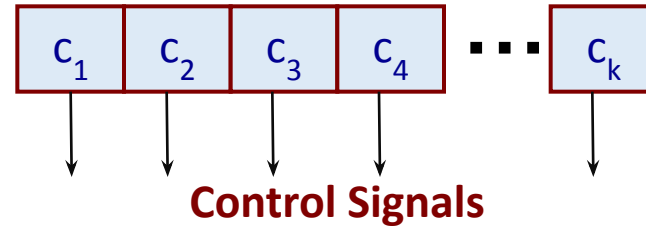
In particular, show the micro-routines for the various instructions, and the detailed contents of CS.

Horizontal versus Vertical Microinstruction Encoding

- Broadly there are two alternate schemes to code the control signals in the control memory.
 - a) **Horizontal Micro-instruction Encoding**
 - Each control signal is represented by a bit in the micro-instruction.
 - Fewer control store words, with more bits per word.
 - b) **Vertical Micro-instruction Encoding**
 - Each control word represents a single micro-instruction in encoded form.
 - k-bit control word can support up to 2^k micro-instructions.
 - More control store words, with fewer bits per word.

- There can be a tradeoff between horizontal and vertical micro-instruction encoding.
 - Sometimes referred to as ***Diagonal Micro-instruction Encoding***.
 - The control signals are grouped into sets S_1 , S_2 , etc., such that the control signals within a set are mutually exclusive.
- **Summary:**
 - Horizontal encoding supports unlimited parallelism among micro-instructions.
 - Vertical encoding supports strictly sequential execution of micro-instructions.
 - Diagonal encoding does not sacrifice the required level of parallelism, but uses less number of bits per control word as compared to horizontal encoding.

(a) Horizontal Micro-instruction Encoding



- Suppose that there are k control signals: $c_1, c_2, \dots, c_k$.
- In horizontal encoding, every control word stored in control memory (CM) consists of k bits, one bit for every control signal.
- Several bits in a control word can be 1:
 - Parallel activation of several micro-operations in a single time step.



□ c_2, c_4 and c_5 are activated together

- Advantage:

- Unlimited parallelism is possible in the activation of the micro-operations.

- Disadvantage:

- Size of the control memory is large (word size is much longer).
 - Cost of implementation is higher.

Example

few

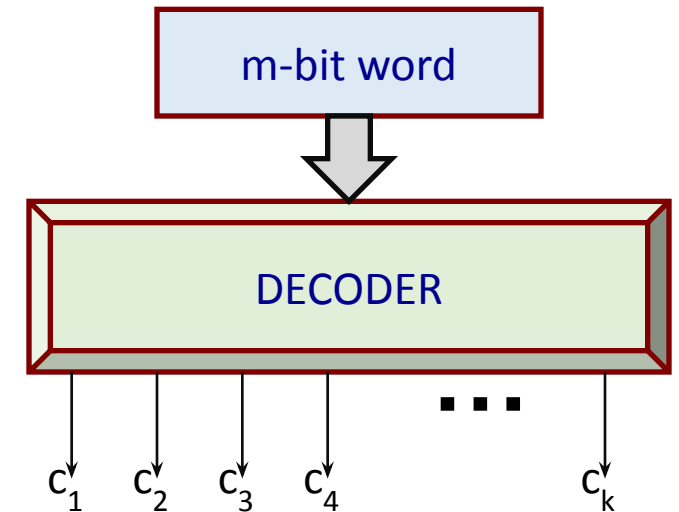
F1 (4 bits)	F2 (3 bits)	F3 (3 bits)	F4 (4 bits)	F5 (2 bits)
0000: No transfer	000: No transfer	000: No transfer	0000: Add	00: No action
0001: PC _{out}	001: PC _{in}	001: MAR _{in}	0001: Sub	01: Read
0010: MDR _{out}	010: IR _{in}	010: MDR _{in}	:	10: Write
0011: Z _{out}	011: Z _{in}	011: TEMP _{in}	:	
0100: R0 _{out}	100: R0 _{in}	100: Y _{in}	1111: XOR	
0101: R1 _{out}	101: R1 _{in}		16 ALU functions	
0110: R2 _{out}	110: R2 _{in}			
0111: R3 _{out}	111: R3 _{in}			
1010: TEMP _{out}				
1011: Offset _{out}				

F6	F7	F8	...
----	----	----	-----

F6 (1 bit)	F7 (1 bit)	F8 (1 bit)
0: SelectY	0: No action	0: Continue
1: Select4	1: WMFC	1: End

(b) Vertical Micro-instruction Encoding

- Again consider that there are k control signals: $c_1, c_2, \dots, c_k$.
- We encode the control signals in an m -bit word in the control memory, where $k \leq 2^m$.
- Depending on the m -bit control word, exactly one control signal will be activated ($= 1$), while all others will remain de-activated ($= 0$).
 - At most one control signal can be activated in a time step.



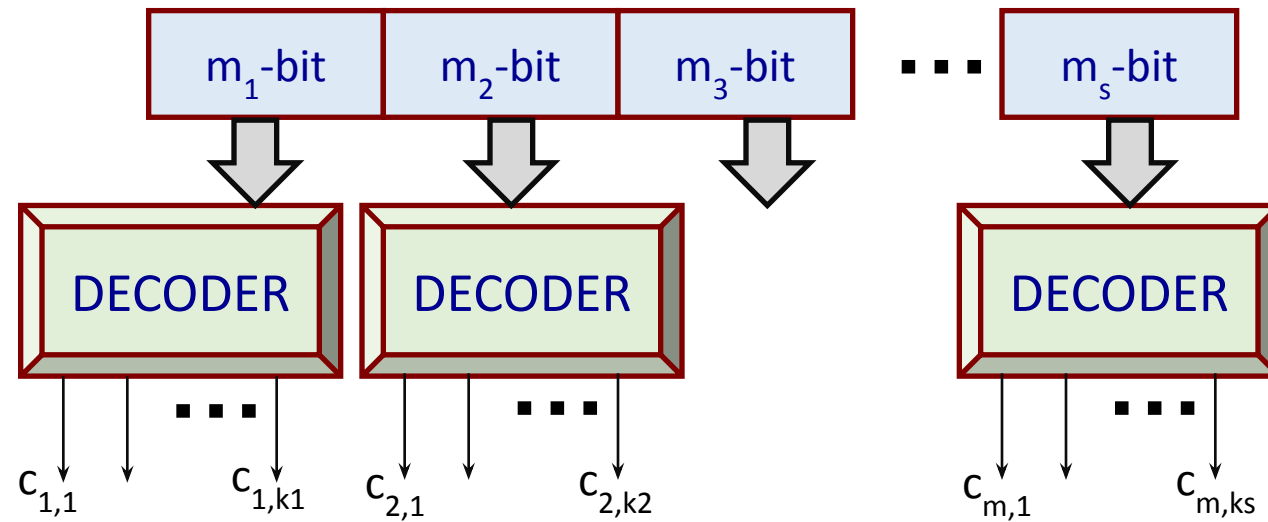
- Advantage:

- Requires much smaller word size in control memory.
- Low cost of implementation.

- Disadvantage:

- More than one control signals cannot be activated at a time.
- Requires sequential activation of the control signals, and hence more number of time steps.

(c) Diagonal Micro-instruction Encoding



- Suppose we group the set of k control signals into s groups, containing $k_1, k_2, \dots, k_s$ signals.
- We encode the control signals in groups as shown, where $k_i \leq 2^{m_i}$.
 - Within a group, at most one control signal can be activated in a time step.
 - Parallelism across groups is allowed.

- Advantages:

- Maximum parallelism as required by the micro-programs can be supported.
- Word size of control memory is less than that for horizontal encoding.
- Used in practice.

- Disadvantages:

- Multiple decoders (though smaller in sizes) are required.

Example 1

- Suppose there are 100 control signals in a processor data path.
 - a) For horizontal encoding, control word size = 100 bits.
 - b) For vertical encoding, control word size = $\lceil \log_2 100 \rceil = 7$ bits.
 - c) For diagonal encoding, suppose after analysis of the micro-programs, we divide the control signals into 5 groups, containing 25, 15, 40, 5 and 15 control signals respectively.
 - We have: $m_1 = 5, m_2 = 4, m_3 = 6, m_4 = 3, m_5 = 4$
 - Control word size = $5 + 4 + 6 + 3 + 4 = 22$ bits.

$$\begin{array}{ll} 25 \leq 2^5 & 15 \leq 2^4 \\ 40 \leq 2^6 & 5 \leq 2^3 \\ 15 \leq 2^4 & \end{array}$$

A Single Cycle Data Path

A small set of instructions

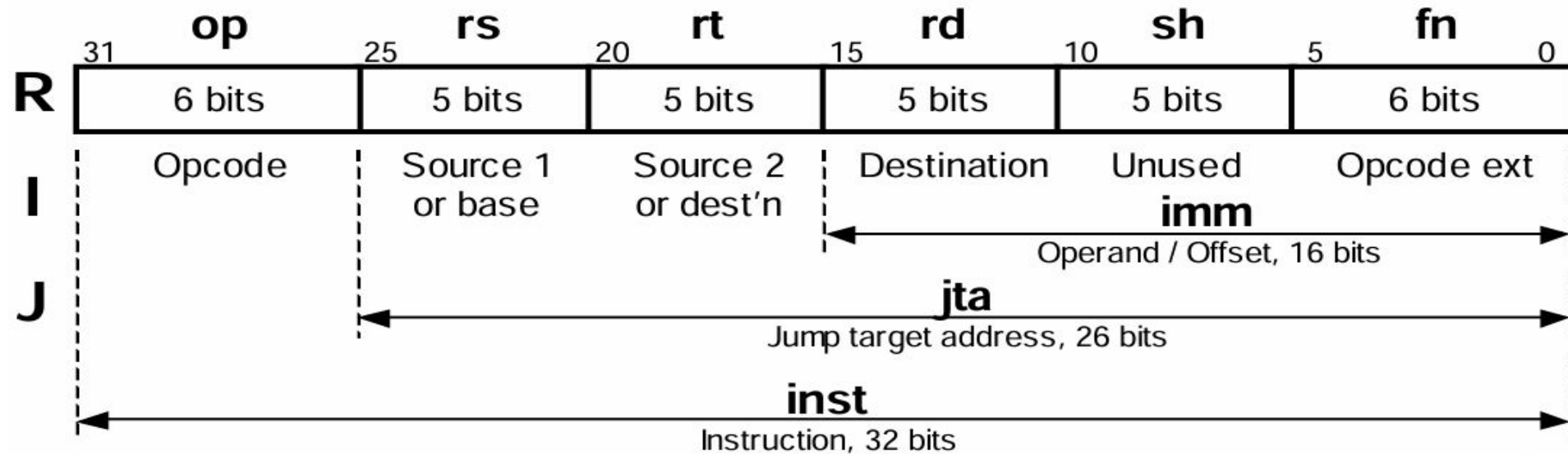


Fig. 13.1 MicroMIPS instruction formats and naming of the various fields.

We will refer to this diagram later

Seven R-format ALU instructions (add, sub, slt, and, or, xor, nor)

Six I-format ALU instructions (lui, addi, slti, andi, ori, xori)

Two I-format memory access instructions (lw, sw)

Three I-format conditional branch instructions (bltz, beq, bne)

Four unconditional jump instructions (j, jr, jal, syscall)

The instruction Set

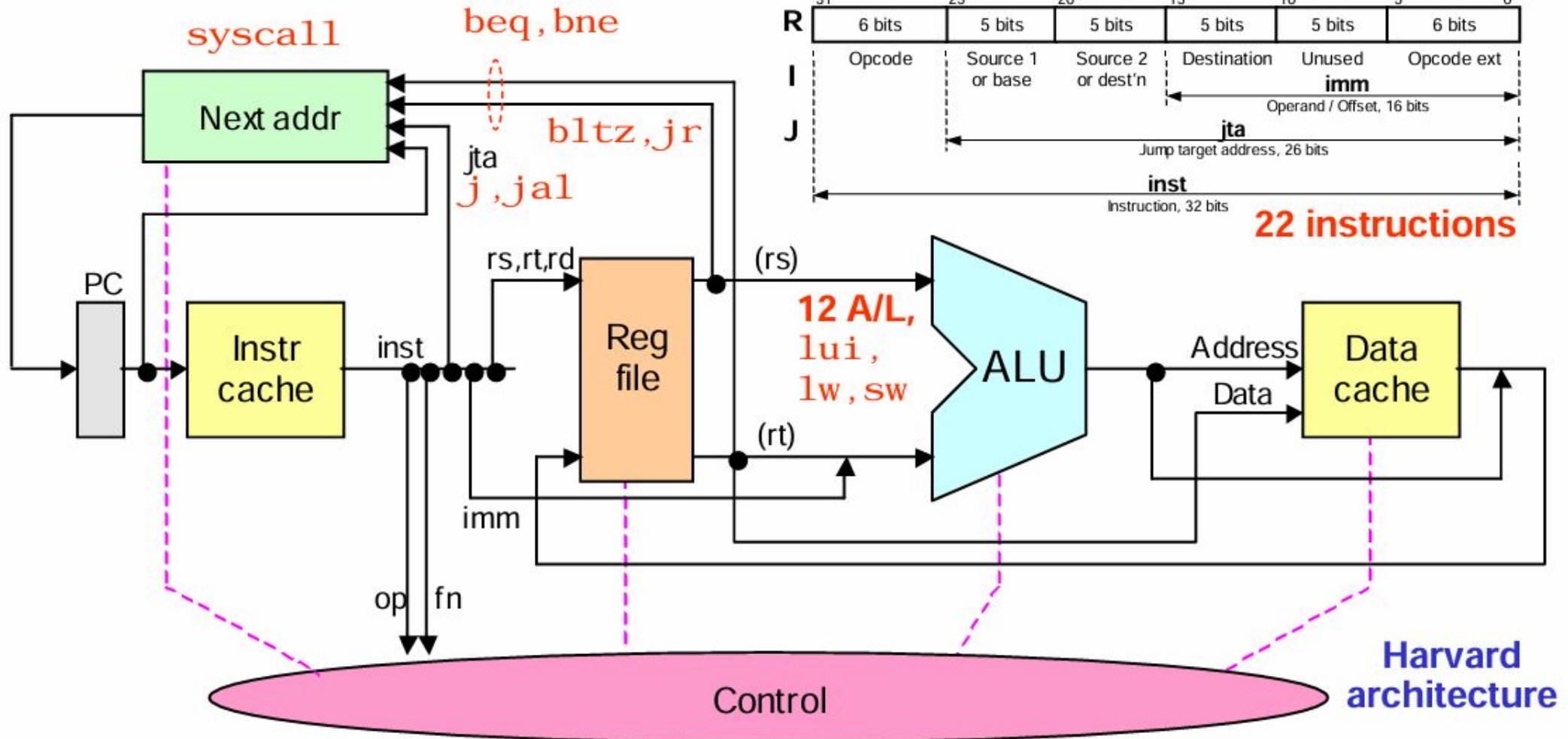
- Copy
- Arithmetic
- Logic
- Memory Access
- Control Transfer

Instruction	Usage
Load upper immediate	lui rt,imm
Add	add rd,rs,rt
Subtract	sub rd,rs,rt
Set less than	slt rd,rs,rt
Add immediate	addi rt,rs,imm
Set less than immediate	slti rd,rs,imm
AND	and rd,rs,rt
OR	or rd,rs,rt
XOR	xor rd,rs,rt
NOR	nor rd,rs,rt
AND immediate	andi rt,rs,imm
OR immediate	ori rt,rs,imm
XOR immediate	xori rt,rs,imm
Load word	lw rt,imm(rs)
Store word	sw rt,imm(rs)
Jump	j L
Jump register	jr rs
Branch less than 0	bltz rs,L
Branch equal	beq rs,rt,L
Branch not equal	bne rs,rt,L
Jump and link	jal L
System call	syscall

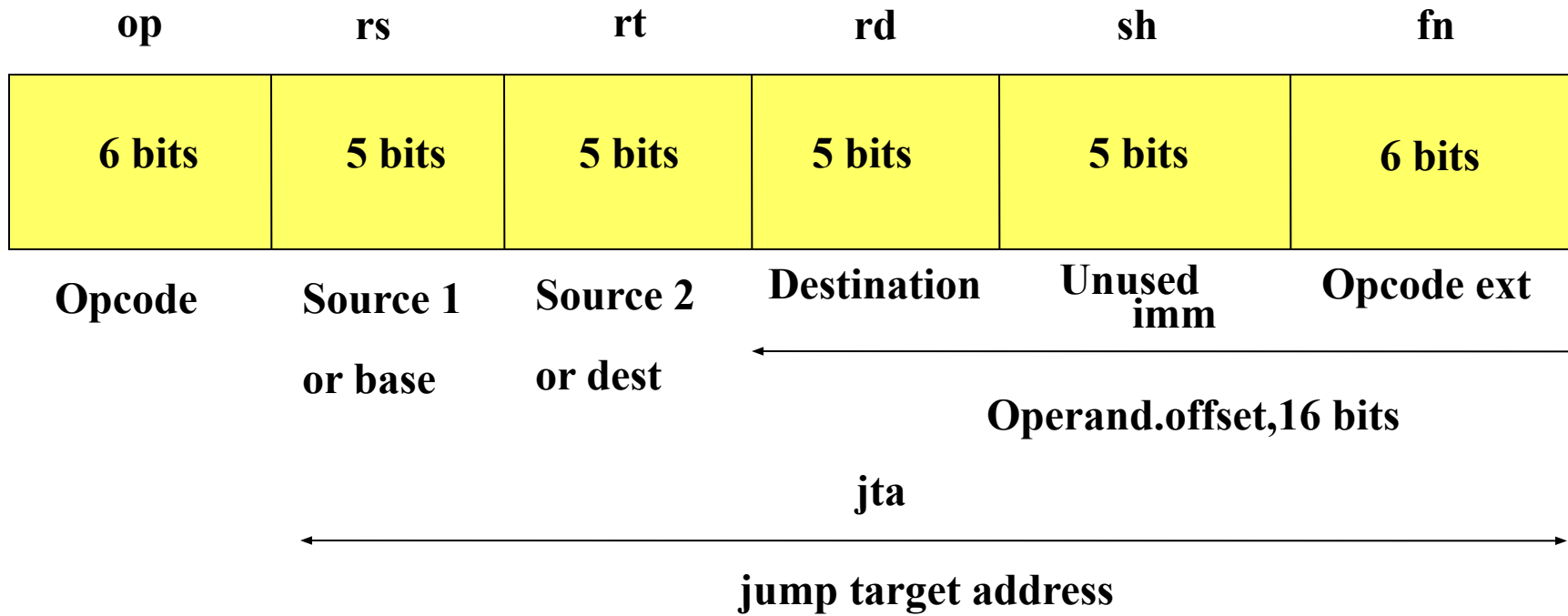
The Micro Mips ISA

Class	Instruction	Usage	Meaning	op	fn
Copy	Load upper immediate	lui rt, imm	$rt \leftarrow (imm, 0x0000)$	15	
Arithmetic	Add	add rd,rs,rt	$rd \leftarrow (rs) + (rt)$	0	32
	Subtract	sub rd,rs,rt	$rd \leftarrow (rs) - (rt)$	0	34
	Set less than	slt rd,rs,rt	$rd \leftarrow \text{if } (rs) < (rt) \text{ then } 1 \text{ else } 0$	0	42
	Add Immediate	addi rt,rs,imm	$rt \leftarrow (rs) + imm$	8	
	Set Less than immediate	slti rt,rs,imm	$rt \leftarrow \text{if } (rs) < imm \text{ then } 1 \text{ else } 0$	10	
Logic	AND	and rd,rs,rt	$rd \leftarrow (rs) \wedge (rt)$	0	36
	OR	or rd,rs,rt	$rd \leftarrow (rs) \vee (rt)$	0	37
	XOR	xor rd,rs,rt	$rd \leftarrow (rs) \oplus (rt)$	0	38
	NOR	nor rd,rs,rt	$rd \leftarrow ((rs) \vee (rt))'$	0	39
	AND immediate	andi rt,rs,imm	$rt \leftarrow (rs) \wedge imm$	12	
	OR immediate	ori rt,rs,imm	$rt \leftarrow (rs) \vee imm$	13	
	XOR immediate	xori rt,rs,imm	$rt \leftarrow (rs) \oplus imm$	14	
Memory Word	Load Word	lw rt,imm(rs)	$rt \leftarrow mem[(rs) + imm]$	35	
	Store Word	sw rt,imm,(rs)	$mem[(rs) + imm] \leftarrow (rt)$	43	
Control transfer	Jump	j L	goto L	2	
	Jump register	jr rs	goto (rs)	0	8
	Branch on less than 0	bltz rs,L	if(rs) < 0 then goto L	1	
	Branch on equal	beq rs,rt,L	if (rs) = (rt) then goto L	4	
	Branch on not equal	bne rs,rt,L	if(rs) $\neq$ (rt) then goto L	5	
	Jump and link	jal L	goto L; 31 $\leftarrow$ (PC)+4	3	
	System call	syscall	Associated with an OS system routine	0	12

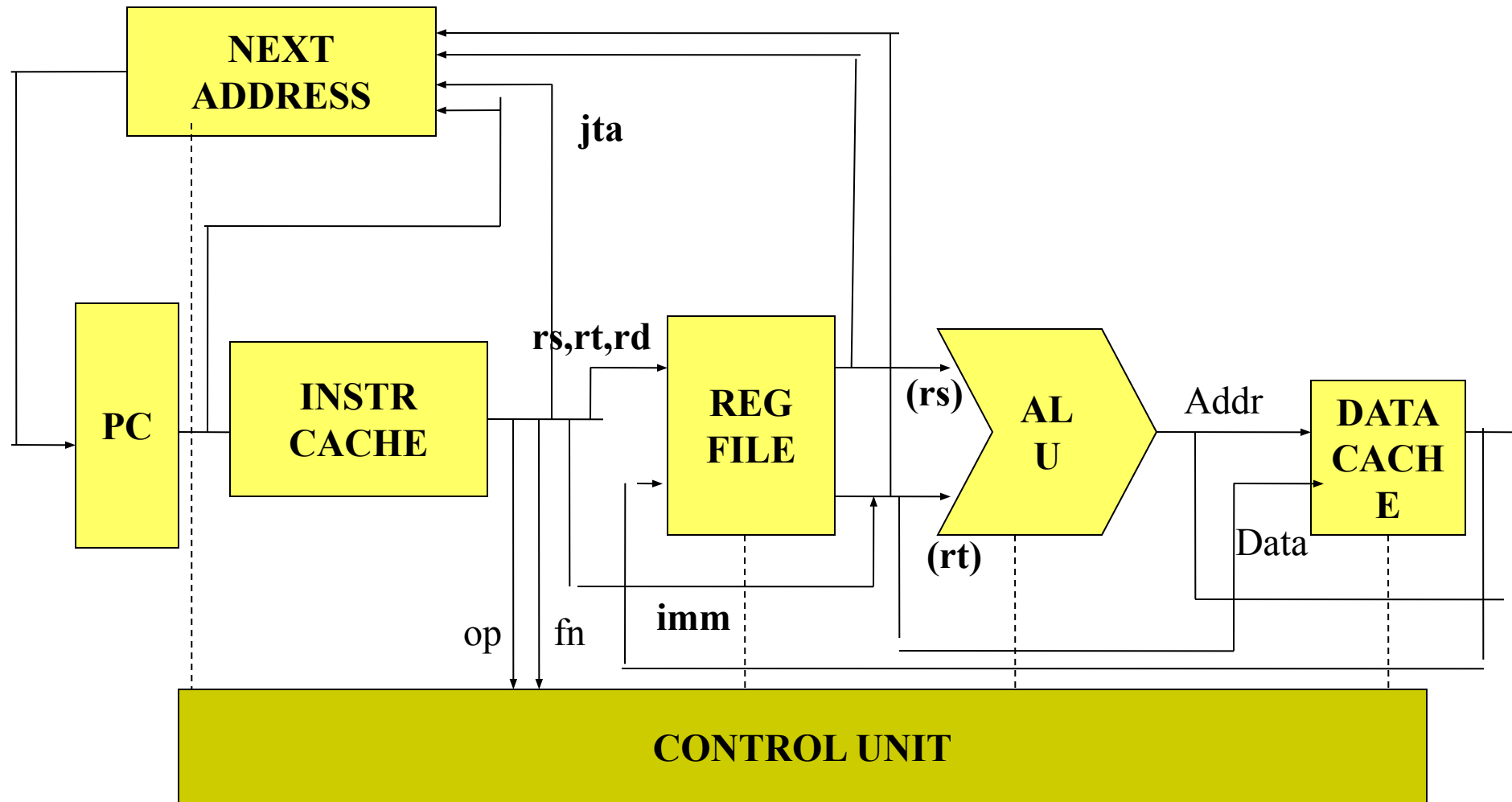
Instruction Execution Unit



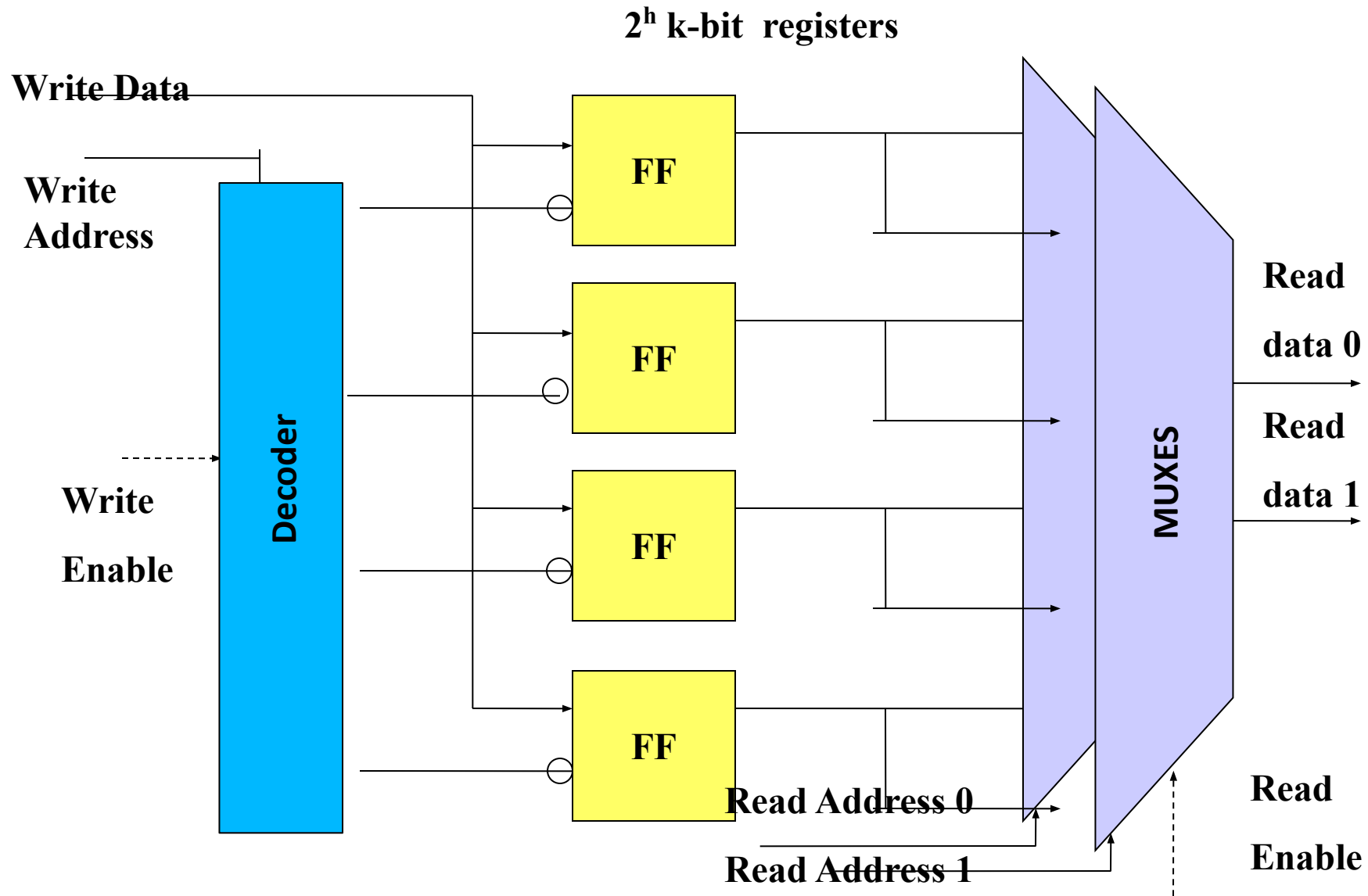
The Instruction Format



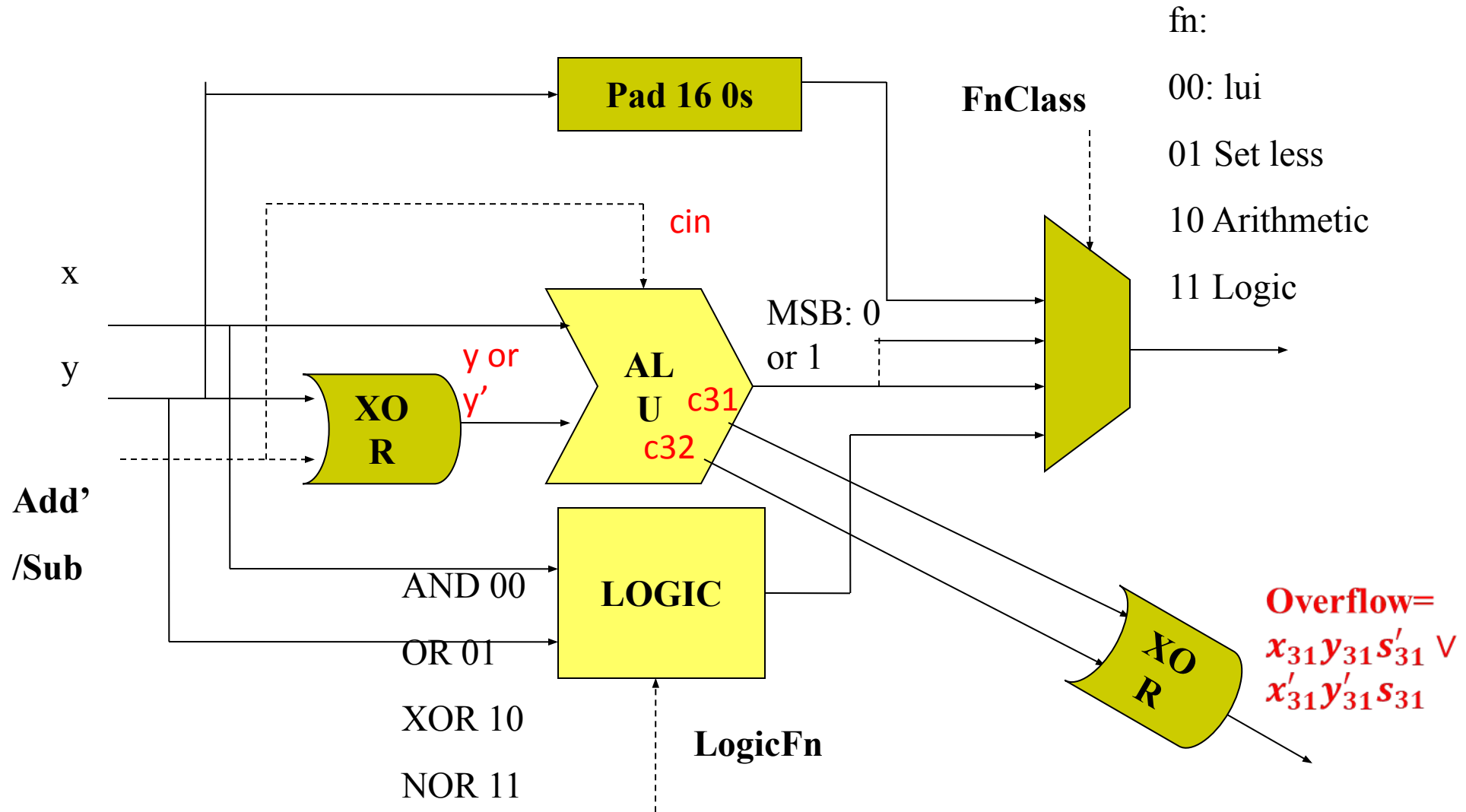
Abstraction of Instruction Execution Unit



Register File



The ALU Architecture



Execution Steps for R-type ALU Instructions

- Read contents from rs and rt, and pass them to the ALU.
- Control the ALU to perform the correct operation, according to the “func”-value of the Instruction.
- Write the output of the ALU in register rd.

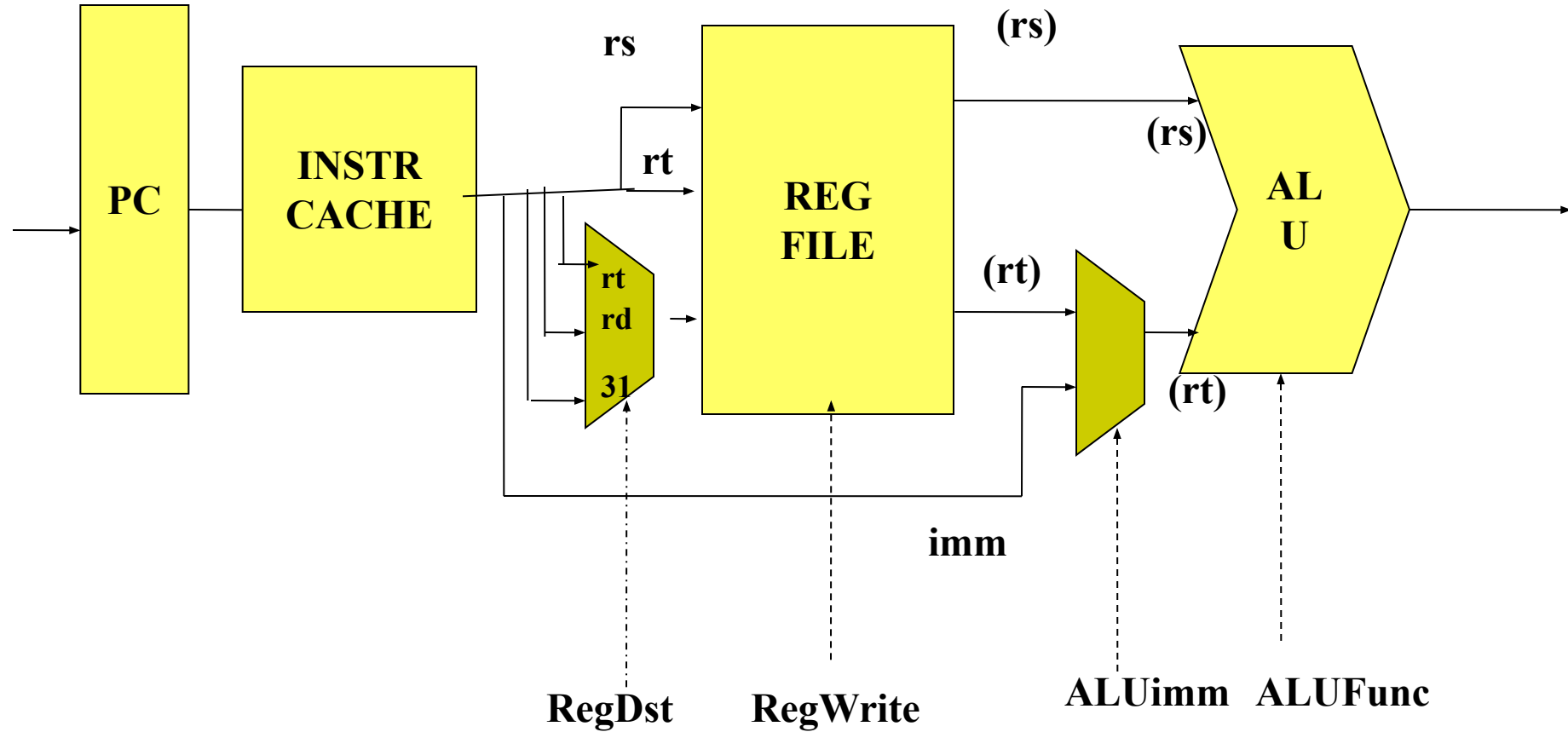
Execution Steps for I-type ALU Instructions

- Contents of rs and immediate value in the instruction are forwarded as inputs to the ALU.
- Control the ALU to perform appropriate function.
- Result is stored in the register rt (rather than rd in case of R-type instructions).

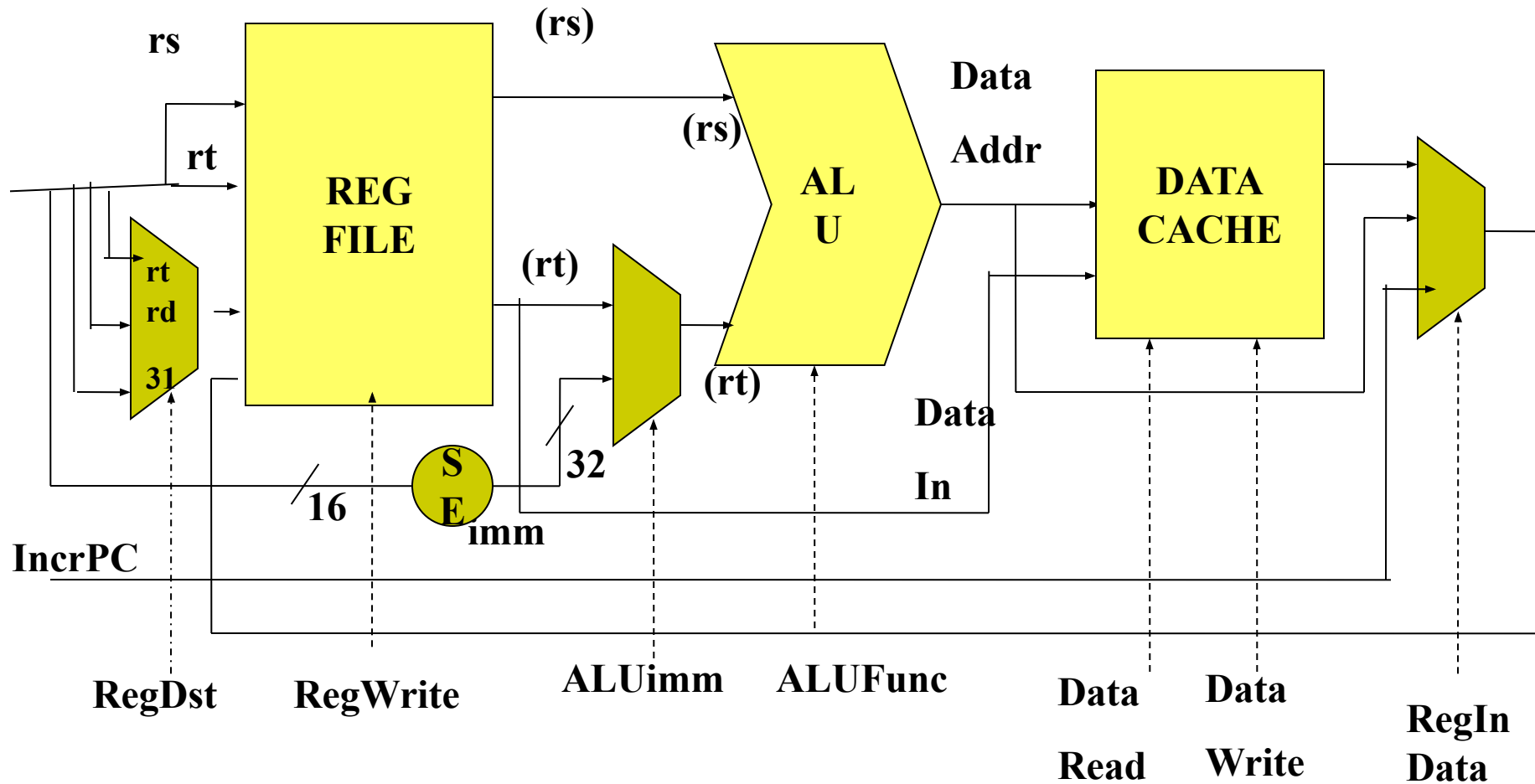
Execution Steps for I-type Memory Instructions

- ❑ Read contents of rs.
- ❑ Add the contents of rs to signed extended immediate value in the instruction to form a memory address.
- ❑ Read from or write to the memory location computed thus.
- ❑ In case of lw, place the result in rt.
- ❑ In case of sw, copy the result from rt.

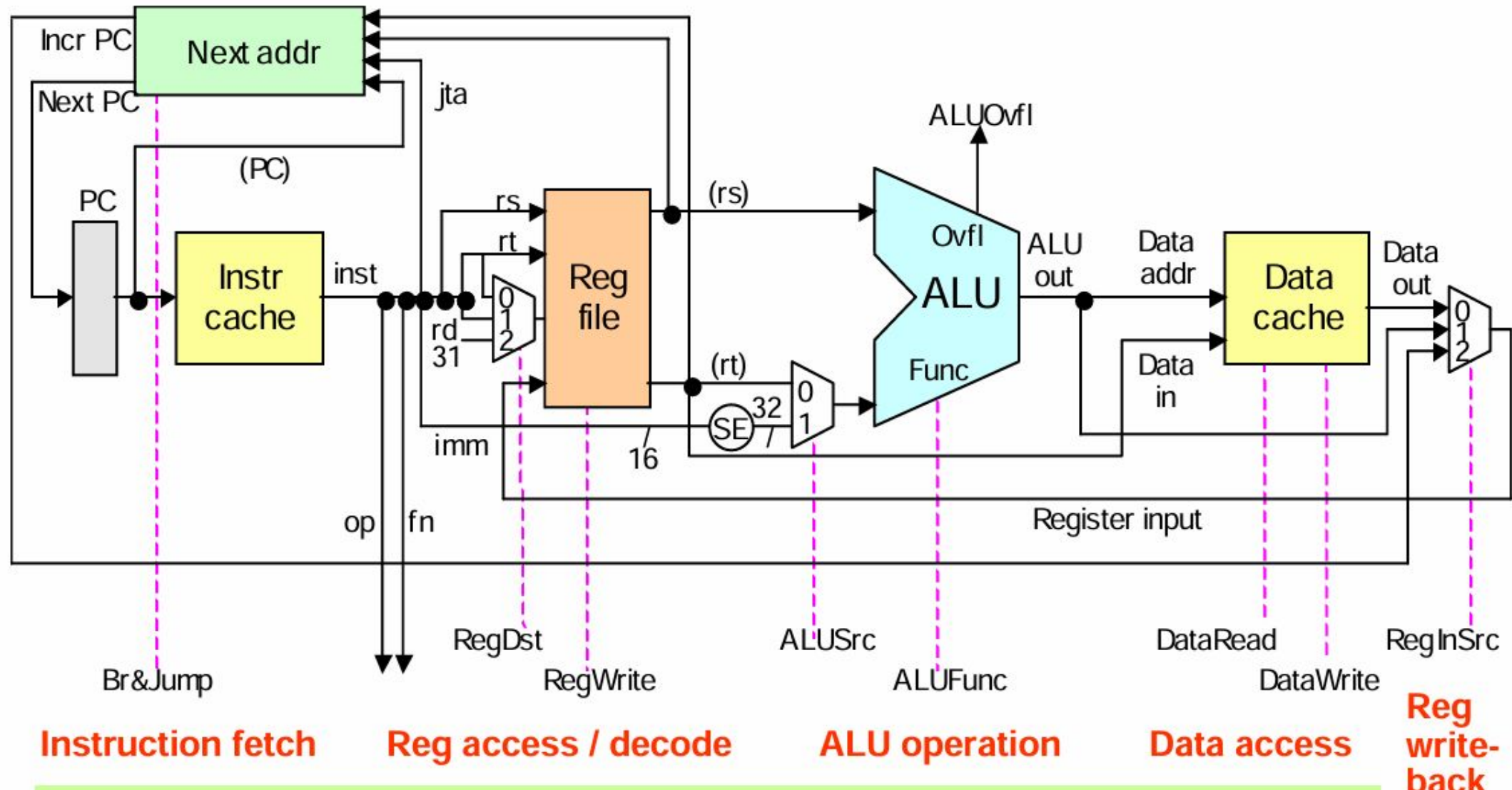
Register Access and ALU



Writing of the ALU output

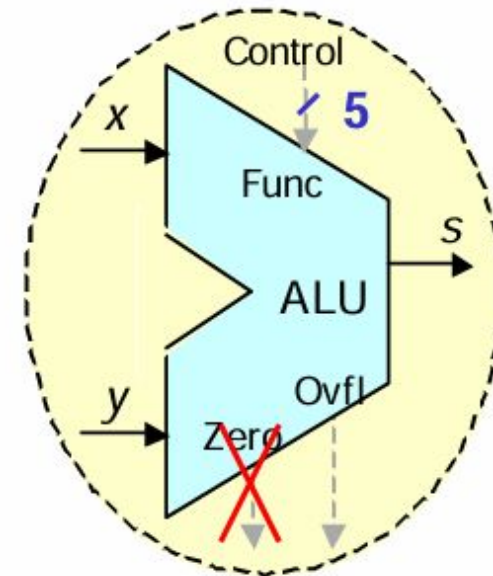
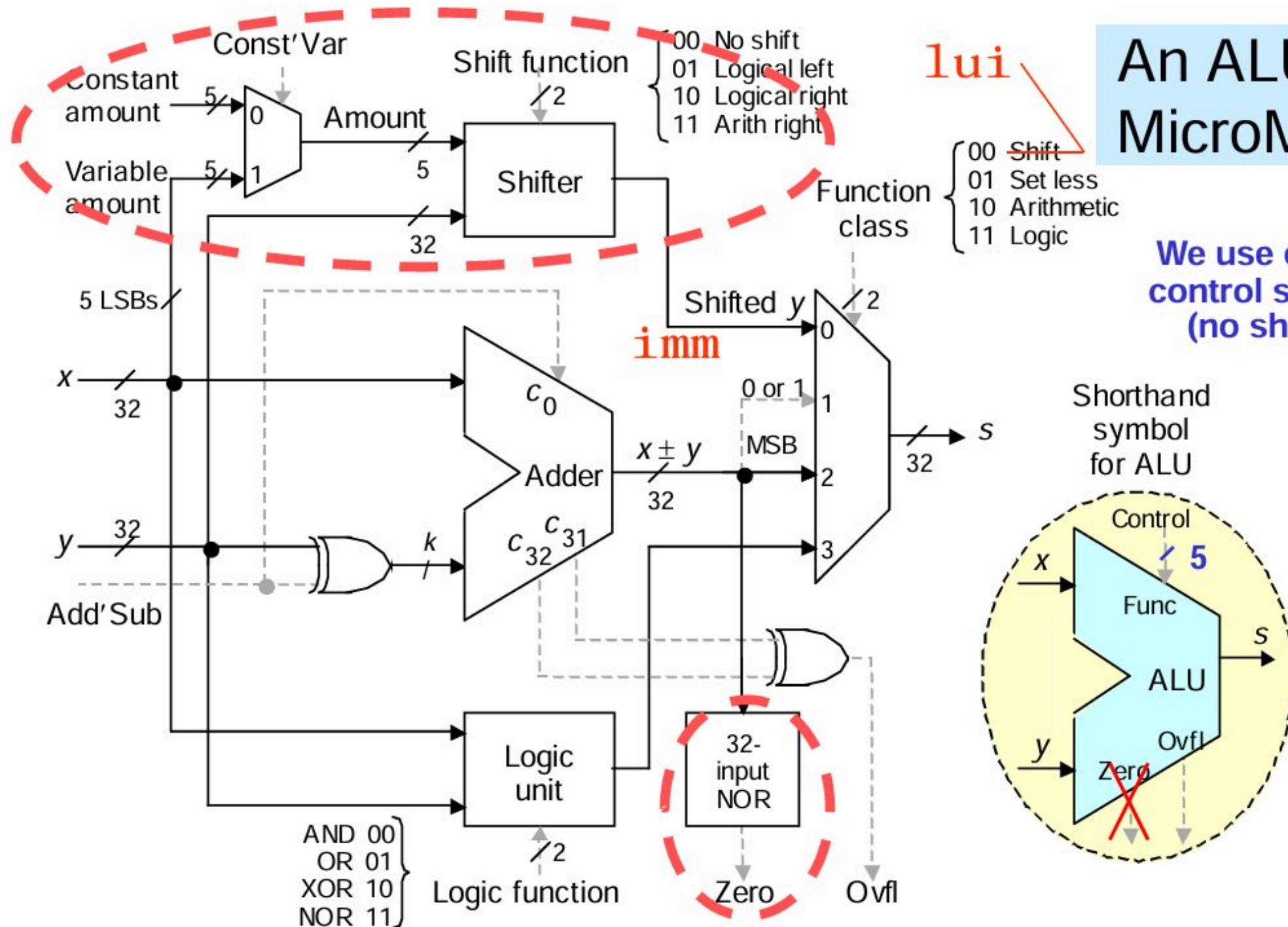


A single cycle data path

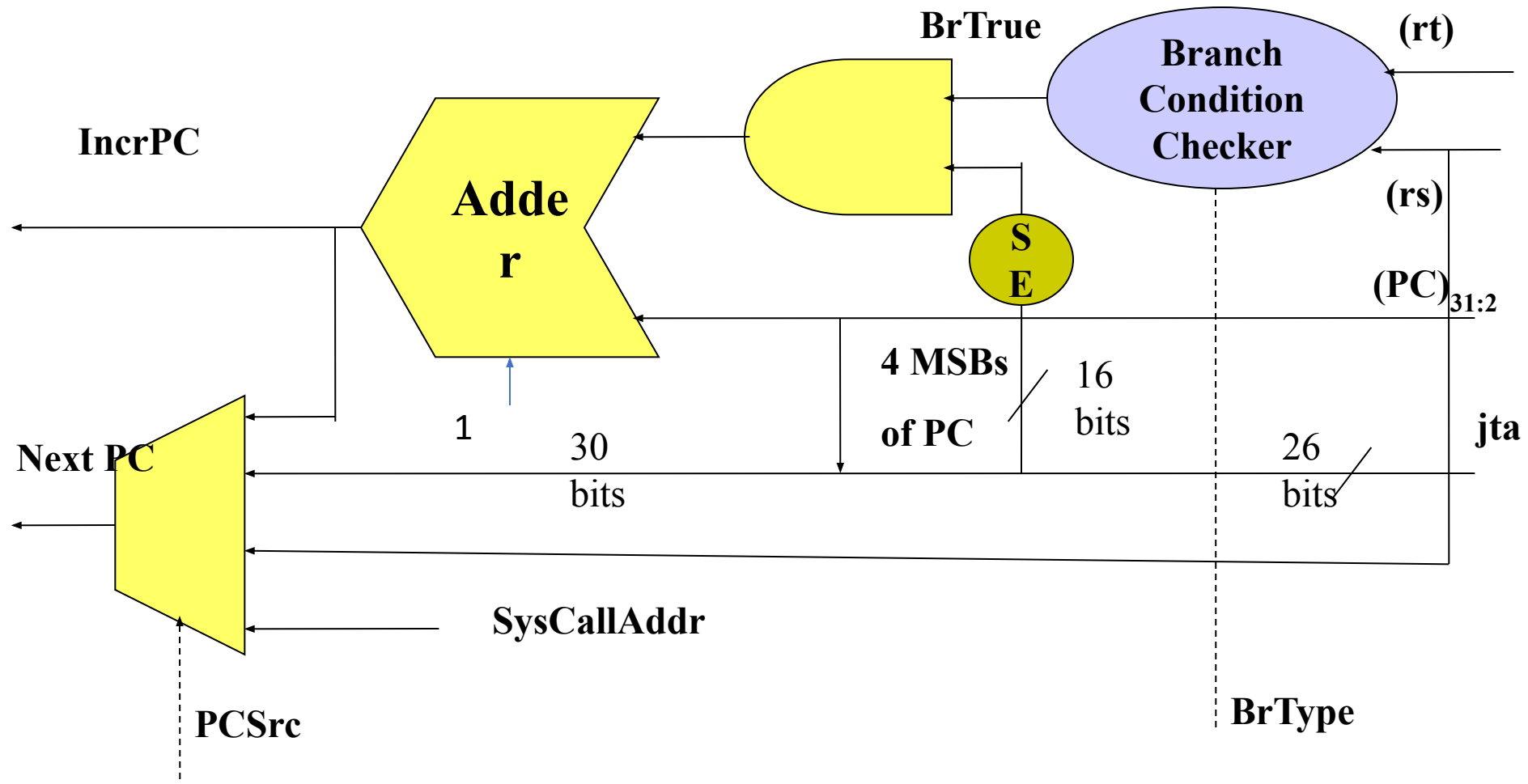


An ALU for MicroMIPS

We use only 5 control signals (no shifts)



The Next Address Architecture



Slightly Modified Branching and jumping (Next Address Logic)

Update
options
for PC

$(PC)_{31:2} + 1$

$(PC)_{31:2} + 1 + \text{imm}$

$(PC)_{31:28} | jta$

$(rs)_{31:2}$

SysCallAddr

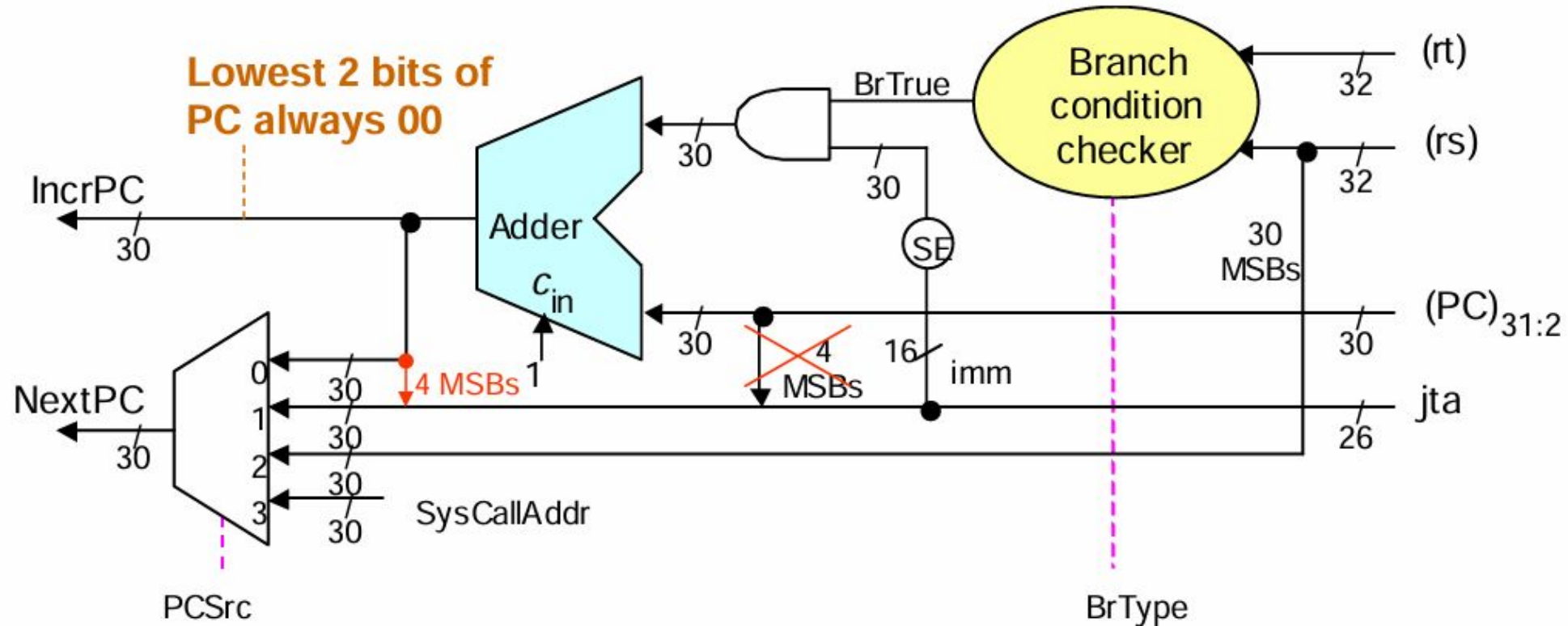
Default option

When instruction is branch and condition is met

When instruction is j or jal

When the instruction is jr

Start address of an operating system routine



Pseudo-Direct Addressing

- Consider a program:

0x1000_0000 LD R1, 0(R2)

0x1000_0100 J Label //Instruction: 0x08000440

0x1000_1000 ...

Label: 0x1000_1100 ...

Why does this work?

Pseudo-Direct Addressing

- Consider a program:

0x1000_0000 LD R1, 0(R2)

0x1000_0100 J Label //Instruction: 0x08000440

0x1000_1000 ...

Label: 0x1000_1100 ...

Why does this work?

Lets move the code to a different memory location

- Consider a program:

0x2000_0000 LD R1, 0(R2)

0x2000_0100 J Label //Instruction: 0x08000440

0x2000_1000 ...

Label: 0x2000_1100 ...

Does this work?

Lets move the code to a different memory location

- Consider a program:

0x2000_0000 LD R1, 0(R2)

0x2000_0100 J Label //Instruction: 0x08000440

0x2000_1000 ...

Label: 0x2000_1100 ...

Does this work?

Partitions a 4GB address space...

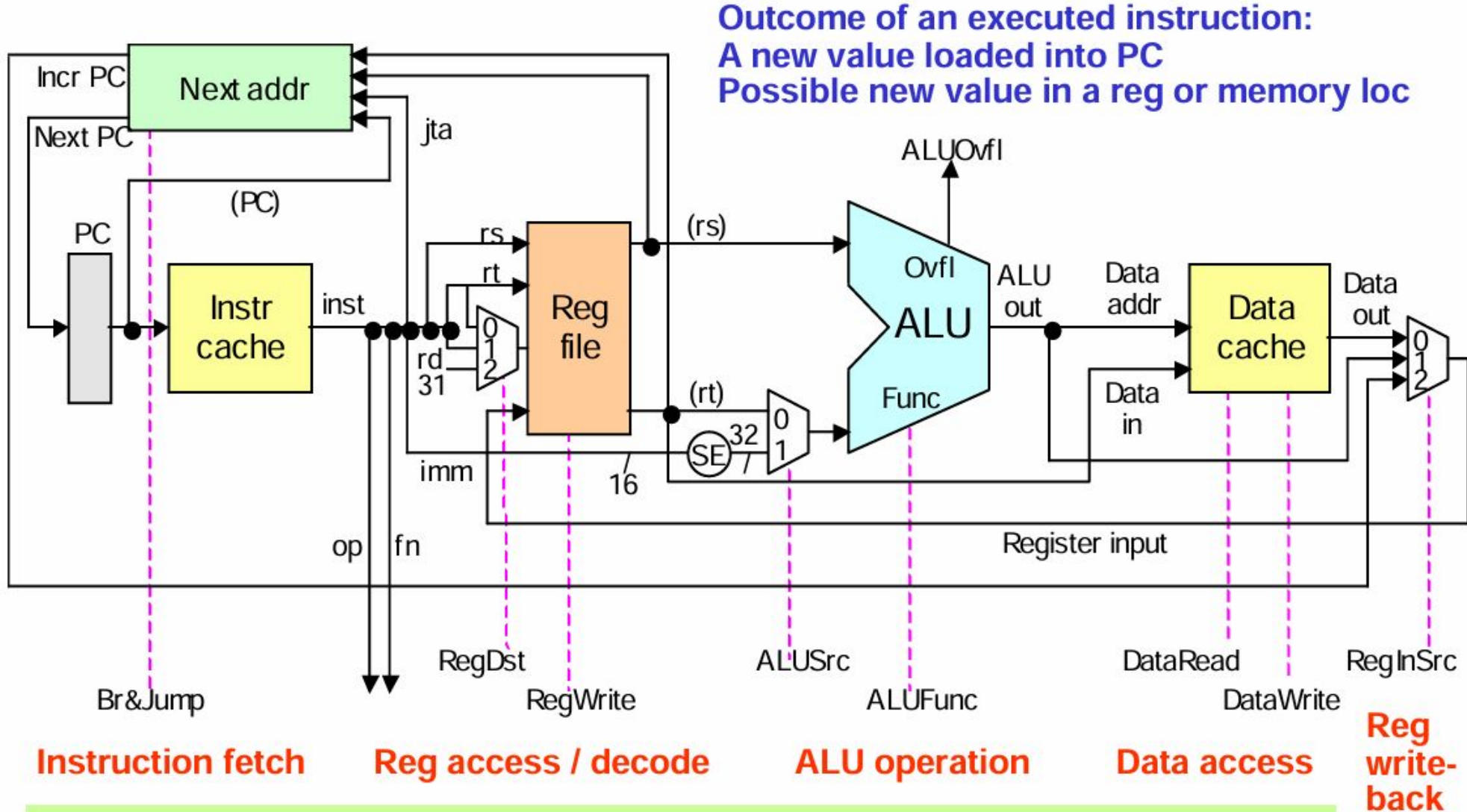
Pseudo-Direct Addressing

- A pseudo-direct jump can reach up to $2^{26} = 67,108,864$ instructions inside the current 256 MB region.
- That is about 67 million instructions maximum per region.

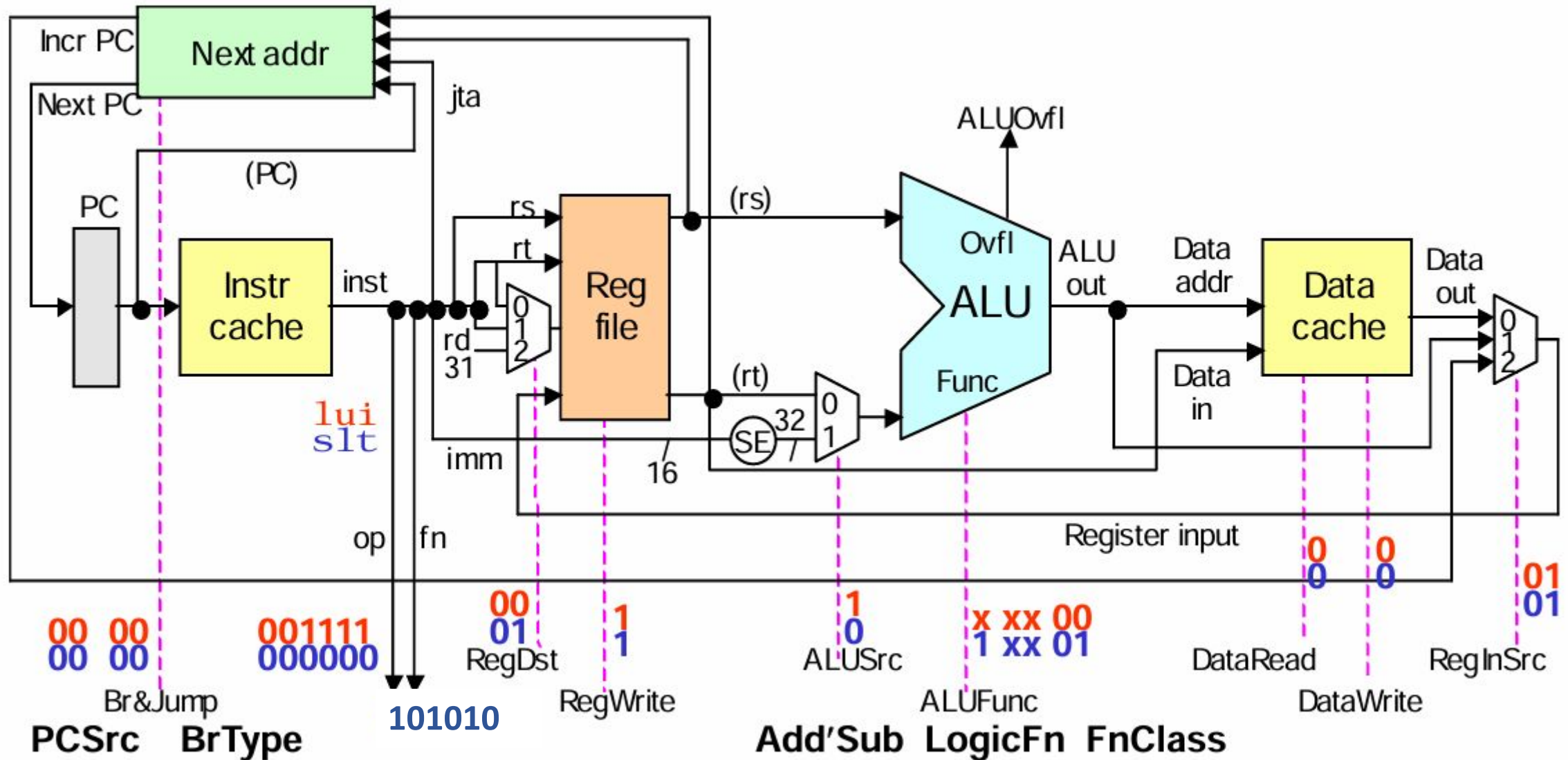
Deriving the Control Signals

Control signal		0	1	2	3
Reg file	RegWrite	Don't write	Write		
	RegDst ₁ , RegDst ₀	rt	rd	\$31	
	RegInSrc ₁ , RegInSrc ₀	Data out	ALU out	IncrPC	
ALU	ALUSrc	(rt)	imm		
	Add'Sub	Add	Subtract		
	LogicFn ₁ , LogicFn ₀	AND	OR	XOR	NOR
	FnClass ₁ , FnClass ₀	lui	Set less	Arithmetic	Logic
Data cache	DataRead	Don't read	Read		
	DataWrite	Don't write	Write		
Next addr	BrType ₁ , BrType ₀	No branch	beq	bne	bltz
	PCSrc ₁ , PCSrc ₀	IncrPC	jta	(rs)	SysCallAddr

Single Cycle Data Path



Control Signals in Single Cycle Data Path



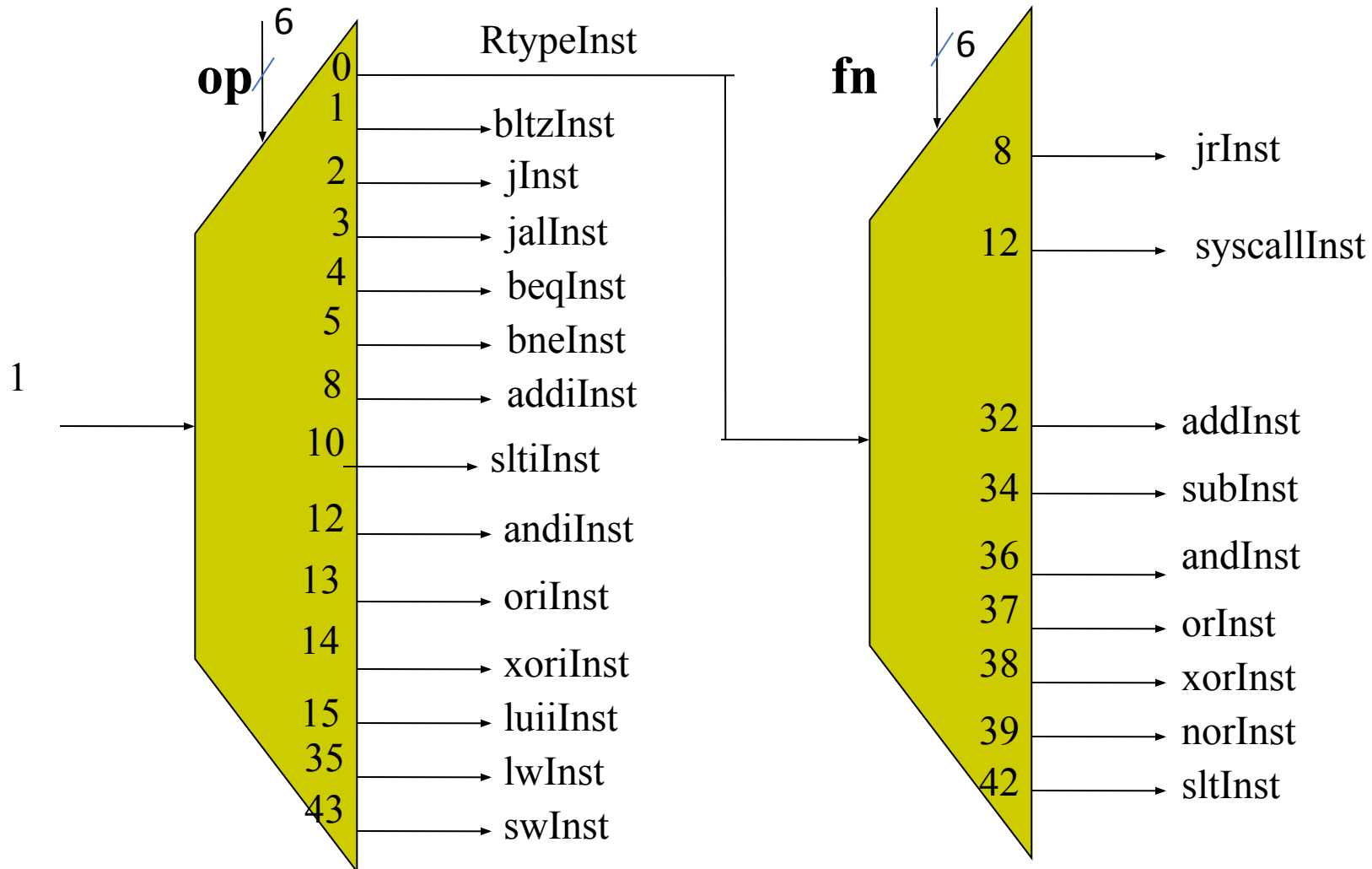
**Control
Signal
Settings:
Repeated
for
Reference**

Instruction	op	fn	RegWrite	RegDst	RegInSrc	ALUSrc	Add' Sub	LogicFn	FnClass	DataRead	DataWrite	BrType	PCSrc
Load upper immediate	001111		1	00	01	1			00	0	0	00	00
Add	000000	100000	1	01	01	0	0		10	0	0	00	00
Subtract	000000	100010	1	01	01	0	1		10	0	0	00	00
Set less than	000000	101010	1	01	01	0	1		01	0	0	00	00
Add immediate	001000		1	00	01	1	0		10	0	0	00	00
Set less than immediate	001010		1	00	01	1	1		01	0	0	00	00
AND	000000	100100	1	01	01	0		00	11	0	0	00	00
OR	000000	100101	1	01	01	0		01	11	0	0	00	00
XOR	000000	100110	1	01	01	0		10	11	0	0	00	00
NOR	000000	100111	1	01	01	0		11	11	0	0	00	00
AND immediate	001100		1	00	01	1		00	11	0	0	00	00
OR immediate	001101		1	00	01	1		01	11	0	0	00	00
XOR immediate	001110		1	00	01	1		10	11	0	0	00	00
Load word	100011		1	00	00	1	0		10	1	0	00	00
Store word	101011		0			1	0		10	0	1	00	00
Jump	000010		0							0	0		01
Jump register	000000	001000	0							0	0		10
Branch on less than 0	000001		0							0	0	11	00
Branch on equal	000100		0							0	0	01	00
Branch on not equal	000101		0							0	0	10	00
Jump and link	000011		1	10	10					0	0	00	01
System call	000000	001100	0							0	0		11

Generating Control Signals

- The control signals are a function of **op** and **fn** fields.
- One can express the control signals as a function of the op and fn bits.
 - It is quite easy
- However, the demerit of such an approach is if the instruction set is modified, or if we add new instructions, the entire control unit needs to be redesigned.

The Decoder based control unit



Expressing the Control Signals

- Auxiliary signals:
 - $\text{arithInst} = \text{addInst} \vee \text{subInst} \vee \text{sltInst} \vee \text{addiInst} \vee \text{sltiInst}$
 - $\text{logicInst} = \text{andInst} \vee \text{orInst} \vee \text{xorInst} \vee \text{norInst} \vee \text{andiInst} \vee \text{oriInst} \vee \text{xoriInst}$
 - $\text{immInst} = \text{luiInst} \vee \text{addiInst} \vee \text{sltiInst} \vee \text{andiInst} \vee \text{oriInst} \vee \text{xoriInst}$
- Some Control Signals:
 - $\text{RegWrite} = \text{luiInst} \vee \text{arithInst} \vee \text{logicInst} \vee \text{lwiInst} \vee \text{jaliInst}$
 - $\text{ALUSrc} = \text{immInst} \vee \text{lwiInst} \vee \text{swiInst}$

Performance of the Single Cycle Architecture

- The above design of control circuit is a stateless and combinational design.
- Each new instruction is read from the PC, and is executed in one single clock.
 - Thus $CPI=1$
- The clock cycle is determined by the longest instruction.

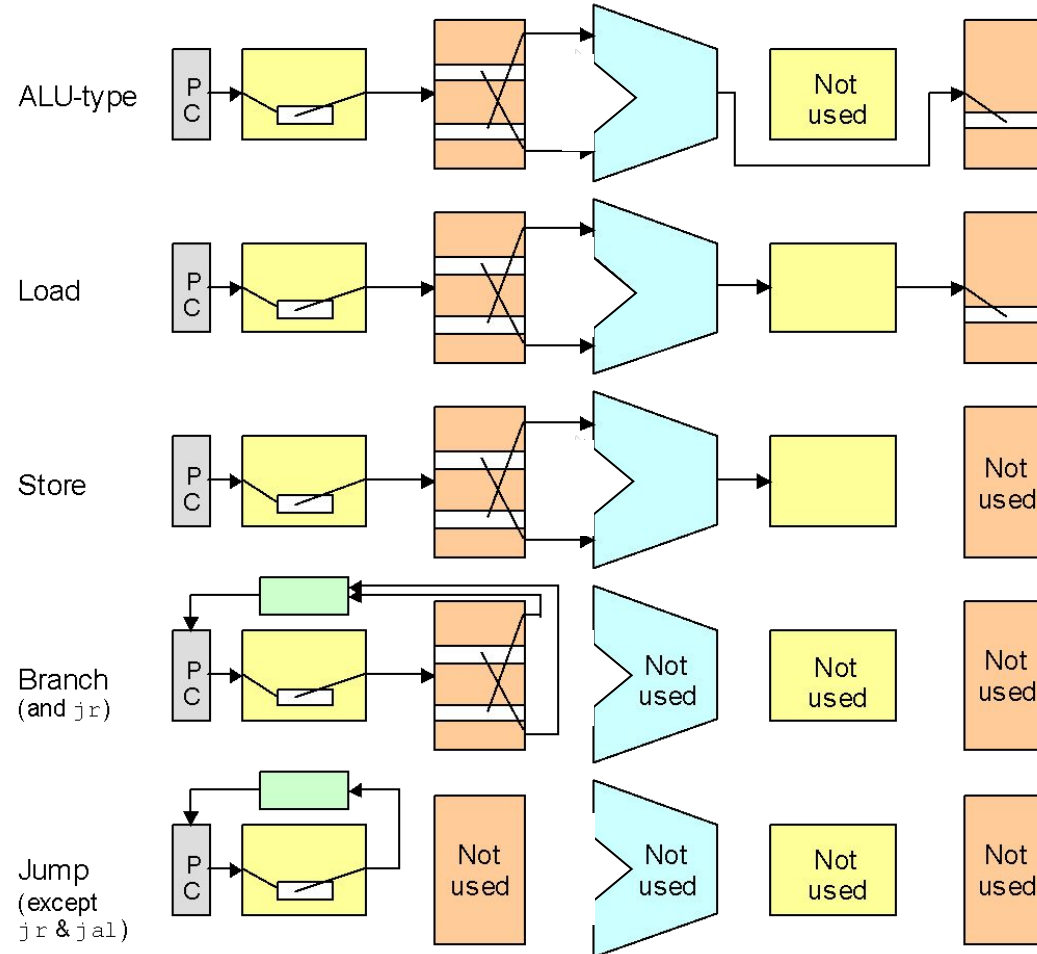
Performance Estimation for Single-Cycle MicroMIPS

Instruction access	2 ns
Register read	1 ns
ALU operation	2 ns
Data cache access	2 ns
Register write	<u>1 ns</u>
Total	8 ns

Single-cycle clock = 125 MHz

R-type	44%	6 ns
Load	24%	8 ns
Store	12%	7 ns
Branch	18%	5 ns
Jump	2%	<u>3 ns</u>

Weighted mean ≈ 6.36 ns



The MicroMIPS data path unfolded (by depicting the register write step as a separate block) so as to better visualize the critical-path latencies.

lw is the longest instruction

□ lw execution includes all the possible steps:

1. Instruction Access: 2 ns
2. Register Read: 1 ns
3. ALU operation: 2 ns
4. Data Cache Access: 2 ns
5. Register Write-back: 1 ns

Total: 8 ns

Thus a clock frequency of 125 MHz suffices.

So, for 1 instruction, $(1/125) \times 10^{-6}$ sec

Thus, 125 Million Instructions are executed per second (125 MIPS)

Obtaining better performance

- Note that the average instruction time is less, depends on the type of instruction, and their percentages in an application.
 - Rtype 44% 6 ns No data cache
 - Load 24% 8 ns
 - Store 12% 7ns No register write-back
 - Branch 18% 5ns Fetch+Register Read+Next-addr formation
 - Jump 2% 3ns Fetch + Instruction Decode
- Weighted average = 6.36 ns
- So, with a variable cycle time implementation, the performance is 157 MIPS
- However, this is not possible.

We see that a single cycle implementation has a poor performance.

Summary

- Clock cycle is determined by the slowest instruction.
- If the MIPS ISA includes more complex instructions, the disadvantage is more.
 - For example if we add a MULT/DIV instruction by k times, then all operations need to be slowed down by k times.
 - Thus MIPS does the MULT/DIV instruction to a separate block (than the ALU block), with separate registers Hi and Lo.
 - sufficient time is kept to write back the results to the register file